

9.7.16. TensorCore 5th Generation Family Instructions

9.7.16.1 Tensor Memory

The 5th generation TensorCore has dedicated on-chip memory that is specialized for use by TensorCore operations. This Tensor Memory is organized as a two-dimensional matrix where the horizontal rows are called lanes and the vertical columns are called columns.

On architecture sm_100a/sm_100f, the 5th generation TensorCore's Tensor Memory has a two-dimensional structure of 512 columns and 128 rows per CTA, with each cell being 32-bits in size.

Restrictions on threads accessing the Tensor Memory via the load and store operations are specified in [Access restrictions](#).

9.7.16.1.1 Tensor Memory Addressing

Tensor Memory addresses are 32-bit wide and specify two components.

1. Lane index
2. Column index

The layout is as follows:

31 16	15 0
Lane index	Column index

Figure 182 shows the view of the Tensor Memory Layout within CTA.

9.7.16.1.2 Tensor Memory Allocation

The Tensor Memory is dynamically allocated. The Tensor Memory must be allocated by a single warp in a CTA using the [Tensor Memory Allocation and Management Instructions](#).

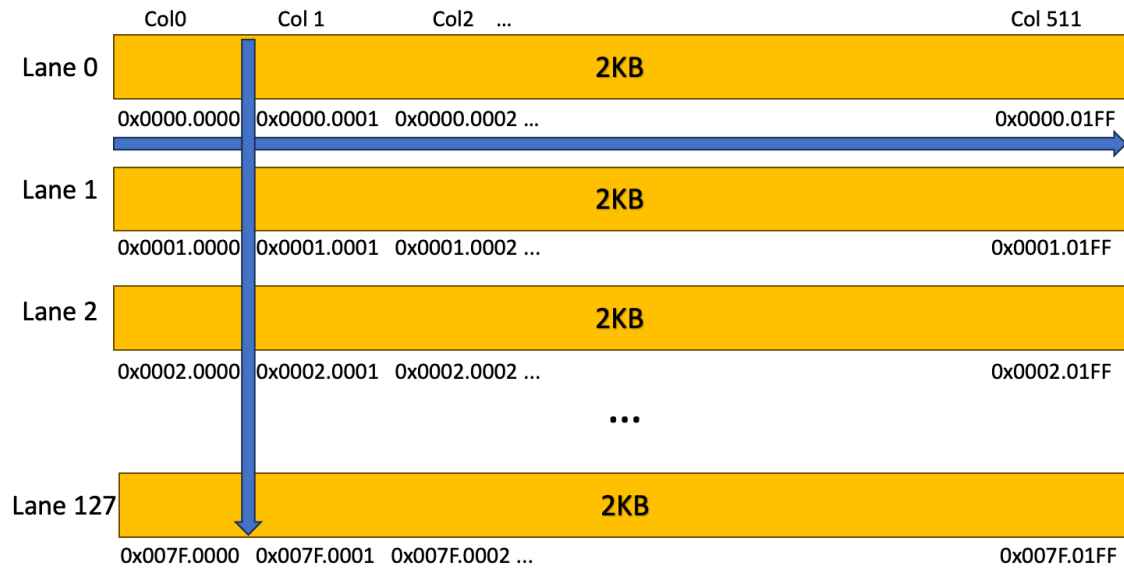
The allocation and deallocation of [Tensor Memory](#) is performed in terms of columns. The unit of allocation is 32 columns and the number of columns being allocated must be a power of 2. When a column is allocated, all 128 lanes of the column are allocated.

All of the Tensor Memory that was allocated in a kernel, must be explicitly deallocated before the kernel exits.

9.7.16.2 Matrix and Data Movement Shape

There are two kinds of shapes involved.

1. Shapes in the data movement operations
2. Shapes in the MMA operations



Tensor Memory Layout and Addressing

Figure 182: Tensor Memory Layout and Addressing

9.7.16.2.1 Matrix Shape

The matrix multiply and accumulate operations support a limited set of shapes for the operand matrices A, B and D. The shapes of all three matrix operands are collectively described by the tuple $M \times N \times K$ where A is $M \times K$ matrix, B is a $K \times N$ matrix, and D is a $M \times N$ matrix.

Table 41 shows matrix shapes that are supported for the specified types for the `tcgen05.mma` operation.

Table 41: Various combinations of .kind and shapes

Various Combinations						Shapes Supported		
.kind::*	Has .ws	CTA Group	Spar-sity	dtype	atype/btype			
kind::f16	No .ws	1	Dense	.f16	.f16	64xNxK 128xNxK	N = {8, 16, 24, ... 256} steps of 8	K = 16
				.f32	.f16, .bf16			
			Sparse	.f16	.f16			K = 32
				.f32	.f16, .bf16			
		2	Dense	.f16	.f16	128xNxK 256xNxK	N = {16, 32, ... 256} steps of 16	K = 16
				.f16	.f16			

continues on next page

Table 41 – continued from previous page

Various Combinations						Shapes Supported		
.kind:: *	Has .ws	CTA Group	Spar- sity	dtype	atype/btype			
				.f32	.f16, .bf16			K = 32
			Sparse	.f16	.f16			
				.f32	.f16, .bf16			
	.ws	1	Dense	.f16	.f16	32xNxK 64xNxK 128xNxK	N = {64, 128, 256}	K = 16
				.f32	.f16, .bf16			
			Sparse	.f16	.f16		N = {64, 128}	K = 32
				.f32	.f16, .bf16			
		2	Either	.f16	.f16	Invalid		
				.f32	.f16, .bf16			
	.kind:: tf32	No .ws	1	Dense	.f32	.tf32	64xNxK 128xNxK	N = {8, 16, 24, ... 256} steps of 8
Sparse				K = 16				
2			Dense	128xNxK 256xNxK			N = {16, 32, ... 256} steps of 16	K = 8
			Sparse					K = 16
.ws		1	Dense	32xNxK 64xNxK 128xNxK			N = {64, 128, 256}	K = 8
			Sparse					N = {64, 128}
		2	Dense	Invalid				
			Sparse					
.kind:: f8f6f4	No .ws	1	Dense	.f32 .f16	.e4m3, .e5m2, .e2m3, .e3m2, .e2m1	64xNxK 128xNxK	N = {8, 16, ... 256} steps of 8	K = 32
			Sparse					K = 64
		2	Dense			128xNxK 256xNxK	N = {16, 32, ... 256} steps of 16	K = 32

continues on next page

Table 41 – continued from previous page

Various Combinations						Shapes Supported						
.kind::*	Has .ws	CTA Group	Spar-sity	dtype	atype/btype							
		1	Sparse			32xNxK 64xNxK 128xNxK	N = {64, 128, 256}	K = 64				
			Dense					K = 32				
			Sparse				N = {64, 128}	K = 64				
		2	Dense			Invalid						
			Sparse									
.kind::mxf8f6f4	No .ws	1	Dense	.f32	.e4m3, .e5m2, .e2m3, .e3m2, .e2m1 X (Scale) .ue8m0	128xNxK	N = {8, 16, ... 256} steps of 8	K = 32				
			Sparse					K = 64				
		2	Dense			128xNxK 256xNxK		N = {16, 32, ... 256} steps of 16	K = 32			
			Sparse						K = 64			
		.ws	1			Dense	Invalid					
						Sparse						
			2			Dense						
						Sparse						
	.kind::i8	No .ws	1			Dense	.s32	.s8, .u8	64xNxK 128xNxK	N = {8, 16, 24, 32, 48, ... 256} steps of 16 after N > 32	K = 32	
						Sparse					K = 64	
			2			Dense			128xNxK 256xNxK		N = {32, 64, ... 256} steps of 32	K = 32
						Sparse						K = 64
.ws			1	Dense	32xNxK 64xNxK 128xNxK	N = {64, 128, 256}			K = 32			
				Sparse					K = 64			
			2	Dense		Invalid						
				Sparse								

continues on next page

Table 41 – continued from previous page

Various Combinations						Shapes Supported			
.kind::*	Has .ws	CTA Group	Spar-sity	dtype	atype/btype				
.kind::mxf4	No .ws	1	Dense	.f32	.e2m1X (Scale) .ue8m0	128xNxK	N = {8, 16, ... 256} steps of 8	K = 64	
			Sparse					K = 128	
		2	Dense			128xNxK 256xNxK 256xNxK ¹	N = {16, 32, ... 256} steps of 16	K = 64 K ¹ = 96	
			Sparse					256xNxK	K = 128
	.ws	1 / 2	Either			Invalid			
.kind::mxf4nvf4	No .ws	1	Dense	.f32	.e2m1X (Scale) .ue8m0, .ue4m3	128xNxK	N = {8, 16, ... 256} steps of 8	K = 64	
			Sparse					K = 128	
		2	Dense			128xNxK 256xNxK 256xNxK ¹	N = {16, 32, ... 256} steps of 16	K = 64 K ¹ = 96	
			Sparse					256xNxK	K = 128
	.ws	1 / 2	Either			Invalid			

9.7.16.2.1.1 Target ISA Note

- K = 96 is only supported for target architecture sm_103a.

9.7.16.2.2 Specifying Matrix Shape

M and N can be specified in the *Instruction descriptor*.

K cannot be explicitly specified but is implicitly determined by the MMA-kind and the sparsity, as shown in the [Table 41](#).

9.7.16.2.3 Data Movement Shape

The data movement shape indicates the dimension of the data to be moved to or from the *Tensor Memory*. These shapes are described as a tuple `lane x size` where:

- ▶ `lane` indicates the number of rows in the *Tensor Memory*; and
- ▶ `size` indicates the amount of data, in units of bits (b), across the columns in the *Tensor Memory*.

The following shapes are supported by various `tcgen05` operations:

Shape	<code>tcgen05.<op></code>
<code>.16x64b</code> , <code>.16x128b</code> , <code>.16x256b</code> , <code>.16x32bx2</code> , <code>.32x32b</code>	<code>.ld / .st</code>
<code>.4x256b</code> , <code>.32x128b</code> , <code>.64x128b</code> , <code>.128x256b</code> , <code>.128x128b</code>	<code>.cp</code>
<code>.31x256b</code> (implicit)	<code>.shift</code>

9.7.16.2.3.1 Memory Layout

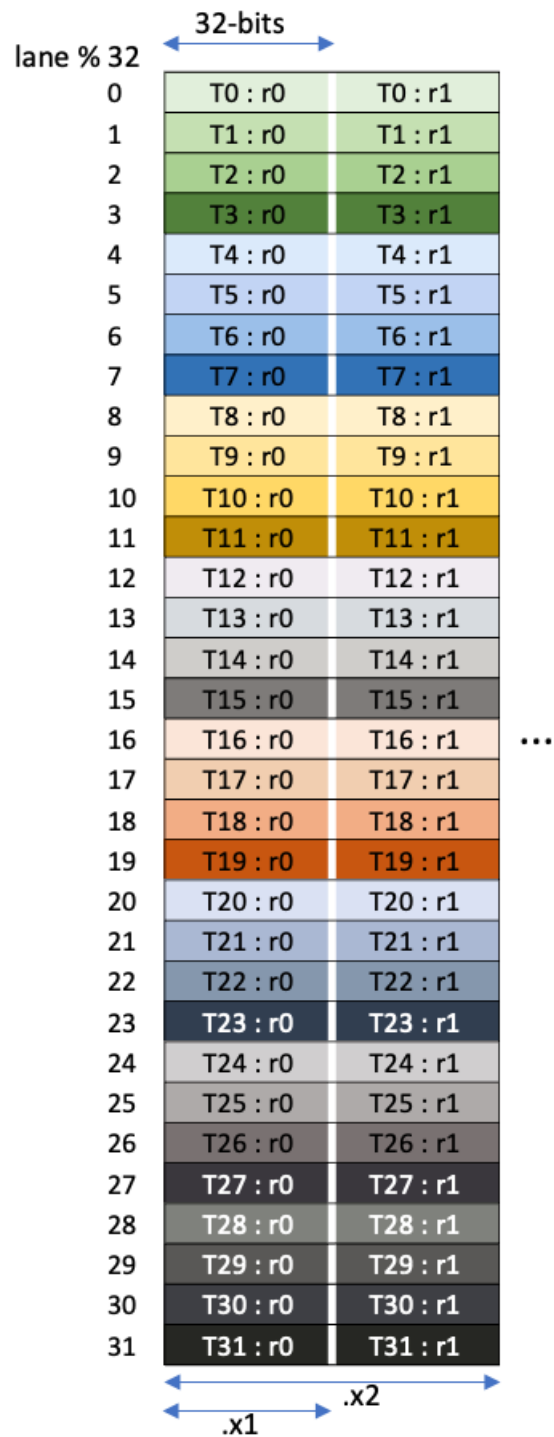
The following shows the layout of the matrix fragments across threads of the warp.

9.7.16.2.3.2 Matrix fragments for shape `.32x32b`

A `tcgen05{.ld, .st}.32x32b` instruction has the following data vector register.

Fragment	Elements (low to high)
A vector expression containing <code>.num</code> number of <code>.b32</code> registers as mentioned in the Table 49 .	<code>r0</code> , <code>r1</code> , ...

A warp executing `tcgen05{.ld, .st}.32x32b` will access 32 lanes of the Tensor Memory. It loads from or stores to each of the lane (`32 * .num`)-bits of data as shown in [Figure 183](#).

Figure 183: Matrix Fragment for shape `.32x32b`

9.7.16.2.3.3 Matrix fragments for shape .16x64b

A `tcgen05{.ld, .st}.16x64b` instruction has the following data vector register.

Fragment	Elements (low to high)
A vector expression containing <code>.num</code> number of <code>.b32</code> registers as mentioned in the Table 49 .	<code>r0, r1, ...</code>

A warp executing `tcgen05{.ld, .st}.16x64b` will access 16 lanes of the Tensor Memory. It loads from or stores to each of the lane $(64 * .num)$ -bits of data as shown in [Figure 184](#).

9.7.16.2.3.4 Matrix fragments for shape .16x128b

A `tcgen05{.ld, .st}.16x128b` instruction has the following data vector register.

Fragment	Elements (low to high)
A vector expression containing <code>.num</code> number of <code>.b32</code> registers as mentioned in the Table 49 .	<code>r0, r1, ...</code>

A warp executing `tcgen05{.ld, .st}.16x128b` will access 16 lanes of the Tensor Memory. It loads from or stores to each of the lane $(128 * .num)$ -bits of data as shown in [Figure 185](#).

9.7.16.2.3.5 Matrix fragments for shape .16x256b

A `tcgen05{.ld, .st}.16x256b` instruction has the following data vector register.

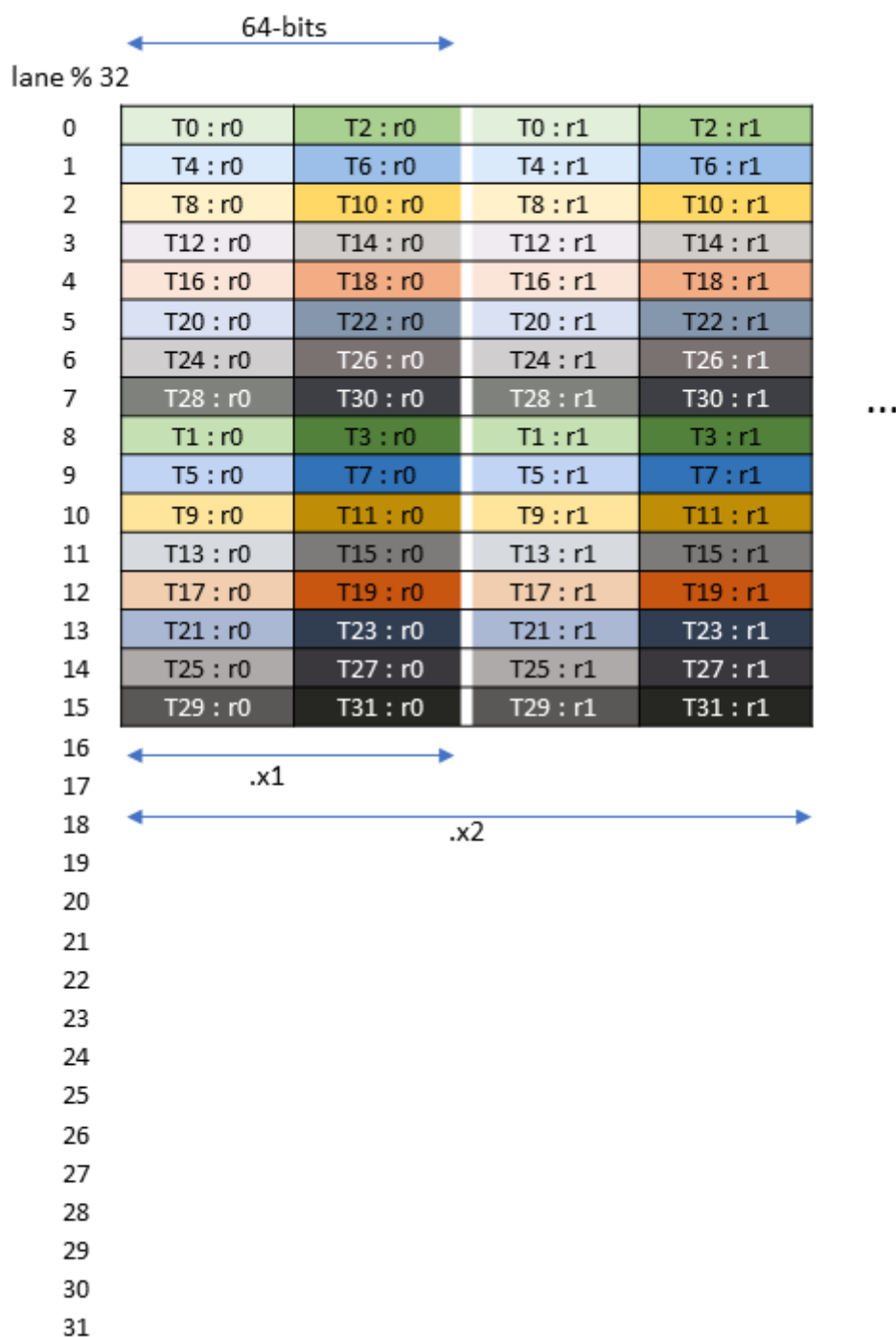
Fragment	Elements (low to high)
A vector expression containing <code>.num</code> number of <code>.b32</code> registers as mentioned in the Table 49 .	<code>r0, r1, r2, r3, ...</code>

A warp executing `tcgen05{.ld, .st}.16x256b` will access 16 lanes of the Tensor Memory. It loads from or stores to each of the lane $(256 * .num)$ -bits of data as shown in [Figure 186](#).

9.7.16.2.3.6 Matrix fragments for shape .16x32bx2

A `tcgen05{.ld, .st}.16x32bx2` instruction has the following data vector register.

Fragment	Elements (low to high)
A vector expression containing <code>.num</code> number of <code>.b32</code> registers as mentioned in the Table 49 .	<code>r0, r1, ...</code>

Figure 184: Matrix Fragment for shape `.16x64b`

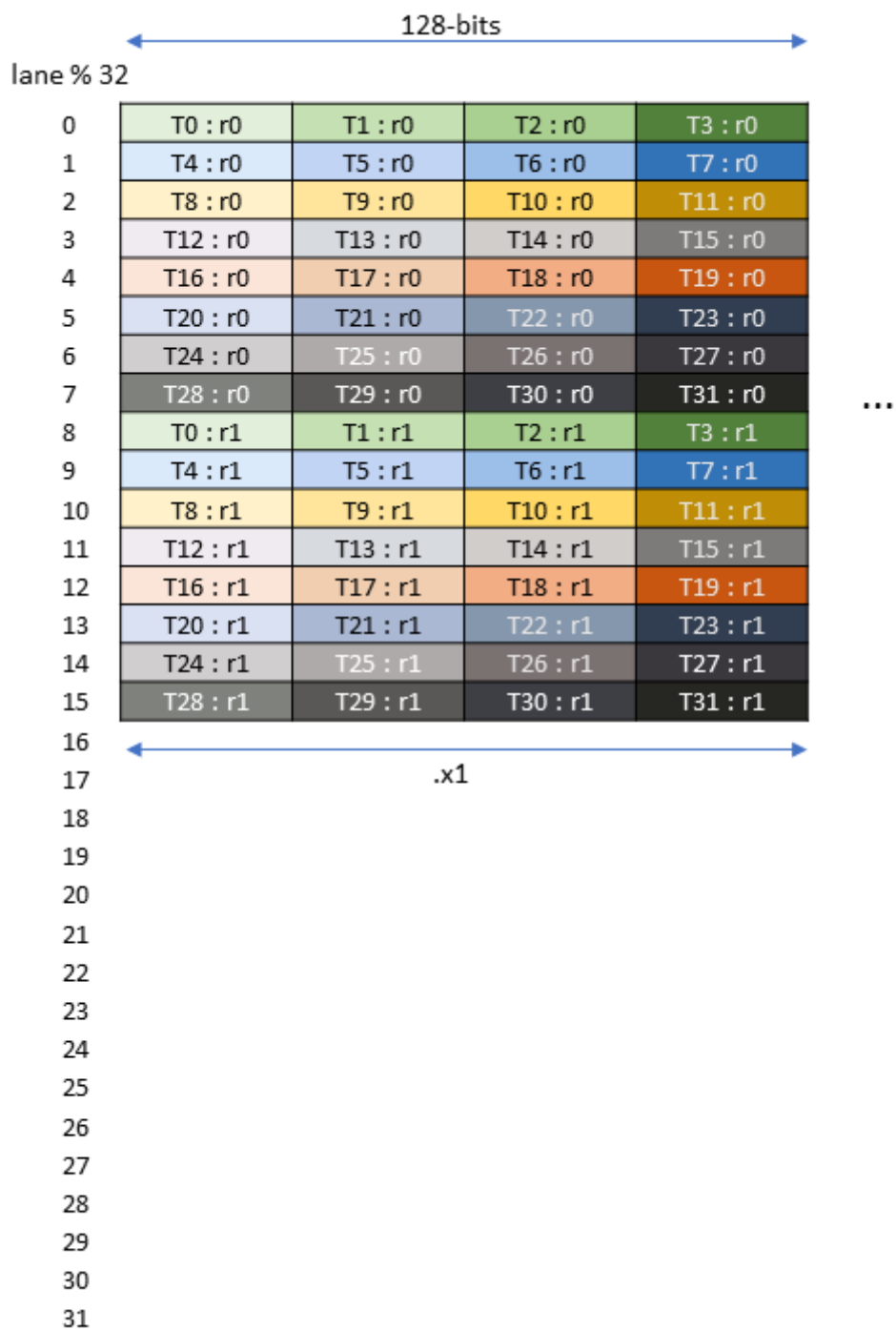
Figure 185: Matrix Fragment for shape `.16x128b`

Figure 186: Matrix Fragment for shape .16x256b

A warp executing `tcgen05{.ld, .st}.16x32bx2` will access 16 lanes of the Tensor Memory. It loads from or stores to each of the lane $(32 * .num)$ -bits of data as shown in [Figure 187](#).

9.7.16.3 Major-ness supported by Strides

There are two strides involved while accessing a matrix from shared memory:

1. Leading dimension stride (byte offset or absolute address)
2. Stride dimension byte offset

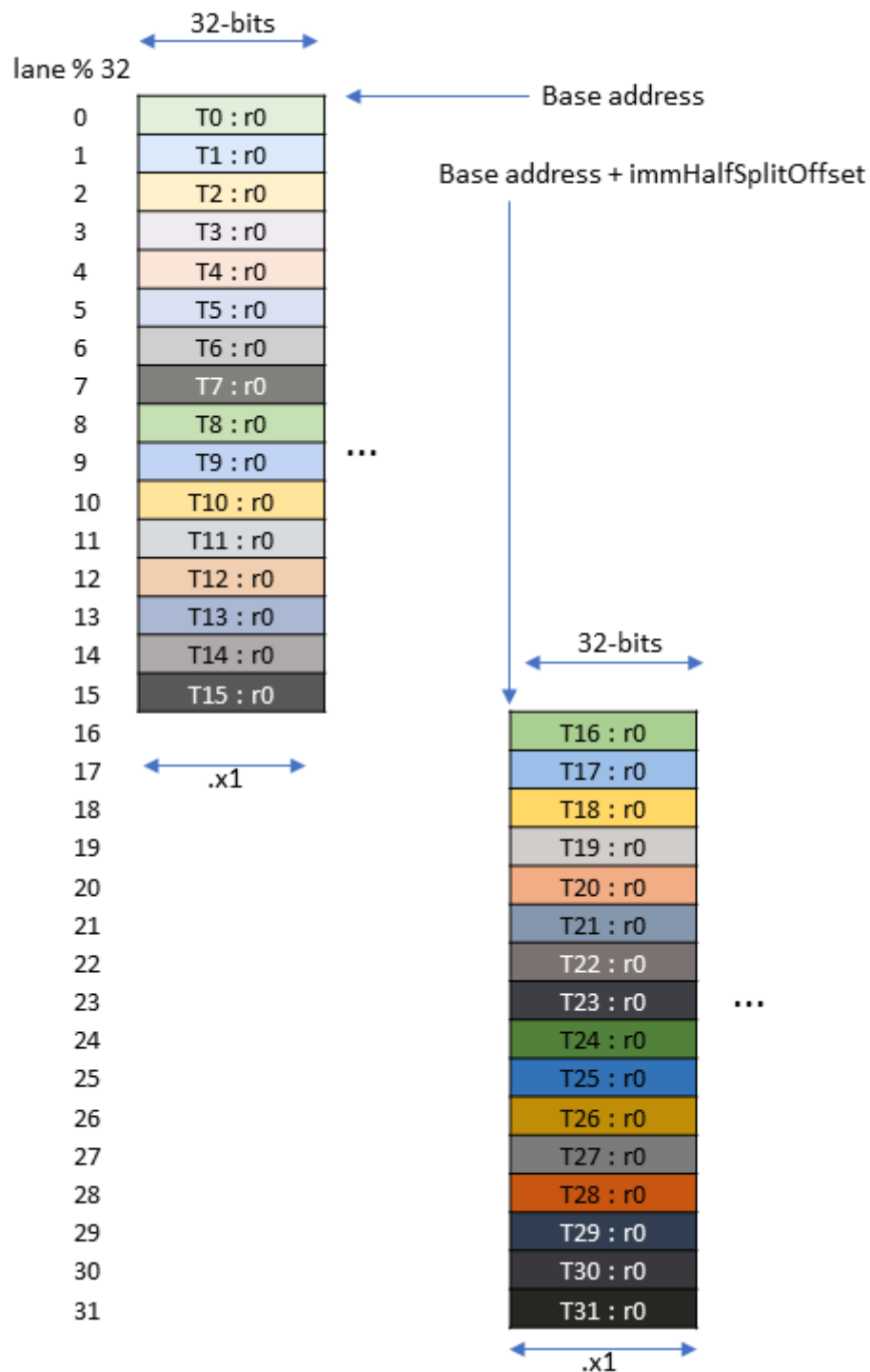
9.7.16.3.1 Leading Dimension Stride: relative offset or absolute address

There are two modes of Leading Dimension Strides as described below. Bit #52 in the *Shared memory descriptor* is used to distinguish between two modes.

9.7.16.3.1.1 Relative offset mode

In this mode, the leading dimension stride is specified as a relative byte offset between the columns as explained in the below table. The leading dimension stride can either be specified as a relative offset between the columns or as an absolute byte address of next buffer. The leading dimension stride is defined differently for transposed and non-transposed matrices. The leading dimension stride is defined as follows for matrices whose element types are normalized to 128-bits:

Major-ness	Definition
K-Major	▶ No-Swizzling: the stride from the first column to the second column of the 8x2 tile in the 128-bit element type normalized matrix. ▶ Swizzled layouts: not used, assumed to be 1.
MN-Major	▶ Interleave: stride from the first 8 columns to the next 8 columns. ▶ Swizzled layouts: stride from the first $(\text{swizzle-byte-size}/16)$ rows to the next $(\text{swizzle-byte-size}/16)$ rows.

Figure 187: Matrix Fragment for shape `.16x32bx2`

9.7.16.3.1.2 Absolute address mode for K dimension being 48B

The `tcgen05.mma` instruction with *K-dimension* of 48B would overflow the 128B shared memory boundary if the data is packed contiguously.

In this case, the absolute address mode can be used to break up the data in the shared memory into two chunks such that both these chunks are laid out within the aligned 128-byte address boundary. The leading dimension absolute address can point to the second data chunk in the shared memory.

9.7.16.3.1.3 Restrictions on the Leading Dimension Absolute Address Stride

Following are the restrictions on the absolute address stride mode:

1. Only 128B swizzle (with 16B atomicity) is supported.
2. Only K-Major mode is supported. That is, the transpose bits(bits #15 and #16) in *Instruction descriptor* must be 0.
3. The matrix base offset must be 0.

9.7.16.3.2 Stride Dimension Byte Offset

The stride dimension byte offset is defined differently for transposed and non-transposed matrices. The stride dimension byte offset is defined as follows for matrices whose element types are normalized to 128-bits:

Major-ness	Definition
K-Major	The offset from the first 8 rows to the next 8 rows.
MN-Major	► Interleave: offset from the first row to the next row. ► Swizzled layout: offset from the first 8 columns to the next 8 columns

9.7.16.3.3 Canonical Layouts

In terms of *CuTe layouts* the canonical layout can be expressed as follows:

Major-ness	Swizzling mode	Canonical Layout without swizzling	Swizzling on the previous column
MN- major	No-swizzling or Inter-leaved	$((T,1,m),(8,k)):((1,T,SBO),(1T,LBO))$	Swizzle<0, 4, 3>
	32B Swizzling	$((T,2,m),(8,k)):((1,T,LBO),(2T,SBO))$	Swizzle<1, 4, 3>
	64B Swizzling	$((T,4,m),(8,k)):((1,T,LBO),(4T,SBO))$	Swizzle<2, 4, 3>
	128B Swizzling	$((T,8,m),(8,k)):((1,T,LBO),(8T,SBO))$	Swizzle<3, 4, 3>
K- major	No-swizzling or Inter-leaved	$((8,m),(T,2k)):((1T,SBO),(1,LBO))$	Swizzle<0, 4, 3>
	32B Swizzling	$((8,m),(T,2k)):((2T,SBO),(1,T))$	Swizzle<1, 4, 3>
	64B Swizzling	$((8,m),(T,2k)):((4T,SBO),(1,T))$	Swizzle<2, 4, 3>
	128B Swizzling	$((8,m),(T,2k)):((8T,SBO),(1,T))$	Swizzle<3, 4, 3>

where

- $T = 128 / \text{sizeof-elements-in-bits}$ T represents scale factor which normalizes matrix element types to 128-bits.
- m represents the number of repeating patterns across rows.
- k represents the number of repeating patterns across columns.

Examples

- K-Major, no-swizzling and tf32 type: [Figure 188](#)

the strides and related details are as follows:

Exact layout : Swizzle<0,4,3> o $((8,2),(4,4)):((4,32),(1,64))$

Canonical Layout : Swizzle<0,4,3> o $((8,m),(T,2k)):((1T,SBO),(1,LBO))$

Parameters	Value
T	4
m	2
k	2
LBO (relative offset)	$64 * \text{sizeof}(\text{tf32})$
SBO	$32 * \text{sizeof}(\text{tf32})$
Encoding of LBO in descriptor	$(LBO) \gg 4 = 16$
Encoding of SBO in descriptor	$(SBO) \gg 4 = 8$

- K-Major, 32B swizzling and tf32 type: [Figure 189](#)

the strides and related details are as follows:

Exact layout : Swizzle<1,4,3> o $((8,2),(4,4)):((8,64),(1,4))$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	2	3	64	65	66	67	128	129	130	131	192	193	194	195
1	4	5	6	7	68	69	70	71	132	133	134	135	196	197	198	199
2	8	9	10	11	72	73	74	75	136	137	138	139	200	201	202	203
3	12	13	14	15	76	77	78	79	140	141	142	143	204	205	206	207
4	16	17	18	19	80	81	82	83	144	145	146	147	208	209	210	211
5	20	21	22	23	84	85	86	87	148	149	150	151	212	213	214	215
6	24	25	26	27	88	89	90	91	152	153	154	155	216	217	218	219
7	28	29	30	31	92	93	94	95	156	157	158	159	220	221	222	223
8	32	33	34	35	96	97	98	99	160	161	162	163	224	225	226	227
9	36	37	38	39	100	101	102	103	164	165	166	167	228	229	230	231
10	40	41	42	43	104	105	106	107	168	169	170	171	232	233	234	235
11	44	45	46	47	108	109	110	111	172	173	174	175	236	237	238	239
12	48	49	50	51	112	113	114	115	176	177	178	179	240	241	242	243
13	52	53	54	55	116	117	118	119	180	181	182	183	244	245	246	247
14	56	57	58	59	120	121	122	123	184	185	186	187	248	249	250	251
15	60	61	62	63	124	125	126	127	188	189	190	191	252	253	254	255

Figure 188: K major, no-swizzling and tf32 type

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	1	2	3	4	5	6	7	128	129	130	131	132	133	134	135
1	8	9	10	11	12	13	14	15	136	137	138	139	140	141	142	143
2	16	17	18	19	20	21	22	23	144	145	146	147	148	149	150	151
3	24	25	26	27	28	29	30	31	152	153	154	155	156	157	158	159
4	36	37	38	39	32	33	34	35	164	165	166	167	160	161	162	163
5	44	45	46	47	40	41	42	43	172	173	174	175	168	169	170	171
6	52	53	54	55	48	49	50	51	180	181	182	183	176	177	178	179
7	60	61	62	63	56	57	58	59	188	189	190	191	184	185	186	187
8	64	65	66	67	68	69	70	71	192	193	194	195	196	197	198	199
9	72	73	74	75	76	77	78	79	200	201	202	203	204	205	206	207
10	80	81	82	83	84	85	86	87	208	209	210	211	212	213	214	215
11	88	89	90	91	92	93	94	95	216	217	218	219	220	221	222	223
12	100	101	102	103	96	97	98	99	228	229	230	231	224	225	226	227
13	108	109	110	111	104	105	106	107	236	237	238	239	232	233	234	235
14	116	117	118	119	112	113	114	115	244	245	246	247	240	241	242	243
15	124	125	126	127	120	121	122	123	252	253	254	255	248	249	250	251

Figure 189: K major, 32B swizzling and tf32 type

Canonical Layout :Swizzle<1,4,3> o ((8,m),(T,2k)):((2T,SBO),(1,T))

Parameters	Value
T	4
m	2
k	2
LBO (relative offset)	NA
SBO	64*sizeof(tf32)
Encoding of LBO in descriptor	1 (assumed)
Encoding of SBO in descriptor	(SBO) >> 4 = 16

- MN-Major, no-swizzling and bf16 type: [Figure 190](#)

the strides and related details are as follows:

Exact layout : Swizzle<0,4,3> o ((8,1,2),(8,2)):((1,8,64),(8,128))

Canonical Layout :Swizzle<0,4,3> o ((T,1,m),(8,k)):((1,T,SBO),(1T,LBO))

Parameters	Value
T	8
m	2
k	2
LBO (relative offset)	128*sizeof(bf16)
SBO	64*sizeof(bf16)
Encoding of LBO in descriptor	(LBO) >> 4 = 16
Encoding of SBO in descriptor	(SBO) >> 4 = 8

- MN-Major, 32B swizzling and bf16 type: [Figure 191](#)

the strides and related details are as follows:

Exact layout : Swizzle<1,4,3> o ((8,2,2),(8,2)):((1,8,128),(16,256))

Canonical Layout :Swizzle<1,4,3> o ((T,2,m),(8,k)):((1,T,LBO),(2T,SBO))

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	8	16	24	32	40	48	56	128	136	144	152	160	168	176	184
1	1	9	17	25	33	41	49	57	129	137	145	153	161	169	177	185
2	2	10	18	26	34	42	50	58	130	138	146	154	162	170	178	186
3	3	11	19	27	35	43	51	59	131	139	147	155	163	171	179	187
4	4	12	20	28	36	44	52	60	132	140	148	156	164	172	180	188
5	5	13	21	29	37	45	53	61	133	141	149	157	165	173	181	189
6	6	14	22	30	38	46	54	62	134	142	150	158	166	174	182	190
7	7	15	23	31	39	47	55	63	135	143	151	159	167	175	183	191
8	64	72	80	88	96	104	112	120	192	200	208	216	224	232	240	248
9	65	73	81	89	97	105	113	121	193	201	209	217	225	233	241	249
10	66	74	82	90	98	106	114	122	194	202	210	218	226	234	242	250
11	67	75	83	91	99	107	115	123	195	203	211	219	227	235	243	251
12	68	76	84	92	100	108	116	124	196	204	212	220	228	236	244	252
13	69	77	85	93	101	109	117	125	197	205	213	221	229	237	245	253
14	70	78	86	94	102	110	118	126	198	206	214	222	230	238	246	254
15	71	79	87	95	103	111	119	127	199	207	215	223	231	239	247	255

Figure 190: MN major, no-swizzling and bf16 type

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	16	32	48	72	88	104	120	256	272	288	304	328	344	360	376
1	1	17	33	49	73	89	105	121	257	273	289	305	329	345	361	377
2	2	18	34	50	74	90	106	122	258	274	290	306	330	346	362	378
3	3	19	35	51	75	91	107	123	259	275	291	307	331	347	363	379
4	4	20	36	52	76	92	108	124	260	276	292	308	332	348	364	380
5	5	21	37	53	77	93	109	125	261	277	293	309	333	349	365	381
6	6	22	38	54	78	94	110	126	262	278	294	310	334	350	366	382
7	7	23	39	55	79	95	111	127	263	279	295	311	335	351	367	383
8	8	24	40	56	80	96	112	128	264	280	296	312	320	336	352	368
9	9	25	41	57	81	97	113	129	265	281	297	313	321	337	353	369
10	10	26	42	58	82	98	114	130	266	282	298	314	322	338	354	370
11	11	27	43	59	83	99	115	131	267	283	299	315	323	339	355	371
12	12	28	44	60	84	100	116	132	268	284	300	316	324	340	356	372
13	13	29	45	61	85	101	117	133	269	285	301	317	325	341	357	373
14	14	30	46	62	86	102	118	134	270	286	302	318	326	342	358	374
15	15	31	47	63	87	103	119	135	271	287	303	319	327	343	359	375
16	128	144	160	176	200	216	232	248	384	400	416	432	456	472	488	504
17	129	145	161	177	201	217	233	249	385	401	417	433	457	473	489	505
18	130	146	162	178	202	218	234	250	386	402	418	434	458	474	490	506
19	131	147	163	179	203	219	235	251	387	403	419	435	459	475	491	507
20	132	148	164	180	204	220	236	252	388	404	420	436	460	476	492	508
21	133	149	165	181	205	221	237	253	389	405	421	437	461	477	493	509
22	134	150	166	182	206	222	238	254	390	406	422	438	462	478	494	510
23	135	151	167	183	207	223	239	255	391	407	423	439	463	479	495	511
24	136	152	168	184	192	208	224	240	392	408	424	440	448	464	480	496
25	137	153	169	185	193	209	225	241	393	409	425	441	449	465	481	497
26	138	154	170	186	194	210	226	242	394	410	426	442	450	466	482	498
27	139	155	171	187	195	211	227	243	395	411	427	443	451	467	483	499
28	140	156	172	188	196	212	228	244	396	412	428	444	452	468	484	500
29	141	157	173	189	197	213	229	245	397	413	429	445	453	469	485	501
30	142	158	174	190	198	214	230	246	398	414	430	446	454	470	486	502
31	143	159	175	191	199	215	231	247	399	415	431	447	455	471	487	503

Figure 191: MN major, 32B swizzling and bf16 type

Parameters	Value
T	8
m	2
k	2
LBO (relative offset)	128*sizeof(bf16)
SBO	256*sizeof(bf16)
Encoding of LBO in descriptor	(LBO) >> 4 = 16
Encoding of SBO in descriptor	(SBO) >> 4 = 32

- MN-Major, 64B swizzling and bf16 type: [Figure 192](#)

the strides and related details are as follows:

Exact layout : Swizzle<2,4,3> o ((8,4,2),(8,2)):((1,8,256),(32,512))

Canonical Layout :Swizzle<2,4,3> o ((T,4,m),(8,k)):((1,T,LBO),(4T,SBO))

Parameters	Value
T	8
m	2
k	2
LBO (relative offset)	256*sizeof(bf16)
SBO	512*sizeof(bf16)
Encoding of LBO in descriptor	(LBO) >> 4 = 32
Encoding of SBO in descriptor	(SBO) >> 4 = 64

9.7.16.4 Matrix Descriptors

There are three kinds of matrix descriptors used by the `tcgen05` family of instructions.

9.7.16.4.1 Shared memory descriptor

The shared memory descriptor describes the properties of multiplicand matrix in shared memory including its location in the shared memory of the current *CTA*. It is a 64-bit value contained in a register with the following layout:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	9	32	72	104	144	176	216	248	312	344	384	436	488	528	576	608
1	1	33	73	105	145	177	217	249	313	345	385	437	489	529	577	609
2	2	34	74	106	146	178	218	250	314	346	386	438	490	530	578	610
3	3	35	75	107	147	179	219	251	315	347	387	439	491	531	579	611
4	4	36	76	108	148	180	220	252	316	348	388	440	492	532	580	612
5	5	37	77	109	149	181	221	253	317	349	389	441	493	533	581	613
6	6	38	78	110	150	182	222	254	318	350	390	442	494	534	582	614
7	7	39	79	111	151	183	223	255	319	351	391	443	495	535	583	615
8	8	40	80	112	152	184	224	256	320	352	392	444	496	536	584	616
9	9	41	81	113	153	185	225	257	321	353	393	445	497	537	585	617
10	10	42	82	114	154	186	226	258	322	354	394	446	498	538	586	618
11	11	43	83	115	155	187	227	259	323	355	395	447	499	539	587	619
12	12	44	84	116	156	188	228	260	324	356	396	448	500	540	588	620
13	13	45	85	117	157	189	229	261	325	357	397	449	501	541	589	621
14	14	46	86	118	158	190	230	262	326	358	398	450	502	542	590	622
15	15	47	87	119	159	191	231	263	327	359	399	451	503	543	591	623
16	16	48	88	120	160	192	232	264	328	360	400	452	504	544	592	624
17	17	49	89	121	161	193	233	265	329	361	401	453	505	545	593	625
18	18	50	90	122	162	194	234	266	330	362	402	454	506	546	594	626
19	19	51	91	123	163	195	235	267	331	363	403	455	507	547	595	627
20	20	52	92	124	164	196	236	268	332	364	404	456	508	548	596	628
21	21	53	93	125	165	197	237	269	333	365	405	457	509	549	597	629
22	22	54	94	126	166	198	238	270	334	366	406	458	510	550	598	630
23	23	55	95	127	167	199	239	271	335	367	407	459	511	551	599	631
24	24	56	96	128	168	200	240	272	336	368	408	460	512	552	600	632
25	25	57	97	129	169	201	241	273	337	369	409	461	513	553	601	633
26	26	58	98	130	170	202	242	274	338	370	410	462	514	554	602	634
27	27	59	99	131	171	203	243	275	339	371	411	463	515	555	603	635
28	28	60	100	132	172	204	244	276	340	372	412	464	516	556	604	636
29	29	61	101	133	173	205	245	277	341	373	413	465	517	557	605	637
30	30	62	102	134	174	206	246	278	342	374	414	466	518	558	606	638
31	31	63	103	135	175	207	247	279	343	375	415	467	519	559	607	639
32	32	64	104	136	176	208	248	280	344	376	416	468	520	560	608	640
33	33	65	105	137	177	209	249	281	345	377	417	469	521	561	609	641
34	34	66	106	138	178	210	250	282	346	378	418	470	522	562	610	642
35	35	67	107	139	179	211	251	283	347	379	419	471	523	563	611	643
36	36	68	108	140	180	212	252	284	348	380	420	472	524	564	612	644
37	37	69	109	141	181	213	253	285	349	381	421	473	525	565	613	645
38	38	70	110	142	182	214	254	286	350	382	422	474	526	566	614	646
39	39	71	111	143	183	215	255	287	351	383	423	475	527	567	615	647
40	40	72	112	144	184	216	256	288	352	384	424	476	528	568	616	648
41	41	73	113	145	185	217	257	289	353	385	425	477	529	569	617	649
42	42	74	114	146	186	218	258	290	354	386	426	478	530	570	618	650
43	43	75	115	147	187	219	259	291	355	387	427	479	531	571	619	651
44	44	76	116	148	188	220	260	292	356	388	428	480	532	572	620	652
45	45	77	117	149	189	221	261	293	357	389	429	481	533	573	621	653
46	46	78	118	150	190	222	262	294	358	390	430	482	534	574	622	654
47	47	79	119	151	191	223	263	295	359	391	431	483	535	575	623	655
48	48	80	120	152	192	224	264	296	360	392	432	484	536	576	624	656
49	49	81	121	153	193	225	265	297	361	393	433	485	537	577	625	657
50	50	82	122	154	194	226	266	298	362	394	434	486	538	578	626	658
51	51	83	123	155	195	227	267	299	363	395	435	487	539	579	627	659
52	52	84	124	156	196	228	268	300	364	396	436	488	540	580	628	660
53	53	85	125	157	197	229	269	301	365	397	437	489	541	581	629	661
54	54	86	126	158	198	230	270	302	366	398	438	490	542	582	630	662
55	55	87	127	159	199	231	271	303	367	399	439	491	543	583	631	663
56	56	88	128	160	200	232	272	304	368	400	440	492	544	584	632	664
57	57	89	129	161	201	233	273	305	369	401	441	493	545	585	633	665
58	58	90	130	162	202	234	274	306	370	402	442	494	546	586	634	666
59	59	91	131	163	203	235	275	307	371	403	443	495	547	587	635	667
60	60	92	132	164	204	236	276	308	372	404	444	496	548	588	636	668
61	61	93	133	165	205	237	277	309	373	405	445	497	549	589	637	669
62	62	94	134	166	206	238	278	310	374	406	446	498	550	590	638	670
63	63	95	135	167	207	239	279	311	375	407	447	499	551	591	639	671

Figure 192: MN major, 64B swizzling and bf16 type

Table 42: Shared memory descriptor layout

Bit-field	Size in bits	Description
0-13	14	matrix-descriptor-encode (Matrix start address)
16-29	14	matrix-descriptor-encode (<i>Leading dimension byte offset relative</i>) OR matrix-descriptor-encode (<i>Leading dimension byte address absolute</i>)
32-45	14	matrix-descriptor-encode (<i>Stride dimension byte offset</i>)
46-48	3	Fixed constant value of 0b001
49-51	3	Matrix base offset
52	1	Leading dimension stride mode: - 0: byte offset relative - 1: byte address absolute
53-60	8	Fixed constant value of 0xb00000000
61-63	3	Specifies the swizzling mode to be used: 0. No swizzling 1. 128-Byte with 32B atomic swizzling 2. 128-Byte swizzling 4. 64-Byte swizzling 6. 32-Byte swizzling Note: Values 3, 5 and 7 are invalid

where $\text{matrix-descriptor-encode}(x) = (x \& 0x3FFFF) \gg 4$

The value of base offset is 0 when the repeating pattern of the specified swizzling mode starts as per shown in Table 43.

Table 43: Starting address of repeating pattern for various swizzling modes

Swizzling mode	Starting address of the repeating pattern
128-Byte swizzle	1024-Byte boundary
64-Byte swizzle	512-Byte boundary
32-Byte swizzle	256-Byte boundary

Otherwise, the base offset must be a non-zero value, computed using the following formula: $\text{base offset} = (\text{pattern start addr} \gg 0x7) \& 0x7$

The following must be 16-byte aligned:

1. Matrix start address
2. Leading dimension byte offset
3. Stride dimension byte offset

9.7.16.4.1.1 Target ISA Note

- The byte address mode for the leading dimension stride is supported on sm_103a.

9.7.16.4.2 Instruction descriptor

The instruction descriptor describes the shapes, types and other details of all the matrices and the matrix-multiplication-and-accumulation operation. It is a 32-bit value in registers and the exact layout is dependent on the MMA-Kind:

Table 44: Instruction descriptor format for .kind::tf32, .kind::f16, .kind::f8f6f4 and .kind::i8

Bits	Size (bits)	Description	Values			
			.kind::tf32	.kind::f16	.kind::f8f6f4	.kind::i8
0-1	2	<i>Sparsity selector</i> , if Sparsity is enabled	0-3			
2	1	Sparsity	Dense = 0 Sparse = 1			
3	1	Saturate for integer types	0 (NA)			No Saturate = 0 Saturate = 1
4-5	2	dtype (Matrix D type)	F32 = 1	F16 = 0 F32 = 1		S32 = 2
6	1	Reserved	0			
7-9	3	atype (Matrix A type)	TF32 = 2	F16 = 0 BF16 = 1	E4M3 = 0 E5M2 = 1 E2M3 = 3 E3M2 = 4 E2M1 = 5	Unsigned 8b = 0 Signed 8b = 1
10-12	3	btype (Matrix B type)				
13	1	Negate A Matrix	No Negate = 0 Negate = 1			No Negate = 0
14	1	Negate B Matrix				
15	1	Transpose A Matrix	No Transpose = 0 Transpose = 1			
16	1	Transpose B Matrix				
17-22	6	N, Dimension of Matrix B (3 LSBs not included)	N >> 3			
23	1	Reserved	0			
24-28	5	M, Dimension of Matrix A (4 LSBs not included)	M >> 4			
29	1	Reserved	0			
30-31	2	Maximum shift while attempting B matrix-reuse in .ws	no shift = 0 maximum shift of 8 = 1 maximum shift of 16 = 2 maximum shift of 32 = 3			

Table 45: Instruction descriptor format for .kind::mxf8f6f4

Bits	Size (bits)	Description	Values
			.kind::mxf8f6f4
0-1	2	Reserved	0
2	1	Sparsity	Dense = 0 Sparse = 1
3	1	Reserved	0
4-5	2	<i>Matrix B Scale Factor Data ID</i>	0-3
6	1	Reserved	0
7-9	3	atype (Matrix A type)	E4M3 = 0 E5M2 = 1 E2M3 = 3 E3M2 = 4 E2M1 = 5
10-12	3	btype (Matrix B type)	
13	1	Negate A Matrix	No Negate = 0 Negate = 1
14	1	Negate B Matrix	
15	1	Transpose A Matrix	No Transpose = 0 Transpose = 1
16	1	Transpose B Matrix	
17-22	6	N, Dimension of Matrix B (3 LSBs not included)	N >> 3
23	1	Scale Matrix Type, for both scale_A / scale_B	UE8M0 = 1
24-26	3	Reserved	0
27-28	2	M, Dimension of Matrix A (7 LSBs not included)	M >> 7
29-30	2	<i>Matrix A Scale Factor Data ID</i>	0-3
31	1	Reserved	0

Table 46: Instruction descriptor format for .kind::mxf4 and .kind::mxf4nvvf4

Bits	Size (bits)	Description	Values	
			.kind::mxf4	.kind::mxf4nvvf4
0-1	2	Reserved	0	
2	1	Sparsity	Dense = 0 Sparse = 1	
3	1	Reserved	0	
4-5	2	<i>Matrix B Scale Factor Data ID</i>	0 or 2	
6	1	Reserved	0	
7-9	3	atype (Matrix A type)	E2M1 = 1	
10-11	2	btype (Matrix B type)		
12	1	Reserved	0	
13	1	Negate A Matrix	No Negate = 0 Negate = 1	
14	1	Negate B Matrix		
15	1	Transpose A Matrix	No Transpose = 0	
16	1	Transpose B Matrix		
17-22	6	N, Dimension of Matrix B (3 LSBs not included)	N >> 3	
23	1	Scale Matrix Type, for both scale_A / scale_B	UE8M0 = 1	UE4M3 = 0
24-26	3	Reserved	0	
27-28	2	M, Dimension of Matrix A (7 LSBs not included)	M >> 7	
29-30	2	<i>Matrix A Scale Factor Data ID</i>	0 or 2	
31	1	K Dimension	(Dense K=64 / Sparse K=128) = 0 (Dense K=96) = 1	

9.7.16.4.3 Zero-Column Mask Descriptor

The zero-column mask descriptor is used to generate a mask that specifies which columns of B matrix will have zero value for the MMA operation regardless of the values present in the shared memory. The total size of the generated mask is N-bits.

A 0-bit in the mask specifies that values of the corresponding column in matrix B should be used for the MMA operation. A 1-bit in the mask specifies 0s must be used for the entire column for the MMA operation.

The zero-column mask descriptor is a 64-bit value in registers with the following layout:

Table 47: Zero-Column Mask descriptor layout

Bits	Size (bits)	Field Name	Description
0-7	8	Start Count 0 (sc0)	Specifies the LSBs that must be skipped for sub-mask mask-i
8-15	8	Start Count 1 (sc1)	
16-23	8	Start Count 2 (sc2)	
24-31	8	Start Count 3 (sc3)	
32	1	First Span 0 (fs0)	Specifies the starting value for sub-mask mask-i
33	1	First Span 1 (fs1)	
34	1	First Span 2 (fs2)	
35	1	First Span 3 (fs3)	
36-38	3	Reserved	
39	1	Non-Zero Mask	Value 0 indicates generated mask will have all 0s Value 1 indicates the mask has to be generated
40-47	8	Skip Span	(Count of consecutive columns where B matrix is used) - 1
48-55	8	Use Span	(Count of consecutive columns where 0s are used) - 1
56-61	6	Column Shift	Shifts column by specified amount. Thus allows MMA on non-0 starting column. Max shift amount = 16 for M=32 Max shift amount = 32 otherwise

The zero-column mask is made up of one or more sub-mask depending on M, as shown in the table:

M	Zero-Column Mask breakup	Sub-masks	First Span used	Start Column used
128	Single sub-mask of size N-bits	mask0	fs0	sc0
64	Two sub-masks, each with size of N/2 bits	mask0, mask1	fs0, fs1	sc0, sc1
32	Four sub-masks, each with size of N/4 bits	mask0, mask1 mask2, mask3	fs0, fs1, fs2, fs3	sc0, sc1, sc2, sc3

The following table shows the coverage of the sub-masks across N-dimension:

Sub-mask	M		
	128	64	32
mask0	Columns [0, N-1]	Columns [0, N/2-1]	Columns [0, N/4-1]
mask1	–	Columns [N/2, N-1]	Columns [N/4, N/2-1]
mask2	–	–	Columns [N/2, (N/4*3)-1]
mask3	–	–	Columns [(N/4*3), N-1]

The following examples shows zero-column mask descriptor and their corresponding mask generated:

1. Example 1: M = 128

Input zero-column mask descriptor:

Start count	First span	Non-Zero Mask	Skip Span	Use Span	Shift
{0, 0, 0, 0}	{0, 0, 0, 0}	0	4	3	0

Output zero-column mask: 0x0.

As Non-Zero Mask field is 0, the mask is 0x0. All the columns of the matrix B will be used for the MMA operation.

2. Example 2: M = 128

Input zero-column mask descriptor:

Start count	First span	Non-Zero Mask	Skip Span	Use Span	Shift
{-, -, -, 0}	{-, -, -, 0}	1	2	3	0

Output mask0: 0b ... 111 0000 111 0000 (size = N)

3. Example 3: M = 64

Input zero-column mask descriptor:

Start count {..., sc1, sc0}	First span {..., fs1, fs0}	Non-Zero Mask	Skip Span	Use Span	Shift
{-, -, 0, 0}	{-, -, 0, 1}	1	2	3	0

Output mask0: 0b ... 111 0000 111 0000 111

Output masl1: 0b ... 0000 111 0000 111 0000

4. Example 4: M = 32

Input zero-column mask descriptor:

Start count {sc3, sc2, sc1, sc0}	First span {fs3, fs2, fs1, fs0}	Non-Zero Mask	Skip Span	Use Span	Shift
{1, 2, 1, 0}	{0, 0, 1, 1}	1	2	3	2

Output mask0: 0b ... 0000 111 0000 111

Output mask1: 0b ... 0000 111 0000 11

Output mask2: 0b ... 111 0000 111 00

Output mask3: 0b ... 111 0000 111 000

If $N = 128$ then B Matrix with columns from 2 to 129 will be used for the MMA operation, due to the shift of 2.

9.7.16.5 Issue Granularity

Each of the tcgen05 operation has different requirements for the number of threads/warps that needs to issue them.

The following table lists the execution granularity requirements of each of the tcgen05 operation:

Table 48: Execution granularity requirements for tcgen05 operations

tcgen05 operation	.cta_group	Issue Granularity
.mma, .cp, .shift, .commit	::1	An issue from a single thread in the current CTA would initiate the base operation.
	::2	Issue from a single thread from the <i>CTA-Pair</i> would initiate the base operation. When the current CTA issues the operation, the peer CTA should be active and should not have exited.
.alloc, .dealloc, .relinquish_alloc_permit	::1	Issue from a single warp in the current CTA would initiate the allocation management instruction.
	::2	Issue from two warps, one in each of the current CTA and its <i>Peer CTA</i> , collectively needs to perform the operation. When the current CTA issues the operation, the peer CTA should be active and should not have exited.

continues on next page

Table 48 – continued from previous page

tcgen05 operation	.cta_group	Issue Granularity
<code>.ld,</code> <code>.st,</code> <code>.wait::{ld, st}</code>	N/A	Issue from a warp in the current CTA can access only 1/4 of the Tensor Memory of the current CTA. So, a warp-group is needed to access the entire Tensor Memory of the current CTA.
<code>.fence::*</code>	N/A	A thread needs to fence all its accesses to the tensor memory that it wants to order with other accesses to the tensor memory from other threads.

9.7.16.5.1 CTA Pair

Any 2 CTAs within the cluster whose `%cluster_ctarank` differs by the last bit only is said to form a CTA pair.

Within a CTA pair, the CTA whose last bit in the `%cluster_ctarank` is:

- ▶ 0 is termed the even numbered CTA within the CTA pair.
- ▶ 1 is termed as the odd numbered CTA within the CTA pair.

Most of the `tcgen05` operations can either execute at a single CTA level granularity OR at a CTA pair level granularity. When a `tcgen05` operation is performed at CTA pair granularity, the Tensor Memory of both the CTAs within the CTA pair are accessed. The set of threads that need to issue the `tcgen05` operation is listed in the *Issue Granularity*.

9.7.16.5.2 Peer CTA

The peer CTA of the odd CTA within the CTA pair is the even CTA in the same pair. Similarly, the peer CTA of the even CTA within the CTA pair is the odd CTA in the same pair.

9.7.16.6 Memory Consistency Model for 5th generation of TensorCore operations

Ordering of `tcgen05` instructions is described in terms of two key concepts:

1. Pipelined `tcgen05` instructions
2. Specialized `tcgen05`-specific inter-thread synchronization mechanisms.

These concepts combine to form four canonical synchronization patterns, as described further below.

9.7.16.6.1 Asynchronous Operations

The `tcgen05` family of instructions are divided into 2 categories:

1. Asynchronous instructions:

These `tcgen05` operations are not inherently ordered with respect to other `tcgen05` operations in the same thread (unless pipelined as mentioned below).

2. Synchronous instructions:

These `tcgen05` operations are inherently ordered with respect to other `tcgen05` operations in the same order.

The Tensor Memory allocation related instructions that access shared memory maintain same-address ordering with respect to non-`tcgen05` instructions.

The following table lists the category of each of the `tcgen05` instruction:

tcgen05.* operation	Category
<code>.alloc</code>	Synchronous instructions
<code>.dealloc</code>	
<code>.relinquish_alloc_permit</code>	
<code>.fence::*</code>	
<code>.wait::*</code>	
<code>.commit</code>	
<code>.mma</code>	Asynchronous instructions
<code>.cp</code>	
<code>.shift</code>	
<code>.ld</code>	
<code>.st</code>	

9.7.16.6.2 Pipelined tcgen05 Instructions

The asynchronous `tcgen05` operations may execute and complete in a different order than they were issued. However, some specific pairs of the asynchronous `tcgen05` instructions form `tcgen05` pipelines, where in the two asynchronous operations are guaranteed to execute in the same order as the instructions that issued them. The specific pairings are as follows:

1. `tcgen05.mma.cta_group::N -> tcgen05.mma.cta_group::N` (same N and accumulator and shape)
2. `tcgen05.copy.cta_group::N -> tcgen05.mma.cta_group::N` (same N)
3. `tcgen05.shift.cta_group::N -> tcgen05.mma.cta_group::N` (same N)
4. `tcgen05.shift.cta_group::N -> tcgen05.cp.4x256b.cta_group::N` (same N)
5. `tcgen05.mma.cta_group::N -> tcgen05.shift.cta_group::N` (same N)

9.7.16.6.2.1 Implicitly pipelined tcgen05 Instructions

Instructions `tcgen05.commit` and `tcgen05.wait` are implicitly pipelined with respect to previously issued `tcgen05.{mma, cp, shift}` and `tcgen05.{ld, st}` instructions respectively that they track from the same thread.

9.7.16.6.2.2 mbarrier based completion mechanism

Completion of the following instruction's asynchronous operations is observed through the `mbarrier` based waiting mechanism:

1. `tcgen05.mma`
2. `tcgen05.cp`
3. `tcgen05.shift`

`tcgen05.commit` is used to track the completion of the above asynchronous instructions.

Following are the implicitly pipelined `tcgen05` instruction pairing that uses `mbarrier` based completion mechanism:

- ▶ `tcgen05.mma.cta_group::N -> tcgen05.commit.cta_group::N` (same N)
- ▶ `tcgen05.cp.cta_group::N -> tcgen05.commit.cta_group::N` (same N)
- ▶ `tcgen05.shift.cta_group::N -> tcgen05.commit.cta_group::N` (same N)

9.7.16.6.2.3 tcgen05.wait instruction based completion mechanism

Completion of the following instruction's asynchronous operations is observed through `tcgen05.wait` based waiting mechanism:

1. `tcgen05.ld`
2. `tcgen05.st`

`tcgen05.wait::ld` and `tcgen05.wait::st` is used to track the completion of the `tcgen05.ld` and `tcgen05.st` asynchronous instructions.

Following are the implicitly pipelined `tcgen05` instruction pairing that uses `tcgen05.wait` based completion mechanism:

- ▶ `tcgen05.ld -> tcgen05.wait::ld`
- ▶ `tcgen05.st -> tcgen05.wait::st`

9.7.16.6.3 Specialized Inter-thread Synchronization for tcgen05 instructions

The `tcgen05` instructions support a specialized inter-thread synchronization which are optimized for `tcgen05` family of instructions. The standard memory consistency model synchronization mechanisms also apply to the `tcgen05` family of instructions.

The *TensorCore 5th Generation Specialized Synchronization Operations* section contains the specialized inter-thread synchronization for `tcgen05` instructions.

The `tcgen05.fence::before_thread_sync` and `tcgen05.fence::after_thread_sync` composes with execution ordering instructions, like morally strong `ld/st/atom` instructions, `mbarrier`

instruction, barrier instructions and so on, to establish an ordering between the `tcgen05` operations across threads. The asynchronous `tcgen05` instructions that are ordered across threads also form a `tcgen05` pipeline.

An asynchronous `tcgen05` operation prior to a `tcgen05.fence::before_thread_sync` is ordered before all subsequent `tcgen05` and the execution ordering operations.

An asynchronous `tcgen05` operation subsequent to a `tcgen05.fence::after_thread_sync` is ordered after all the prior `tcgen05` and the execution ordering operations.

9.7.16.6.4 Canonical synchronization patterns

Using the above rules, the following are the five canonical synchronization patterns:

9.7.16.6.4.1 Pipelined instructions, same thread

In this pattern, no explicit ordering mechanism is needed and the ordering guarantee is provided by the pipelined instruction pairing.

Example:

```
tcgen05.mma
tcgen05.mma (same shape and accumulator)
```

The two instructions will be executed in program order.

9.7.16.6.4.2 Non-pipelined instructions, same thread

In this pattern, explicit waiting mechanisms are used to wait for the completion of the asynchronous `tcgen05` operations.

Example 1:

```
tcgen05.st
tcgen05.wait::st
tcgen05.ld
```

`tcgen05.wait::st` is used to wait for the completion of the prior asynchronous instruction `tcgen05.st`.

Example 2:

```
tcgen05.mma [d], ...
tcgen05.commit.mbarrier::arrive::one
mbarrier.try_wait.relaxed.cluster (loop until successful)
tcgen05.fence::after_thread_sync
tcgen05.ld [d], ...
```

For the completion of the asynchronous `tcgen05.mma`, `tcgen05.commit` is used.

As `tcgen05.ld` is an asynchronous operation, the instruction `tcgen05.fence::after_thread_sync` is needed.

No explicit `tcgen05.fence::before_thread_sync` is needed as this is implicitly performed by `tcgen05.commit`. The combination of `tcgen05.mma` and `tcgen05.commit` forms a conceptual asynchronous pipeline and establishes execution ordering.

```
tcgen05.mma [d], ...
tcgen05.fence::before_thread_sync
mbarrier::arrive
```

9.7.16.6.4.3 Pipelined instructions, different thread

In this pattern, no explicit waiting mechanism is needed but proper synchronization between threads is needed.

Example:

Thread 0	Thread 1
<pre>tcgen05.cp tcgen05.fence::before_thread_sync mbarrier.arrive.relaxed.cluster</pre>	
	<pre>mbarrier.try_wait.relaxed.cluster // ↪ loop till success tcgen05.fence::after_thread_sync tcgen05.mma</pre>

9.7.16.6.4.4 Non-pipelined instructions, different thread

In this pattern, the producer threads that issue the asynchronous tcgen05 instructions must explicitly wait for the instructions' completion before synchronizing with the consumer threads.

Example 1:

Thread 0	Thread 1
<pre>tcgen05.ld tcgen05.wait::ld tcgen05.fence::before_thread_sync mbarrier.arrive.relaxed.cluster</pre>	
	<pre>mbarrier.try_wait.relaxed.cluster // ↪ loop till success tcgen05.fence::after_thread_sync tcgen05.mma</pre>

Example 1:

Thread 0	Thread 1
<pre>tcgen05.mma tcgen05.commit.mbarrier::arrive::one ↪ [mbar]</pre>	
	<pre>mbarrier.try_wait.relaxed.cluster [mbar] ↪ // loop till success tcgen05.fence::after_thread_sync tcgen05.ld</pre>

The synchronization mechanisms can also be composed with each other. For example:

Thread 0	Thread 1
<pre>tcgen05.mma tcgen05.commit.mbarrier::arrive::one ↪ [bar1] mbarrier.try_wait.relaxed.cluster [bar1] ↪ // loop ... tcgen05.fence::after_thread_sync ... // completion is guaranteed tcgen05.fence::before_thread_sync mbarrier.arrive.relaxed.cluster [bar2] ↪ // loop ...</pre>	
	<pre>mbarrier.try_wait.relaxed.cluster [bar2] ↪ // loop ... tcgen05.fence::after_thread_sync tcgen05.ld</pre>

9.7.16.6.4.5 Register dependencies, same thread

For `tcgen05.ld`, an intra-thread ordering through true register dependency will be respected regardless of the presence or absence of other forms of synchronization. This form of register dependency does not imply any other form of ordering. For example, a register dependency does not imply that a dependee instruction's memory accesses will be performed before a dependent instruction's memory accesses. To enforce such memory orderings and avoiding anti-dependency hazards around `tcgen05.ld`, `tcgen05.wait::ld` must be used.

Example:

```
tcgen05.ld %r1, ...;
tcgen05.mma ..., %r1, ...;
```

9.7.16.6.5 Shared Memory Accesses

The shared memory accesses by `tcgen05.mma` and `tcgen05.cp` operations are performed in the asynchronous proxy (async proxy).

Accessing the same memory location across multiple proxies needs a cross-proxy fence. For the async proxy, `fence.proxy.async` should be used to synchronize memory between generic proxy and the async proxy.

9.7.16.7 Tensor Memory Allocation and Management Instructions

9.7.16.7.1 Tensorcore 5th Generation Instructions: `tcgen05.alloc`, `tcgen05.dealloc`, `tcgen05.relinquish_alloc_permit`

`tcgen05.alloc`, `tcgen05.dealloc`, `tcgen05.relinquish_alloc_permit`

Dynamic *Tensor Memory* allocation management instructions

Syntax

```
tcgen05.alloc.cta_group.sync.aligned{.shared::cta}.b32  [dst], nCols;
tcgen05.dealloc.cta_group.sync.aligned.b32             taddr, nCols;
tcgen05.relinquish_alloc_permit.cta_group.sync.aligned;
.cta_group = { .cta_group::1, .cta_group::2 }
```

Description

`tcgen05.alloc` is a potentially blocking instruction which dynamically allocates the specified number of columns in the *Tensor Memory* and writes the address of the allocated *Tensor Memory* into shared memory at the location specified by address operand `dst`. The `tcgen05.alloc` blocks if the requested amount of *Tensor Memory* is not available and unblocks as soon as the requested amount of *Tensor Memory* becomes available for allocation.

Instruction `tcgen05.dealloc` deallocates the *Tensor Memory* specified by the *Tensor Memory* address `taddr`. The operand `taddr` must point to a previous *Tensor Memory* allocation.

All of the Tensor Memory that was allocated using `tcgen05.alloc` instruction in a kernel, must be explicitly deallocated using `tcgen05.dealloc` before the kernel exits.

The unsigned 32-bit operand `nCols` specify the number of columns to be allocated or de-allocated. The unit of allocation and de-allocation is 32 columns and all of lanes per column. The number of columns must be a power of 2. The operand `nCols` must be within the range [32, 512]. The number of columns allocated should not increase between any two allocations in the execution order within the CTA. Operand `nCols` must be power of 2.

Instruction `tcgen05.relinquish_alloc_permit` specifies that the CTA of the executing thread is relinquishing the right to allocate *Tensor Memory*. So, it is illegal for a CTA to perform `tcgen05.alloc` after any of its constituent threads execute `tcgen05.relinquish_alloc_permit`.

If no state space is specified then *Generic Addressing* is used. If the address specified by `dst` does not fall within the address window of `.shared::cta` state space then the behavior is undefined.

Qualifier `.cta_group` specifies the number of CTAs involved in the allocation and de-allocation operation. When `.cta_group::1` is specified, one warp from the CTA must perform the allocation and de-allocation. When `.cta_group::2` is specified, one warp from each of the *peer CTAs* must collectively perform the allocation and de-allocation. Refer to the *Issue Granularity* section. When `.cta_group::2` is specified, the issuing warp must make sure that peer CTA is launched and is still active.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The mandatory `.sync` qualifier indicates that the instruction causes the executing thread to wait until all threads in the warp execute the same instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same instruction. In conditionally executed code, the instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of the instruction is undefined if all the threads in the warp do not use the same values of `nCols`, or if any thread in the warp has exited.

The store operation in `tcgen05.alloc` is treated as a weak memory operation in the *Memory Consistency Model*.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```
// Example 1:

tcgen05.alloc.cta_group::1.sync.aligned.shared::cta.b32 [sMemAddr1], 32;
ld.shared.b32 taddr, [sMemAddr1];
// use taddr ...
// more allocations and its usages ...
tcgen05.dealloc.cta_group::1.sync.aligned.b32 taddr, 32;
// more deallocations ...
tcgen05.relinquish_alloc_permit.cta_group::1.sync.aligned;
```

(continues on next page)

(continued from previous page)

```
// Example 2:

// Following instructions are performed by current warp and the warp in the peer-CTA:
tcgen05.alloc.cta_group::2.sync.aligned.shared::cta.b32 [sMemAddr2], 32;
ld.shared.b32 taddr, [sMemAddr2];
// use taddr ...
// more allocations and its usages ...
tcgen05.dealloc.cta_group::2.sync.aligned.b32 taddr, 32;
// more deallocations ...
tcgen05.relinquish_alloc_permit.cta_group::2.sync.aligned;
```

9.7.16.8 Tensor Memory and Register Load/Store Instructions

The threads of the CTA can perform the loads and stores to the *Tensor Memory* of the CTA and move data between registers and Tensor Memory. The loads and stores of data can be performed in certain shapes as specified in the *Matrix and Data Movement Shape* section.

9.7.16.8.1 Access restrictions

Not all threads of the CTA can access the entire Tensor Memory via the `tcgen05.ld` and `tcgen05.st` operations.

The Tensor Memory of a CTA is divided into 4 equal chunks such that each warp of a warpgroup in the CTA can access a chunk of the Tensor Memory. All the columns of the Tensor Memory can be accessed by all the four warps of a warpgroup. A lane of the Tensor Memory can be accessed by a single warp in the warpgroup. The following table describes the access restriction.

ID of the warp within the warpgroup	Accessible Lanes
0	0-31
1	32-63
2	64-95
3	96-127

9.7.16.8.2 Packing and Unpacking

Optionally, the following pack and unpack operations can be performed during the load and store:

1. Packing: two 16-bit chunks can be packed into a single 32-bit chunk in the register in `tcgen05.ld`
2. Unpacking: a single 32-bit chunk in the register can be unpacked into two 16-bit chunks in `tcgen05.st`

as shown in the [Figure 193](#).

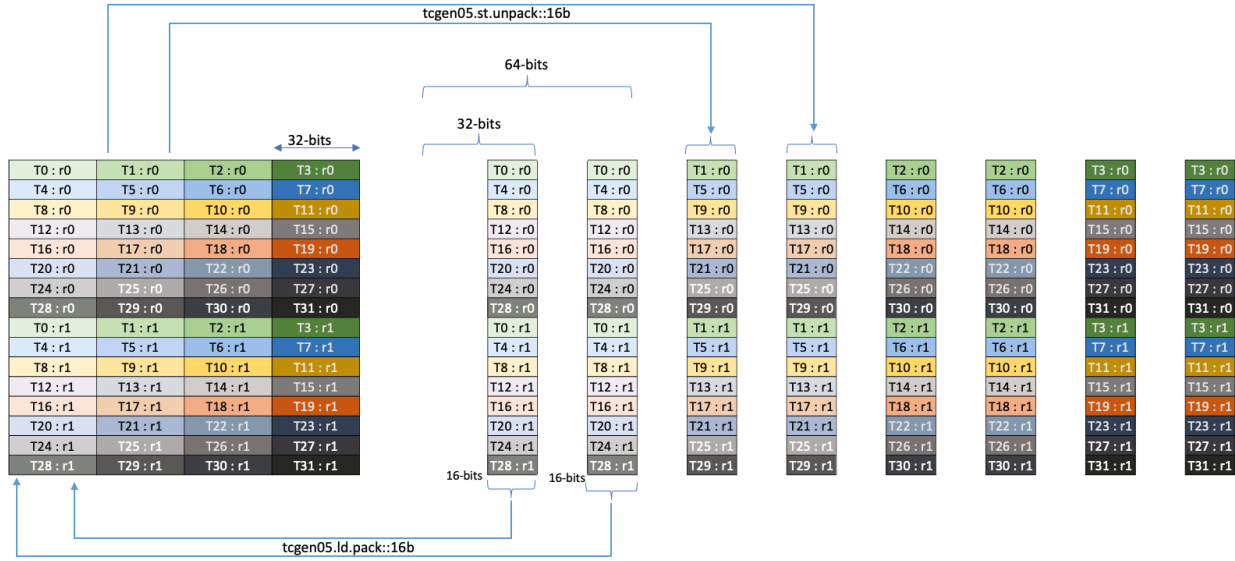


Figure 193: Pack/Unpack operations for tcgen05 ld/st

9.7.16.8.3 Tensorcore 5th Generation Instructions: tcgen05.ld

tcgen05.ld

Asynchronous collective load from tensor memory into registers.

Syntax

```
// Base load instruction:
tcgen05.ld.sync.aligned.shape1.num{.pack}.b32    r, [taddr];
tcgen05.ld.sync.aligned.shape2.num{.pack}.b32    r, [taddr], immHalfSplitoff;

.shape1 = { .16x64b, .16x128b, .16x256b, .32x32b }
.shape2 = { .16x32bx2 }
.num     = { .x1, .x2, .x4, .x8, .x16, .x32, .x64, .x128 }
.pack    = { .pack::16b }

// Floating point type load along with reduction :
tcgen05.ld.red.sync.aligned.shape3.num.redOp{.abs}{.NaN}.f32 r, redval, [taddr];
tcgen05.ld.red.sync.aligned.shape4.num.redOp{.abs}{.NaN}.f32 r, redval, [taddr],
↪immHalfSplitoff;

// Integer type load along with reduction :
```

(continues on next page)

(continued from previous page)

```

tcgen05.ld.red.sync.aligned.shape3.num.redOp.type r, redval, [taddr];

tcgen05.ld.red.sync.aligned.shape4.num.redOp.type r, redval, [taddr], immHalfSplitoff;

.shape3 = { .32x32b    }
.shape4 = { .16x32bx2  }
.redOp  = { .min, .max }
.type   = { .u32, .s32 }

```

Description

Instruction `tcgen05.ld` asynchronously loads data from the *Tensor Memory* at the location specified by the 32-bit address operand `taddr` into the destination register `r`, collectively across all threads of the warps.

All the threads in the warp must specify the same value of `taddr`, which must be the base address of the collective load operation. Otherwise, the behavior is undefined.

The `.shape` qualifier and the `.num` qualifier together determines the total dimension of the data which is loaded from the *Tensor Memory*. The `.shape` qualifier indicates the base dimension of data to be accessed as described in the *Data Movement Shape*. The `.num` qualifier indicates the repeat factor on the base dimension resulting in the total dimension of the data that is accessed.

The shape `.16x32bx2` performs two accesses into Tensor Memory of the shape `.16x32b`. The base address of the first access is specified by `taddr` and the base address of the second access is specified by `taddr+immHalfSplitoff`, where `immHalfSplitoff` is an immediate argument.

The destination operand `r` is a brace-enclosed vector expression consisting of one or more 32-bit registers as per the value of `.shape` and `.num`. The size of the vector for various combinations of `.num` and `.shape` is shown in [Table 49](#).

Table 49: Various-combinations of `.num` and `.shape`

.num	.shape		
	.16x32bx2 / .16x64b / .32x32b	.16x128b	.16x256b
.x1	1	2	4
.x2	2	4	8
.x4	4	8	16
.x8	8	16	32
.x16	16	32	64
.x32	32	64	128
.x64	64	128	NA
.x128	128	NA	NA

The qualifier `.red` specifies that the reduction operation specified by `.redOp` is performed on the data that is loaded across columns in each lane. The result of the reduction operation is written into the corresponding thread's 32-bit destination register operand `redVal`. When `.red` qualifier is specified, `.num` modifier must be at least `.x2`.

The optional qualifier `.pack::16b` can be used to pack two 16-bit elements from adjacent columns into a single 32-bit element during the load as shown in the section *Packing and Unpacking*.

The mandatory `.sync` qualifier indicates that `tcgen05.ld` causes the executing thread to wait until all threads in the warp execute the same `tcgen05.ld` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `tcgen05.ld` instruction. In conditionally executed code, a `tcgen05.ld` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of `tcgen05.ld` is undefined if all threads do not use the same values of `taddr`, or if any thread in the warp has exited.

The instruction `tcgen05.ld` is performed asynchronously and more details are specified in the section *Memory Consistency Model for 5th generation of TensorCore operations*.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

`tcgen05.ld.red` is introduced in PTX ISA version 8.8.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

`tcgen05.ld.red` is supported on following architectures:

- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
 - ▶ `sm_103f` or higher in the same family
- ▶ `sm_110f` or higher in the same family

Examples

```
tcgen05.ld.sync.aligned.32x32b.x2.b32    {r0, r1}, [taddr1];
tcgen05.ld.sync.aligned.16x128b.x4.b32    {r0, r1, r2, r3, r4, r5, r6, r7}, [taddr2];
tcgen05.ld.red.sync.aligned.16x32bx2.x8.u32.max {r0, r1, r2, r3, r4, r5, r6, r7},
                                           redVal, [taddr3], 16;
```

9.7.16.8.4 Tensorcore 5th Generation Instructions: tcgen05.st

tcgen05.st

Asynchronous collective store to tensor memory from registers.

Syntax

```
tcgen05.st.sync.aligned.shape1.num{.unpack}.b32    [taddr], r;
tcgen05.st.sync.aligned.shape2.num{.unpack}.b32    [taddr], immHalfSplitoff, r;

.shape1 = { .16x64b, .16x128b, .16x256b, .32x32b }
.shape2 = { .16x32bx2 }
.num     = { .x1, .x2, .x4, .x8, .x16, .x32, .x64, .x128 }
.unpack  = { .unpack::16b }
```

Description

Instruction `tcgen05.st` asynchronously stores data from the source register `r` into the *Tensor Memory* at the location specified by the 32-bit address operand `taddr`, collectively across all threads of the warps.

All the threads in the warp must specify the same value of `taddr`, which must be the base address of the collective store operation. Otherwise, the behavior is undefined.

The `.shape` qualifier and the `.num` qualifier together determines the total dimension of the data which is stored to the Tensor Memory. The `.shape` qualifier indicates the base dimension of data to be accessed as described in the *Data Movement Shape*. The `.num` qualifier indicates the repeat factor on the base dimension resulting in the total dimension of the data that is accessed.

The shape `.16x32bx2` performs two accesses into Tensor Memory of the shape `.16x32b`. The base address of the first access is specified by `taddr` and the base address of the second access is specified by `taddr+immHalfSplitoff`, where `immHalfSplitoff` is an immediate argument.

The source operand `r` is a brace-enclosed vector expression consisting of one or more 32-bit registers as per the value of `.shape` and `.num`. The size of the vector for various combinations of `.num` and `.shape` is shown in [Table 50](#).

Table 50: Various-combinations of .num and .shape

.num	.shape		
	.16x32bx2 / .16x64b / .32x32b	.16x128b	.16x256b
.x1	1	2	4
.x2	2	4	8
.x4	4	8	16
.x8	8	16	32
.x16	16	32	64
.x32	32	64	128
.x64	64	128	NA
.x128	128	NA	NA

The optional qualifier `.unpack : 16b` can be used to unpack a 32-bit element in the register into two 16-bit elements and store them in adjacent columns as shown in the section [Packing and Unpacking](#).

The mandatory `.sync` qualifier indicates that `tcgen05.st` causes the executing thread to wait until all threads in the warp execute the same `tcgen05.st` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `tcgen05.st` instruction. In conditionally executed code, a `tcgen05.st` instruction should only be used if it is known that all threads in the warp evaluate the condition identically, otherwise behavior is undefined.

The behavior of `tcgen05.st` is undefined if all threads do not use the same values of `taddr`, or if any thread in the warp has exited.

The instruction `tcgen05.st` is performed asynchronously and more details are specified in the section [Memory Consistency Model for 5th generation of TensorCore operations](#).

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```
tcgen05.st.sync.aligned.16x64b.x4.b32      [taddr0], {r0, r1, r2, r3};
tcgen05.st.sync.aligned.16x128b.x1.unpack::16b.b32 [taddr1], {r0, r1};
```

9.7.16.8.5 Tensorcore 5th Generation Instructions: `tcgen05.wait`

`tcgen05.wait`

Waits for the completion of all prior asynchronous `tcgen05.ld` / `tcgen05.st` instructions.

Syntax

```
tcgen05.wait_operation.sync.aligned;
.wait_operation = { .wait::ld, .wait::st }
```

Description

Instruction `tcgen05.wait::st` causes the executing thread to block until all prior `tcgen05.st` operations issued by the executing thread have completed.

Instruction `tcgen05.wait::ld` causes the executing thread to block until all prior `tcgen05.ld` operations issued by the executing thread have completed.

The mandatory `.sync` qualifier indicates that `tcgen05.wait_operation` causes the executing thread to wait until all threads in the warp execute the same `tcgen05.wait_operation` instruction before resuming execution.

The mandatory `.aligned` qualifier indicates that all threads in the warp must execute the same `tcgen05.wait_operation` instruction.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

Example 1:

```
tcgen05.ld.sync.aligned.32x32b.x2.b32    {r0, r1}, [taddr0];
// Prevents subsequent tcgen05.mma from racing ahead of the tcgen05.ld
tcgen05.wait::ld.sync.aligned;
tcgen05.mma.cta_group::1.kind::f16    [taddr0], a-desc, b-desc, idesc, p;
```

Example 2:

```
tcgen05.st.sync.aligned.32x32b.x2.b32    [taddr0], {r0, r1};
// Prevents the write to taddr0 in tcgen05.mma from racing ahead of the tcgen05.st
tcgen05.wait::st.sync.aligned;
tcgen05.mma.cta_group::1.kind::f16    [taddr0], a-desc, b-desc, idesc, p;
```

9.7.16.9 Tensor Memory Data Movement Instructions

Data from the shared memory can be copied asynchronously to the *Tensor Memory* using the *Tensor-core 5th Generation Instructions: tcgen05.cp* operation.

9.7.16.9.1 Optional Decompression

Optionally, during the copy, a vector of 4-bit and 6-bit custom floating point types can be decompressed into 8-bit types.

9.7.16.9.1.1 Decompression of 4-bit floating point to 8-bit type

A contiguous set of 16 elements of 4-bits each followed by 8 bytes of padding can be converted into 16 elements of 8-bits each as shown in [Figure 194](#).

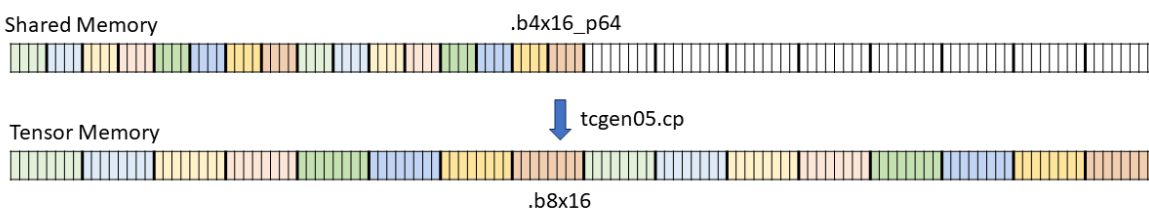


Figure 194: Decompression from 4-bit to 8-bit

The individual 4-bit to 8-bit decompression would look like as shown in [Figure 195](#).

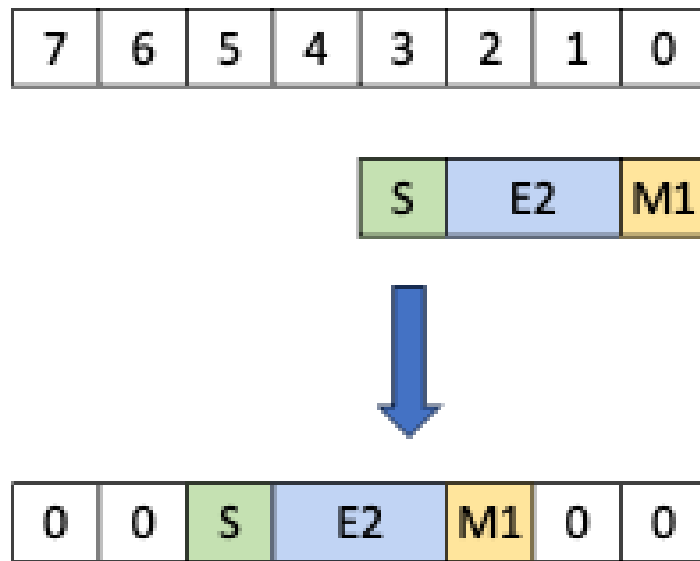


Figure 195: Individual decomposition from 4-bit to 8-bit

9.7.16.9.1.2 Decompression of 6-bit floating point to 8-bit type

A contiguous set of 16 elements of 6-bits each followed by 4 bytes of padding is decompressed into 16 elements of 8-bits each as shown in Figure 196.

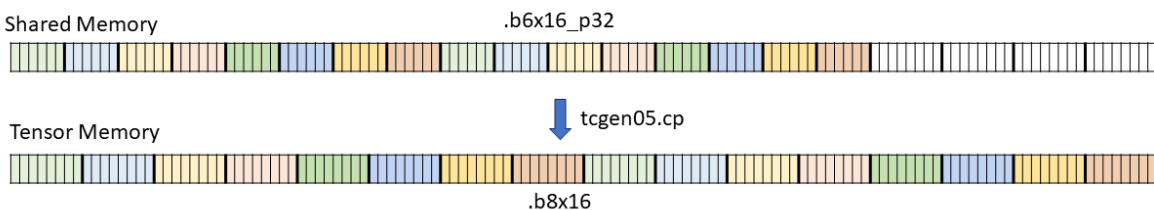


Figure 196: Decompression from 6-bit to 8-bit

The individual 6-bit to 8-bit decompression for types E3M2 and E2M3 is shown in Figure 197 and Figure 198 respectively.

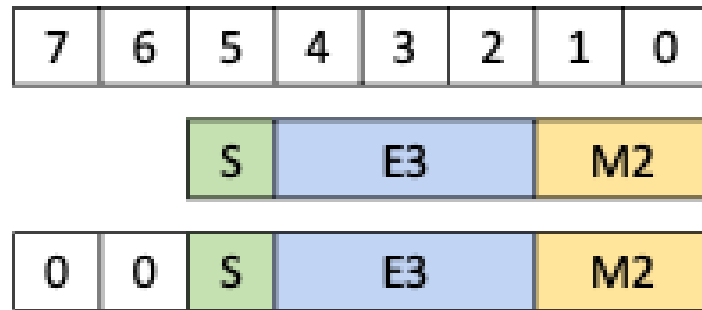


Figure 197: Individual decompression from 6-bit to 8-bit for E3M2 type

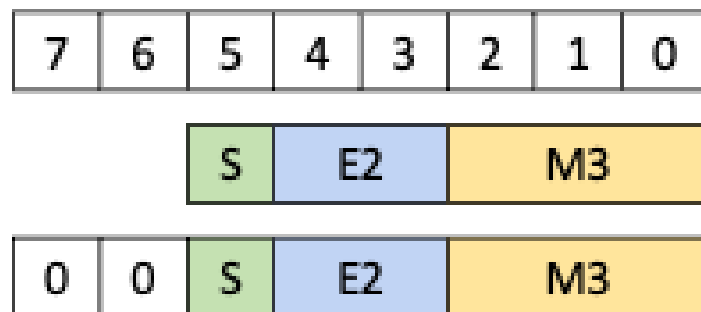


Figure 198: Individual decompression from 6-bit to 8-bit for E2M3 type

9.7.16.9.2 Tensorcore 5th Generation Instructions: tcgen05.cp

tcgen05.cp

Initiates an asynchronous copy operation from shared memory to the *Tensor Memory*.

Syntax

```
tcgen05.cp.cta_group.shape{.multicast}{.dst_fmt.src_fmt} [taddr], s-desc;

.cta_group = { .cta_group::1, .cta_group::2 }
.src_fmt   = { .b6x16_p32 , .b4x16_p64 }
.dst_fmt    = { .b8x16 }
.shape      = { .128x256b, .4x256b, .128x128b, .64x128b**, .32x128b*** }
.multicast  = { .warp2::02_13** , .warp2::01_23** , .warp4*** }
```

Description

Instruction `tcgen05.cp` initiates an asynchronous copy operation from shared memory to the location specified by the address operand `taddr` in the *Tensor Memory*.

The 64-bit register operand `s-desc` is the matrix descriptor which represents the source matrix in the shared memory that needs to be copied. The format of the matrix descriptor is described in *Matrix Descriptors*.

The `.shape` qualifier indicates the dimension of data to be copied as described in the *Data Movement Shape*.

Qualifier `.cta_group` specifies the number of CTAs whose *Tensor Memory* is accessed when a single thread of a single CTA executes the `tcgen05.cp` instruction. When `.cta_group::1` is specified, the data is copied into the *Tensor Memory* of the current CTA. When `.cta_group::2` is specified, the data is copied into the *Tensor Memory* of both the current and the *peer CTAs*.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

When the qualifiers `.dst_fmt` and `.src_fmt` are specified, the data is decompressed from the source format `.src_fmt` in the shared memory to the destination format `.dst_fmt` in *Tensor Memory* by the copy operation. The details of source and the destination formats as specified in the section *Optional Decompression*.

Some of the `.shape` qualifiers require certain `.multicast` qualifiers.

1. `.64x128b` requires `.warp2::02_13` or `.warp2::01_23`
2. `.32x128b` requires `.warp4`

When the `.multicast` qualifier is specified as either `.warp2::02_13` or `.warp2::01_23` then the data being copied is multicasted into warp pairs and each warp in the warp pair receive half of the data. Warp pairs are formed as follows:

1. `.warp2::02_13` : warps 0 and 2 form a pair; warps 1 and 3 form a pair.
2. `.warp2::01_23` : warps 0 and 1 form a pair; warps 2 and 3 form a pair.

When the `.multicast` modifier is specified as `.warp4` then the data being copied is multicasted into all 4 warps.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ sm_100a
- ▶ sm_101a (Renamed to sm_110a from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ sm_100f or higher in the same family
 - ▶ sm_101f or higher in the same family (Renamed to sm_110f from PTX ISA version 9.0)
- ▶ sm_110f or higher in the same family

Examples

```
tcgen05.cp.cta_group::1.128x256b      [taddr0], sdesc0;
tcgen05.cp.cta_group::2.128x128b.b8x16.b6x16_p32 [taddr1], sdesc1;
tcgen05.cp.cta_group::1.64x128b.warp_x2::02_13  [taddr2], sdesc2;
```

9.7.16.9.3 Tensorcore 5th Generation Instructions: tcgen05.shift

tcgen05.shift

Asynchronously shift down the rows of the matrix in the *Tensor Memory* for a warp.

Syntax

```
tcgen05.shift.cta_group.down [taddr];
.cta_group = { .cta_group::1, .cta_group::2 }
```

Description

Instruction `tcgen05.shift` is an asynchronous instruction which initiates the shifting of 32-byte elements downwards across all the rows, except the last, by one row. The address operand `taddr` specifies the base address of the matrix in the *Tensor Memory* whose rows must be down shifted.

The lane of the address operand `taddr` must be aligned to 32.

Qualifier `.cta_group` specifies the number of CTAs whose *Tensor Memory* is touched when a single thread of a single CTA executes the `tcgen05.shift` instruction. When `.cta_group::1` is specified, the shift operation is performed in the *Tensor Memory* of the current CTA. When `.cta_group::2` is specified, the shift operation is performed in the *Tensor Memory* of both the current and the *peer CTAs*.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ sm_100a
- ▶ sm_101a (Renamed to sm_110a from PTX ISA version 9.0)
- ▶ sm_103a
- ▶ sm_110a

Examples

```
tcgen05.shift.down.cta_group::1 [taddr0];  
tcgen05.shift.down.cta_group::2 [taddr1];
```

9.7.16.10 TensorCore 5th Generation Matrix Multiply and accumulate Operations

The 5th generation of TensorCore operations of shape $M \times N \times K$ perform matrix multiplication and accumulation of the form:

$$D = A * B + D$$

where:

- ▶ the A matrix has shape $M \times K$, in either Tensor Memory or Shared Memory
- ▶ the B matrix has shape $K \times N$, in Shared Memory of the current CTA and optionally in peer CTA
- ▶ the D matrix is of the shape $M \times N$, in Tensor Memory

Optionally an input predicate can be used to disable the input from the accumulator matrix and the following operation can be performed as

$$D = A * B$$

The matrix multiplication and accumulation operations are categorized into various kinds based on input types and the throughput of the multiplication operation. The following shows the different kinds of MMA operations that are supported:

1. f16 : supports f16 and bf16 input types.
2. tf32 : supports tf32 input types.
3. f8f6f4 : supports all input combinations of f8, f6 and f4 types.
4. i8 : supports signed and unsigned 8-bit integer input types.
5. mxf8f6f4/mxf4 : supports mx-floating points input types.
6. mxf4nvf4 : supports mxf4 type and a custom NVIDIA floating-point type for inputs where the type of the vector elements is 4 bits and requires a common scaling factor to form the complete floating-point type, similar to other mx-types.

Optionally, the 5th generation of TensorCore MMAs support dense and sparse matrix A. [Sparse Matrices](#) describes the details of the sparse matrices.

Some of the MMA-kinds requires scaling of input matrices from memory to form the matrix A and matrix B before performing the MMA operation. [Block Scaling](#) describes the details of the scaling of matrices.

The following table show the various matrices involved in the MMA operations and the memory in which they can reside:

Matrix Type	Memory
A	Tensor Memory OR Shared Memory
B	Shared Memory
D	Tensor Memory
Sparse Meta Data	
A-Scale / B-Scale	

A sequence of MMA instructions may reuse the same A matrix with a sequence of B matrices or may reuse the same B matrix with a sequence of A matrices. In these patterns the TensorCore may be able to load the unchanged matrix once and reuse it through the sequence without multiple reloads. The A or B matrices are loaded into a TensorCore collector buffer (i.e., special cache).

An MMA instruction has an optional `collector` qualifier to specify when an A or B matrix is new to the sequence and should be loaded, unchanged within the sequence and should be reused, or the last use in the sequence and should be discarded. The `collector` qualifier is used to give the TensorCore permission to reuse a previously loaded A or B matrix; however reuse is opportunistic in that the TensorCore may reload a matrix even when it has permission to reuse that matrix. Thus, the source memory of an A or B matrix must not be modified while the MMA instruction using those matrices has not completed - regardless of `collector` qualifier permissions.

The 5th generation of TensorCore MMAs can be used for general matrix multiplication OR for convolution operations. In case of convolutions, the activations can be stored in either matrix A or matrix B while the weights will be stored in the other matrix.

Activation Matrix	Weights Matrix	Name of the op	Instruction Name	Collector Buffer Applicability
A	B	Activation Stationary	(default <code>tcgen05.mma</code>)	Collector buffer is applicable on matrix A
B	A	Weights Stationary	<code>.ws</code>	Collector buffer is applicable on matrix B

9.7.16.10.1 Transpose and Negate operations

The matrices A and B can be transposed by specifying the Transpose A Matrix and Transpose B Matrix bits in the instruction descriptor respectively.

The elements of the matrices A and B can be negated by specifying the Negate A Matrix and Negate B Matrix bits in the instruction descriptor respectively.

The support for Transpose and Negate operation for various MMA-Kind are shown in Table 51.

Table 51: Transpose and Negate operation for various MMA-Kind

MMA-Kind	Is Transpose A/B supported	Is Negate A/B supported
<code>.kind::tf32</code>	Yes	Yes
<code>.kind::f16</code>	Yes	Yes
<code>.kind::f8f6f4</code>	Yes	Yes
<code>.kind::mxf8f6f4</code>	Yes	Yes
<code>.kind::i8</code>	Yes	No
<code>.kind::mxf4</code>	No	Yes
<code>.kind::mxf4nvf4</code>	No	Yes

For `.kind::tf32`, the transpose operations on matrices A and B are supported only with 128B swizzling mode with 32B swizzle-atomicity.

For all other MMA-Kinds, the transpose operations on matrices A and B are not supported on 128B swizzling mode with 32B swizzle-atomicity.

Table 52 shows the valid combinations of N shape with `.cta_group` qualifier for 8bit transpose B.

Table 52: Various combinations of N shape with `.cta_group` qualifier for 8bit transpose B

<code>.cta_group</code>	N shape
1	$16 \leq N \leq 256$, step 16
2	$32 \leq N \leq 256$, step 32

9.7.16.10.2 Matrix Layout Organization

Table 53 describes the major-ness used for different matrices.

Table 53: Major-ness for different matrices

Ma-trix	Residing in Mem-ory	Default Major-ness
D	Tensor Memory	Row-Major
A	Tensor Memory	
	Shared Memory	Depends on swizzling mode. Refer <i>Shared Memory Layout and Swizzling</i>
B	Shared Memory	

9.7.16.10.3 Valid Combinations of Type-Size, Major-ness and Swizzling

Table 54: Valid Combinations of Type-Size, Major-ness and Swizzling

Type-Size	Major-ness	Ma-trix	Supported Swizzle
4-bit, 6-bit, 8-bit, 16-bit, 32-bit	Row	A	All swizzling modes
	Column	B	
8-bit 16-bit	Column (transpose)	A	All except 128B swizzling with 32B atomicity
	Row (transpose)	B	
32-bit	Column (transpose)	A	Only 128B swizzling with 32B atomicity
	Row (transpose)	B	

9.7.16.10.4 Packing formats of elements in Tensor and Shared memory

9.7.16.10.4.1 Packing format for matrix D in Tensor Memory

The sub-word elements of matrix D are expected not to be packed within a 32-bit Tensor Memory word. For example, if the type of elements of the matrix D is 16 bits then a Tensor Memory word would contain a single 16-bit element in its lower 16 bits.

9.7.16.10.4.2 Packing format for matrix A and B

The 6-bit and 4-bit floating point types have different packing format requirements for different MMA kinds in both Tensor memory and Shared memory. The requirements are as follows.

9.7.16.10.4.3 Packing format used for matrix A by `.kind::mxf8f6f4` in Tensor Memory

The individual 4-bit and the 6-bit floating point type elements must be packed in an 8-bit container in Tensor memory as shown below. The 8-bit containers must be contiguously packed in a 32-bit Tensor Memory word. For example, if the type of elements of the matrix A is 6 bits then 4 consecutive A elements should be packed in one 32-bit Tensor Memory word.

- 4-bit packing format as shown in [Figure 199](#)

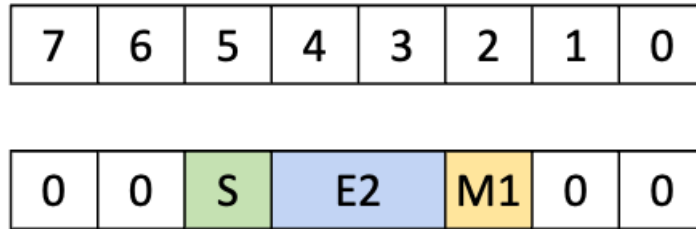


Figure 199: 4-bit packing format with type E2M1

- 6-bit packing format
 - Type E3M2 as shown in [Figure 200](#)

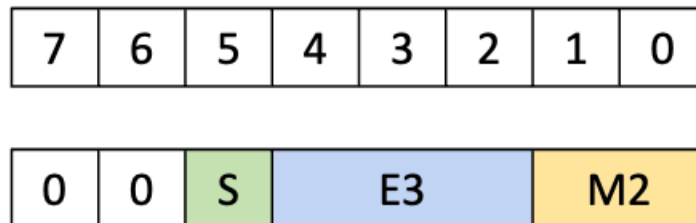


Figure 200: 6-bit packing format with type E3M2

- Type E2M3 as shown in [Figure 201](#)

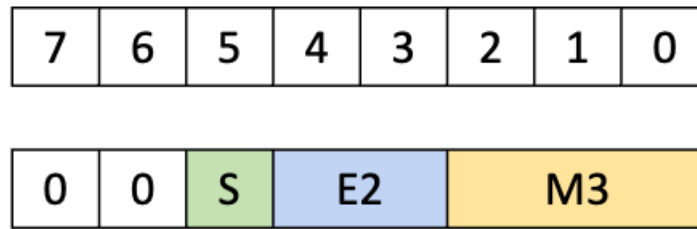


Figure 201: 6-bit packing format with type E2M3

9.7.16.10.4.4 Packing format used for matrix A and B by `.kind:::mx8f6f4` in Shared Memory

The 4-bit and 6-bit floating point elements in shared memory must be contiguously packed along with padding as follows.

- 4-bit packing format as shown in Figure 202

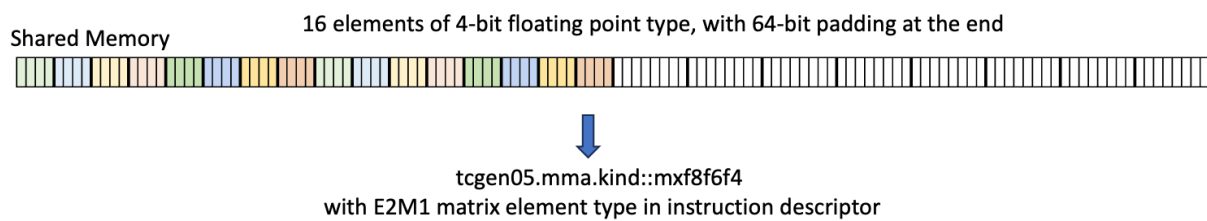


Figure 202: 4-bit packing format

- 6-bit packing format as shown in Figure 203

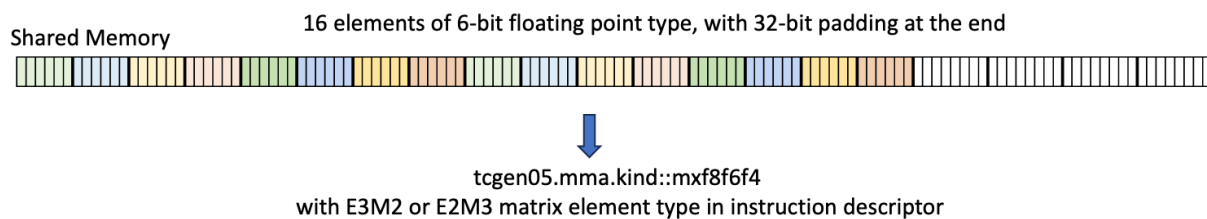


Figure 203: 6-bit packing format

9.7.16.10.4.5 Packing format used for matrix A by .kind::mxf4 and .kind::mxf4nvf4 in Tensor Memory

Two 4-bit floating point type elements must be packed in an 8-bit container in Tensor memory as shown in Figure 204 for mxf4.

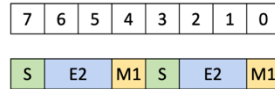


Figure 204: 4-bit packing format with type E2M1

9.7.16.10.4.6 Packing format used for matrix A and B by .kind::mxf4 and .kind::mxf4nvf4 in Shared Memory

The packing format for 4-bit floating point elements in shared memory is to pack two 4-bit elements in a 8-bit container, with no padding.

9.7.16.10.5 Data Path Layout Organization

Different MMA variants access the tensor memory with different layout organization. The following table lists the various layouts:

M	cta_group	A-Sparsity	Is .ws mode	Datapath organization	Layout ID	Tensor Memory Lane Alignment
32	::1	Either	Yes	1x4	<i>Layout G</i>	0
64	::1	Either	Yes	2x3	<i>Layout E</i>	0
64	::1	Either	No	4x1 (1/2 datapath utilized)	<i>Layout F</i>	0 or 16
128	::1	Either	Either	4x1	<i>Layout D</i>	0
128	::2	Dense	N/A	2x2	<i>Layout B</i>	0
128	::2	Sparse	N/A	4x1 (1/2 datapath utilized)	<i>Layout C</i>	0 or 16
256	::2	Either	N/A	4x1	<i>Layout A</i>	0

The layouts which utilize only half the datapath lanes, i.e., *Layout F* and *Layout C*, must use the same Tensor Memory lane alignment across matrices A, D and the sparsity metadata matrix.

The following shows the warps that can access the Tensor Memory regions via tcgen05.ld / tcgen05.st along with the addresses for various Tensor Memory Layouts.

9.7.16.10.5.1 Layout A (M = 256)

Layout organization for M = 256 is shown in Figure 205.

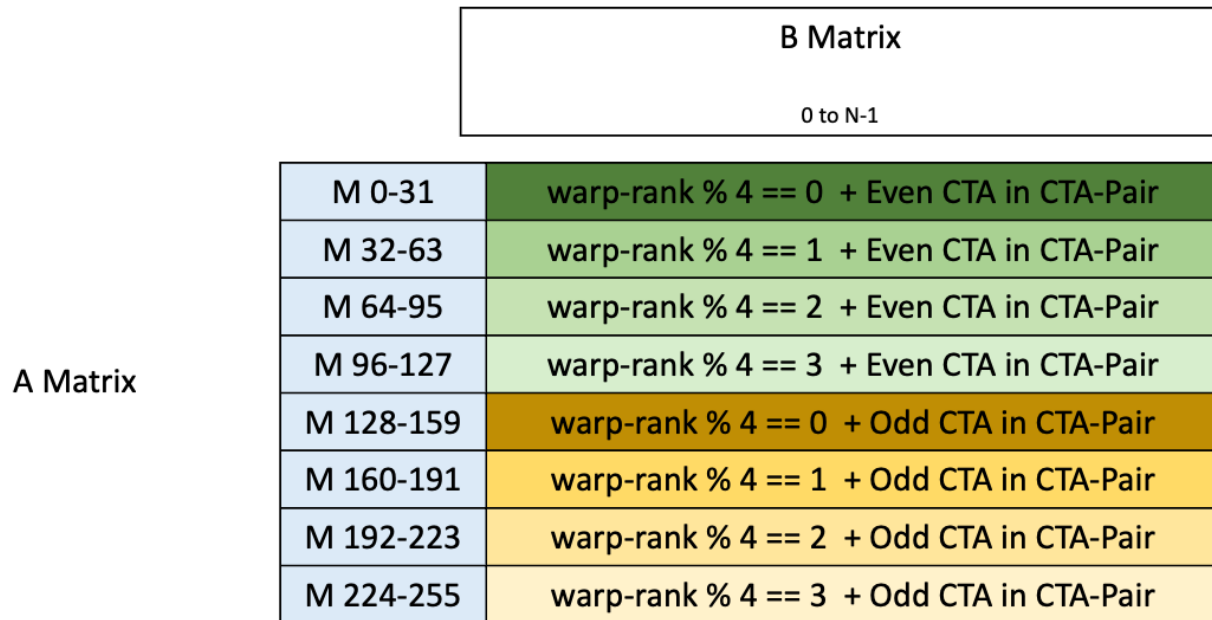


Figure 205: Layout organization for M = 256

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in Figure 206

9.7.16.10.5.2 Layout B (M = 128 + cta-group::2 + Dense A matrix)

Layout organization for M = 128 + .cta_group::2 + Dense A matrix is shown in Figure 207.

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in Figure 208

9.7.16.10.5.3 Layout C (M = 128 + cta-group::2 + Sparse A matrix)

Layout organization for M = 128 + .cta_group::2 + Sparse A matrix is shown in Figure 209.

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in Figure 210

		B Matrix	
		0 to N-1	
A Matrix	M 0-31	0x0000.0XXX	
	M 32-63	0x0020.0XXX	
	M 64-95	0x0040.0XXX	
	M 96-127	0x0060.0XXX	
	M 128-159	0x0080.0XXX	
	M 160-191	0x00A0.0XXX	
	M 192-223	0x00C0.0XXX	
	M 224-255	0x00E0.0XXX	

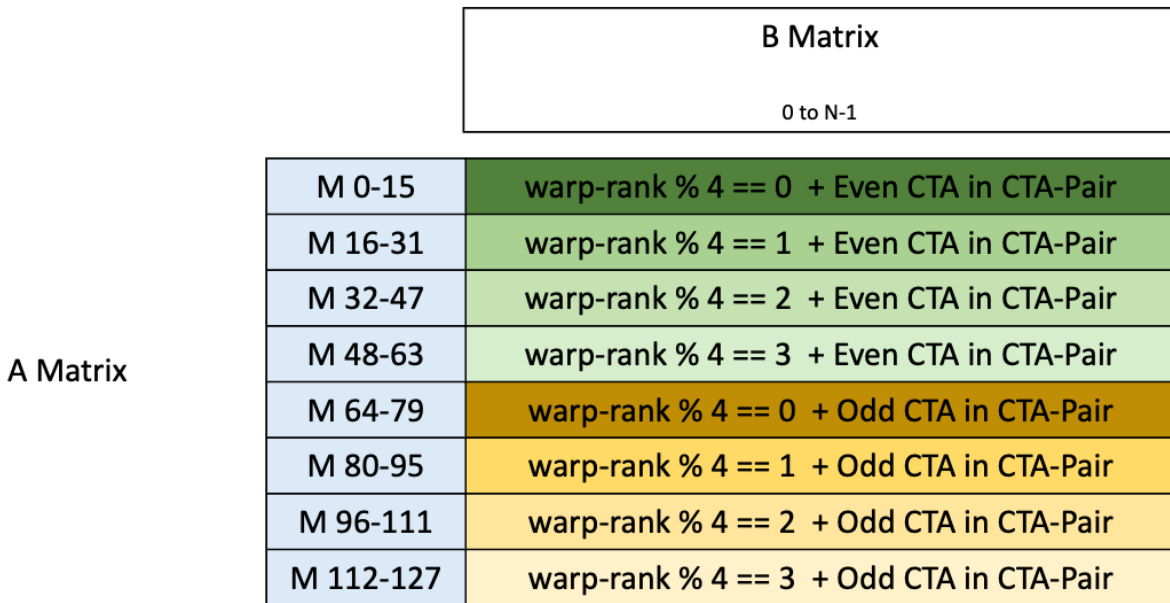
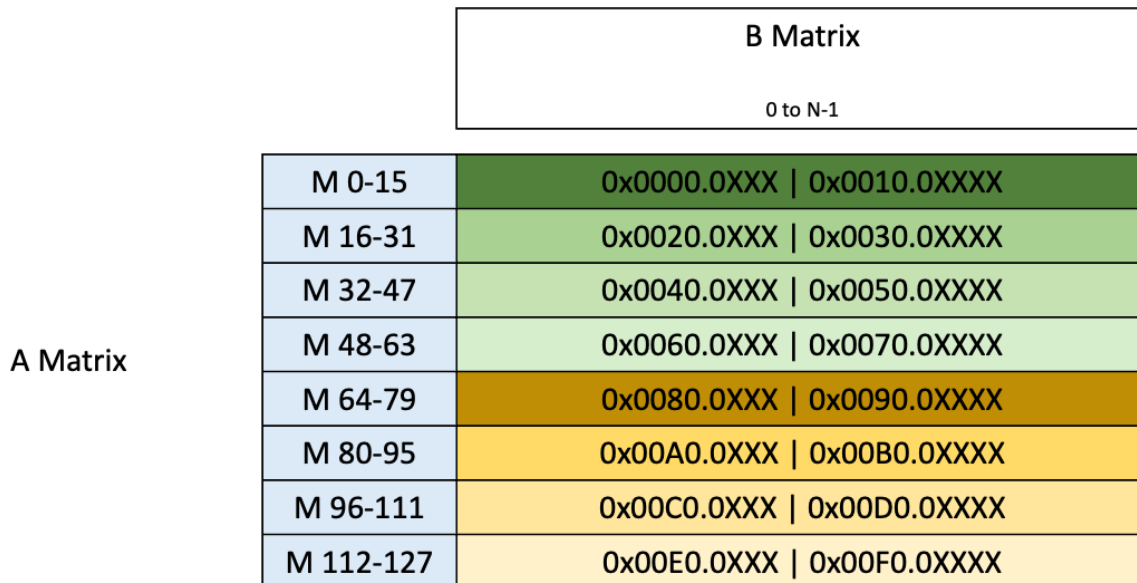
Figure 206: Addresses to use in `tcgen05.ld` / `tcgen05.st`

B Matrix		B Matrix	
0 to (N/2)-1		(N/2) to N-1	
A Matrix	M 0-31	warp-rank % 4 == 0 + Even CTA in CTA-Pair	warp-rank % 4 == 2 + Even CTA in CTA-Pair
	M 32-63	warp-rank % 4 == 1 + Even CTA in CTA-Pair	warp-rank % 4 == 3 + Even CTA in CTA-Pair
	M 64-95	warp-rank % 4 == 0 + Odd CTA in CTA-Pair	warp-rank % 4 == 2 + Odd CTA in CTA-Pair
	M 96-127	warp-rank % 4 == 1 + Odd CTA in CTA-Pair	warp-rank % 4 == 3 + Odd CTA in CTA-Pair

Figure 207: Layout organization for $M = 128 + .cta_group::2$ + Dense A matrix

B Matrix		B Matrix	
0 to (N/2)-1		(N/2) to N-1	
A Matrix	M 0-31	0x0000.0XXX	0x0000.0XXX
	M 32-63	0x0020.0XXX	0x0020.0XXX
	M 64-95	0x0040.0XXX	0x0040.0XXX
	M 96-127	0x0060.0XXX	0x0060.0XXX

Figure 208: Addresses to use in `tcgen05.ld` / `tcgen05.st`

Figure 209: Layout organization for $M = 128 + .cta_group::2 + \text{Sparse A matrix}$ Figure 210: Addresses to use in `tcgen05.ld` / `tcgen05.st`

9.7.16.10.5.4 Layout D ($M = 128 + \text{cta_group}::1$)

Layout organization for $M = 128 + \text{cta_group}::1$ is shown in Figure 211.

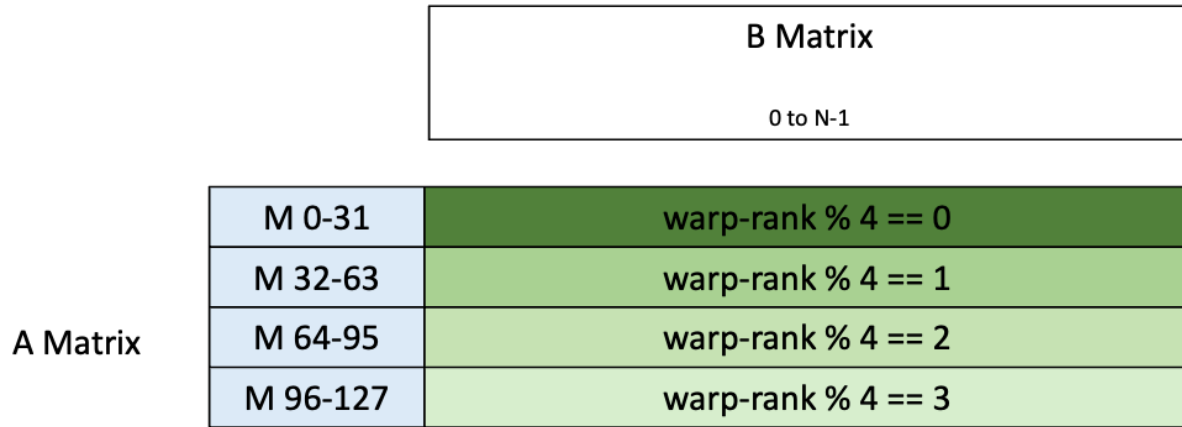


Figure 211: Layout organization for $M = 128 + \text{cta_group}::1$

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in Figure 212

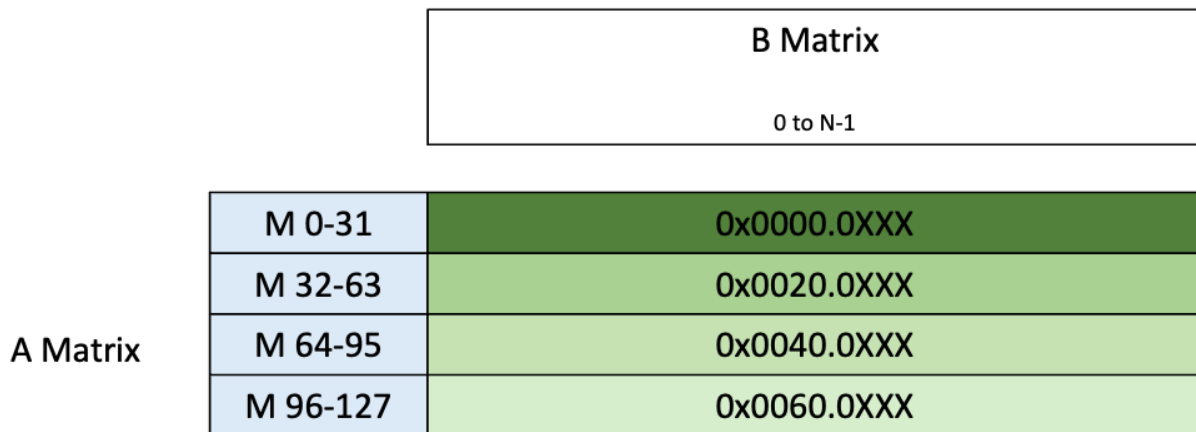


Figure 212: Addresses to use in `tcgen05.ld` / `tcgen05.st`

9.7.16.10.5.5 Layout E (M = 64 + .ws mode)

Layout organization for M = 64 + .ws mode is shown in Figure 213.

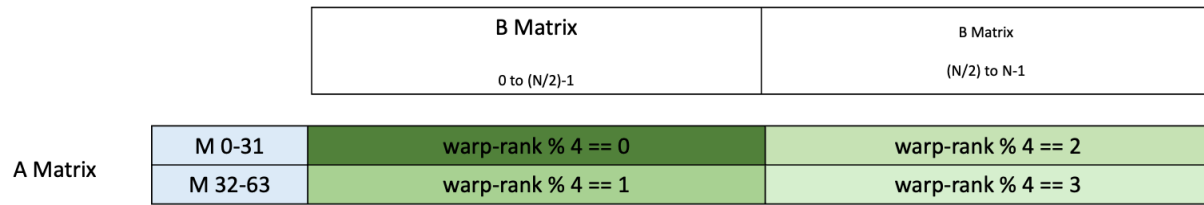


Figure 213: Layout organization for M = 64 + .ws mode

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in Figure 214

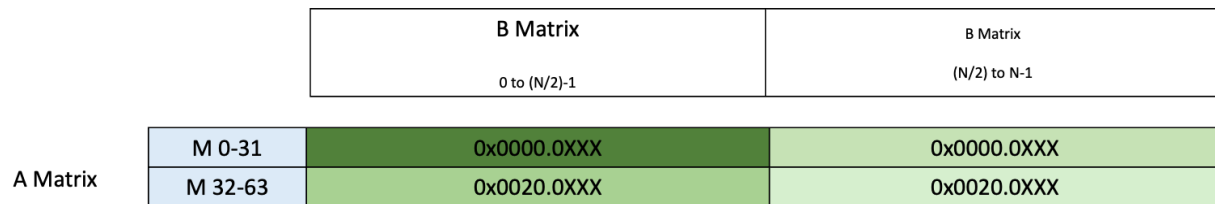


Figure 214: Addresses to use in `tcgen05.ld` / `tcgen05.st`

9.7.16.10.5.6 Layout F (M = 64 + non .ws mode)

Layout organization for M = 64 + non .ws mode is shown in Figure 215.

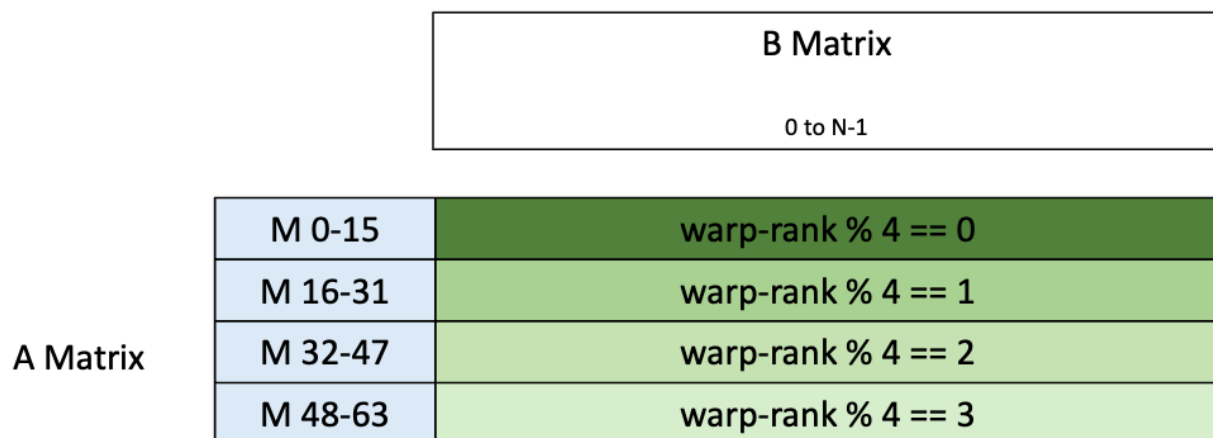
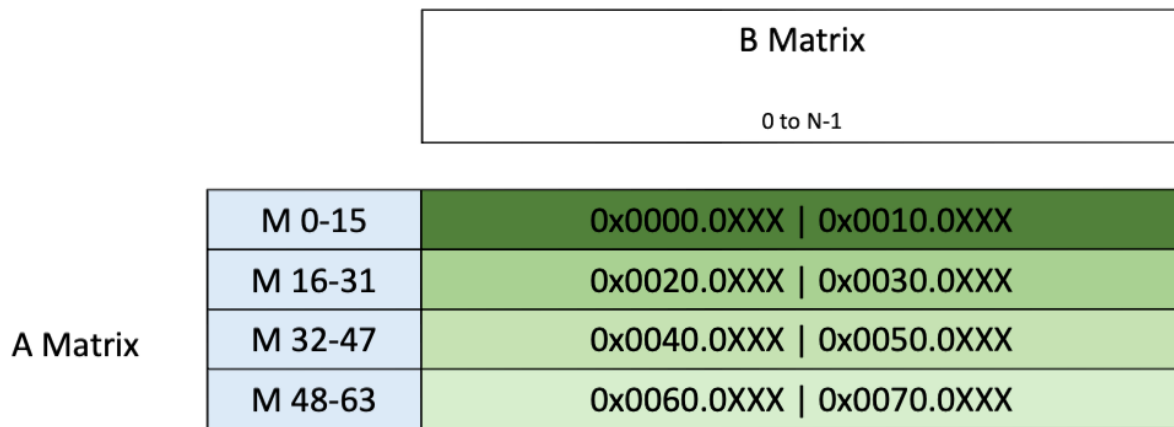


Figure 215: Layout organization for M = 64 + non .ws mode

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in Figure 216

Figure 216: Addresses to use in `tcgen05.ld` / `tcgen05.st`**9.7.16.10.5.7 Layout G (M = 32)**

Layout organization for M = 32 is shown in Figure 217.

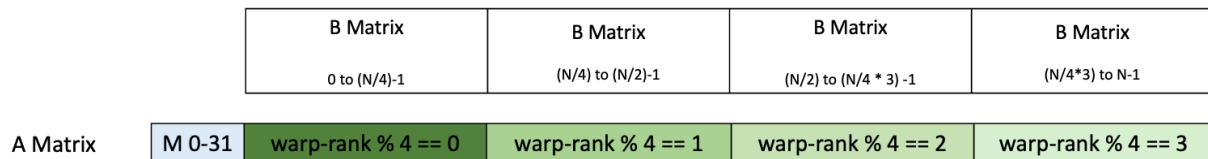
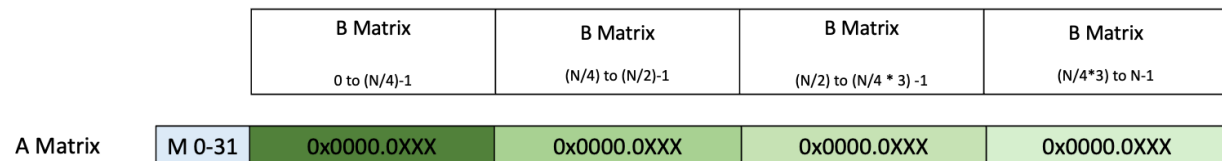


Figure 217: Layout organization for M = 32

Addresses for the above region to be used in `tcgen05.ld` / `tcgen05.st` is shown in Figure 218

Figure 218: Addresses to use in `tcgen05.ld` / `tcgen05.st`

9.7.16.10.6 Shared Memory Layout and Swizzling

If the bit `Transpose A Matrix / Transpose B Matrix` in the *Instruction descriptor* is 0, then *K-major* is used for matrix A / B respectively. If the bit `Transpose A Matrix` in the *Instruction descriptor* is 1 then *M-major* is used for matrix A. If the bit `Transpose B Matrix` in the *Instruction descriptor* is 1, then *N-major* is used for matrix B.

In a column-major default BLAS library such as cuBLAS, the matrices A and B with and without transpose can be classified as either *K-Major* or *M-or-N-Major* as shown in the following table:

	Non-Transposed	Transposed
A	K-major	M-major
B	K-major	N-major

To avoid confusion with A, B, row-major, col-major, transpose, and non-transpose, we will use *MN-Major* and *K-Major* throughout this section.

The matrices in the shared memory are made up of one or more “swizzle layout atom”. The exact layout of these swizzle atoms depends on the swizzling mode, swizzle-atomicity, and the leading dimension. The layout of the swizzle are shown in [Table 55](#)

Table 55: Layout for swizzle atoms

Swizzling mode and Swizzle-Atomicity	Leading Dimension	Swizzle atom layout (128b element)
128B Swizzling with 32B atomicity	M/N	8x4
	–	–
128B Swizzling with 16B atomicity	M/N	8x8
	K	8x8
64B Swizzling Mode	M/N	4x8
	K	8x4
32B Swizzling Mode	M/N	2x8
	K	8x2
None	M/N	1x8
	K	8x1

The above shapes are for elements of size 128 bits. For smaller element sizes, the same shapes would get multiplied along the leading dimension by a factor of $128 / \text{sizeof_bits}(\text{Element})$. For example, 128B MN major swizzle atom would have a shape of $(8 * (128/32)) \times 8 = 32 \times 8$ for tf32 tensor core inputs.

Some example Layouts of $M \times K$ or $K \times N$ matrices with various swizzling modes, and are in units of 128b elements as shown by each colored cell as shown in [Figure 219](#), [Figure 220](#), [Figure 221](#), [Figure 222](#), [Figure 223](#), [Figure 224](#), [Figure 225](#), [Figure 226](#), [Figure 227](#).

Following are some of the examples of the 128B swizzling layout for tf32 element type.

- K-Major: [Figure 228](#)

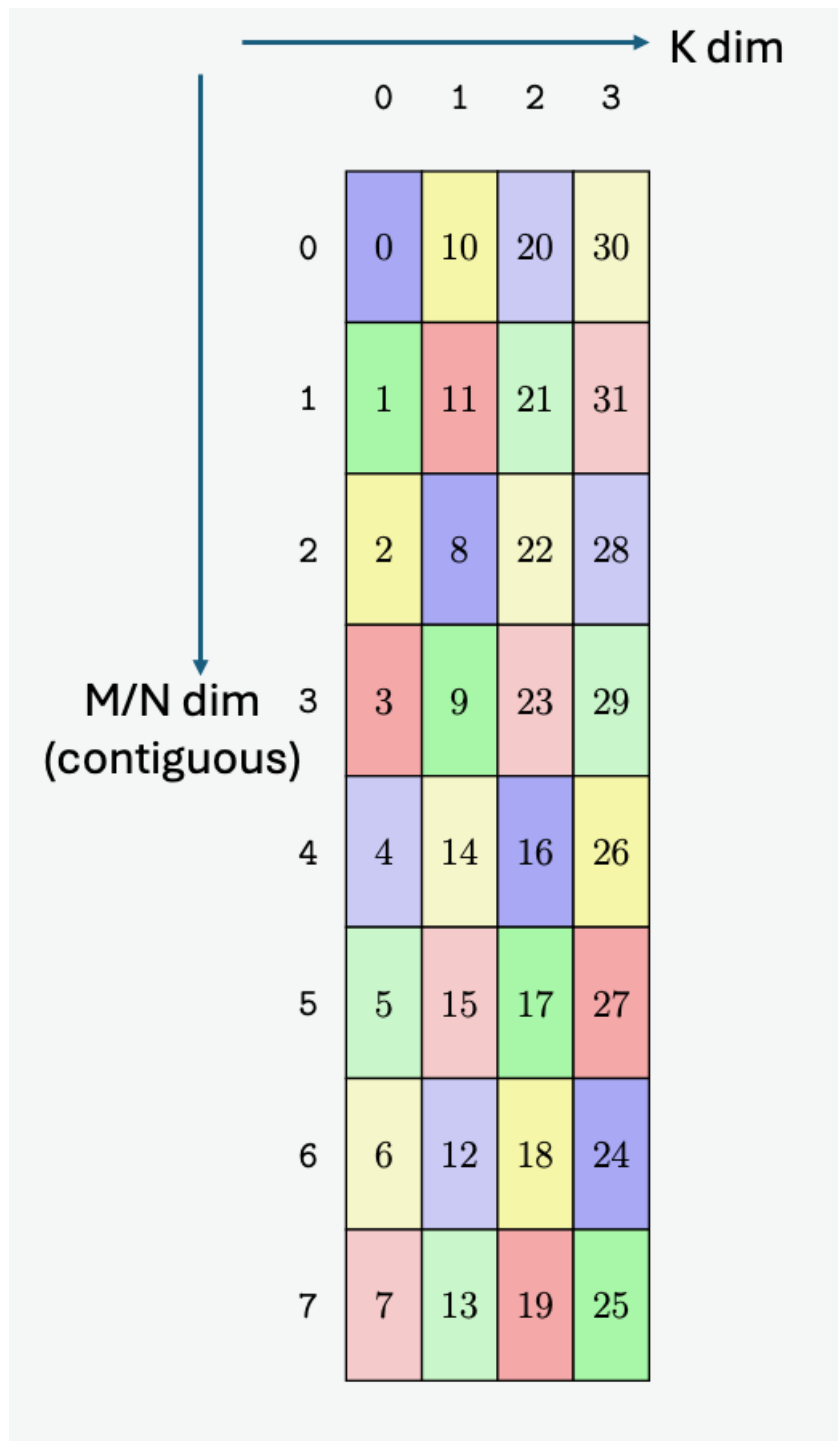


Figure 219: MN major 128B swizzling with 32B atomicity

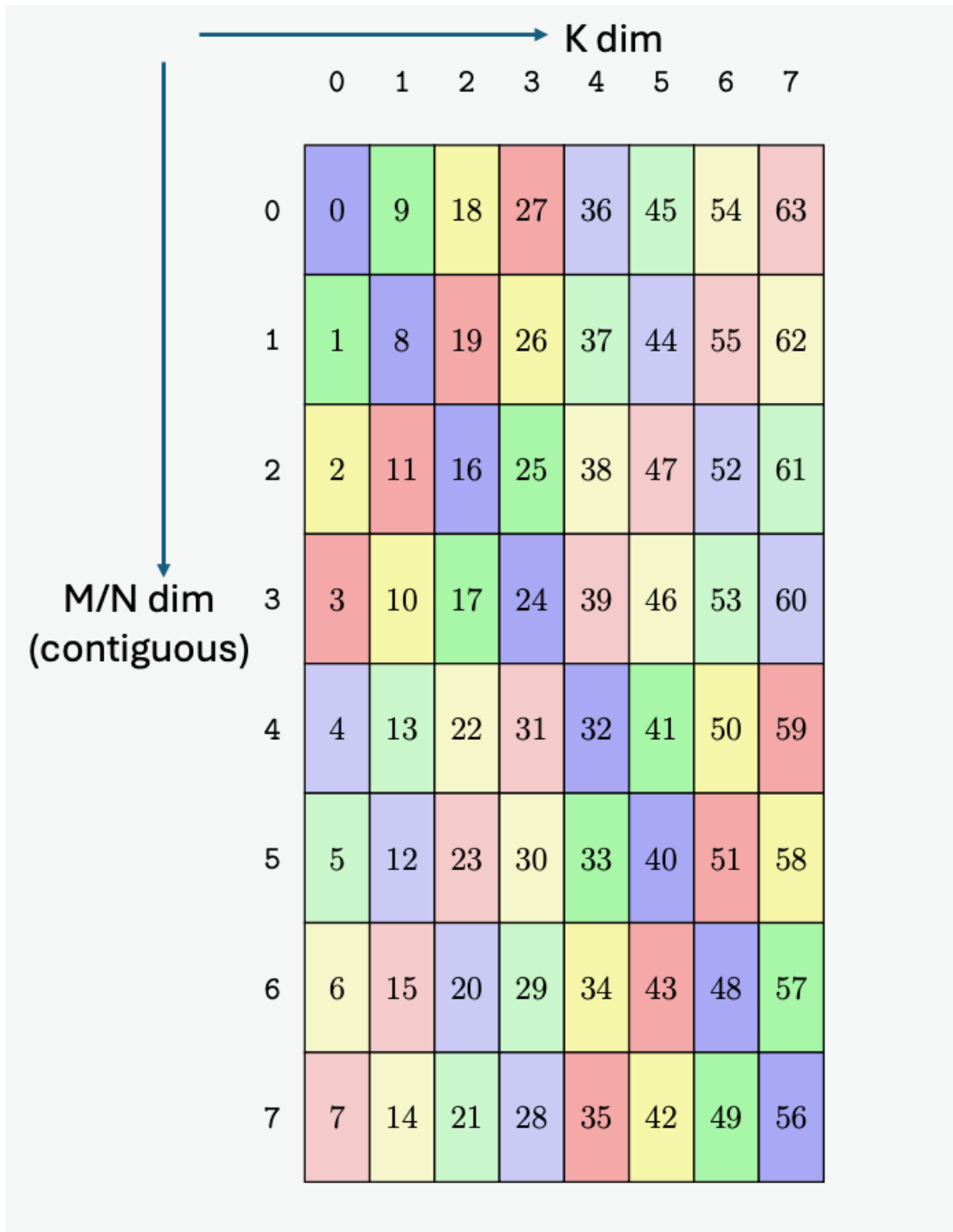


Figure 220: MN major 128B swizzling

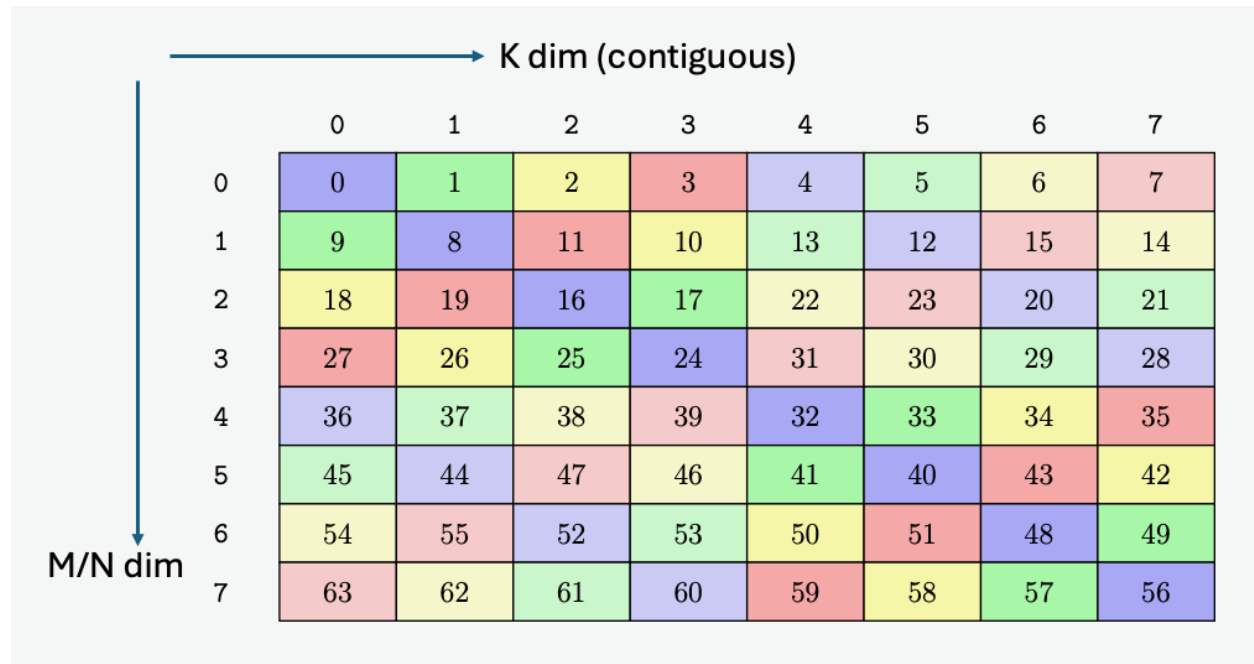


Figure 221: K major 128B swizzling

► MN-Major: [Figure 229](#)

9.7.16.10.7 Block Scaling

The `tcgen05.mma` instructions with the following `.kind` qualifier:

- `.kind::mxf8f6f4`
- `.kind::mxf4`
- `.kind::mxf4nvf4`

perform matrix multiplication with block scaling. This operation has the following form:

$$(A * \text{scale_A}) * (B * \text{scale_B}) + D$$

where `scale_A` and `scale_B` are matrices residing in *Tensor Memory*.

For a `scale_A` matrix of shape $M \times SFA_N$, each row of matrix A is divided into SFA_N number of chunks and each chunk of a row is multiplied with the corresponding element in the SF_A of the same row.

Similarly, for a `scale_B` matrix of shape $SFB_M \times N$, each column of matrix B is divided into the SFB_M number of chunks and each chunk of a column is multiplied with the corresponding element in the SF_B of the same column.

Scale factors for A and B matrices need to be duplicated to all 32 lane partitions of tensor memory.

[Figure 230](#) shows an example of `tcgen05.mma` with block scaling of `scale_vec::2X`.

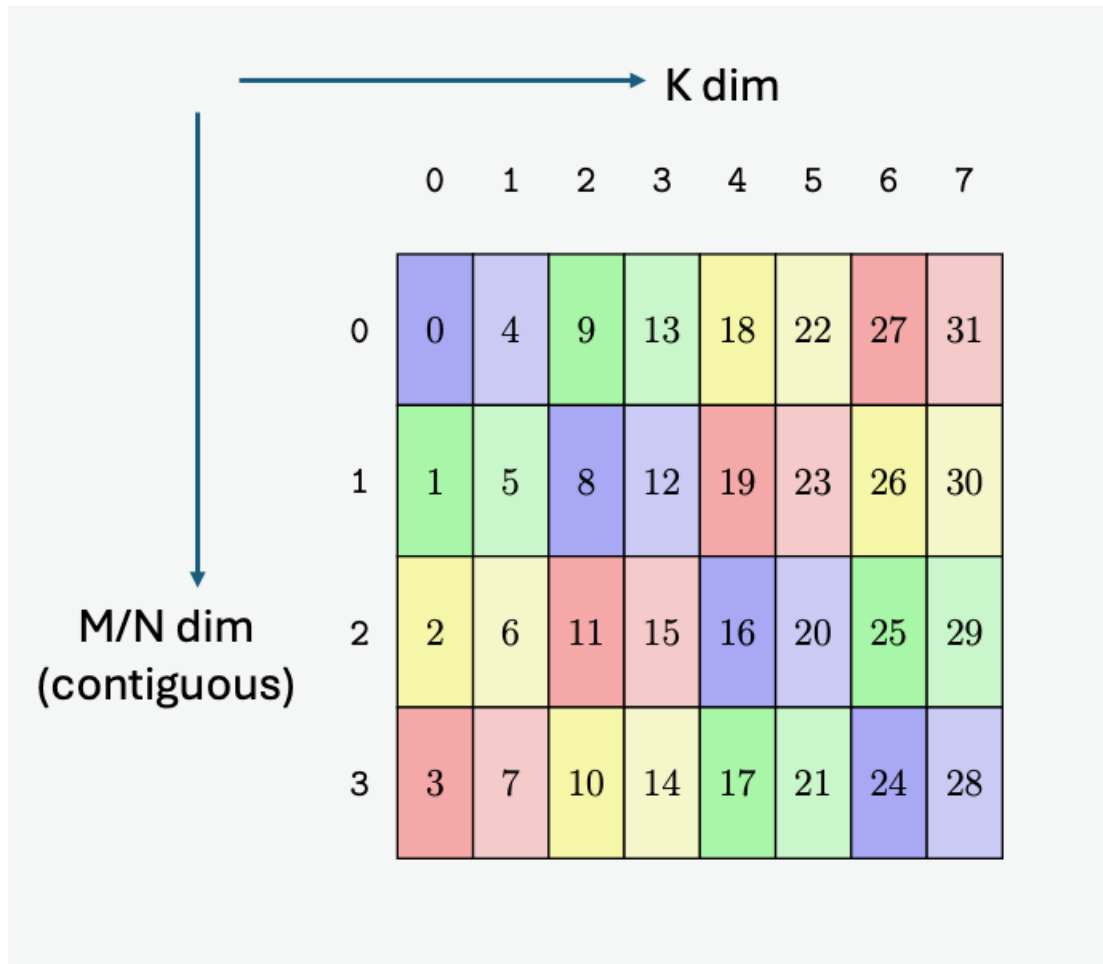


Figure 222: MN major 64B swizzling

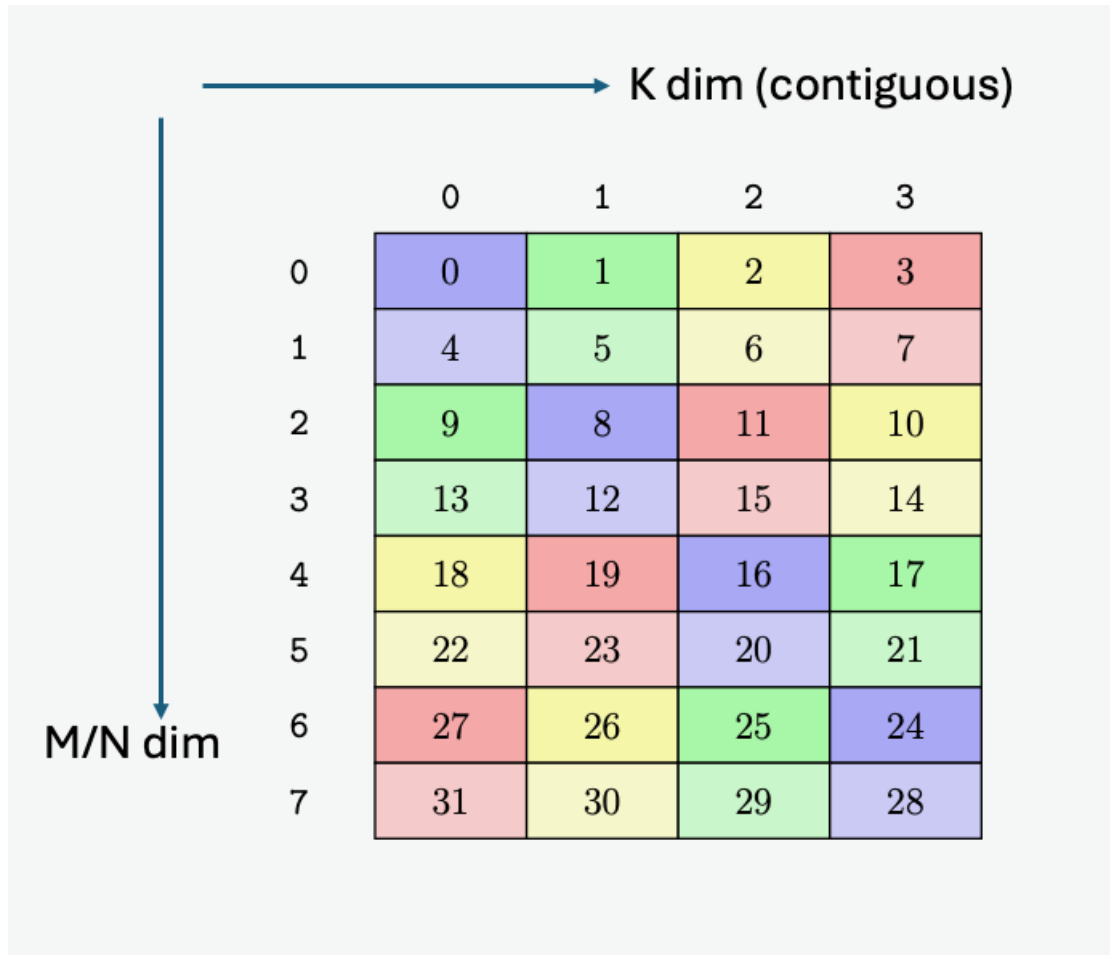


Figure 223: K major 64B swizzling

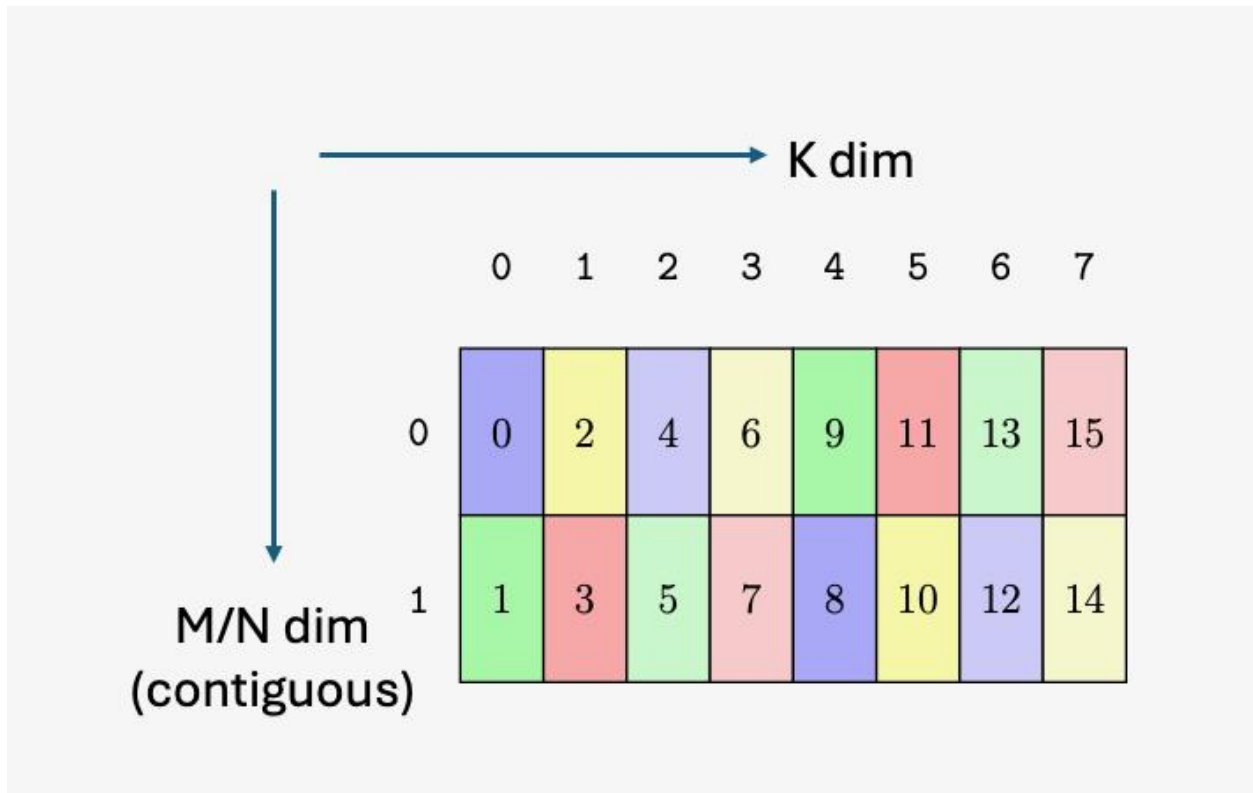


Figure 224: MN major 32B swizzling

9.7.16.10.7.1 Valid combinations of `scale_vectorsize` with types and MMA-Kind

The shape of `scale_A` and `scale_B` matrices depend on the `.scale_vectorsize` as shown in Table 56.

Table 56: Valid combinations of `scale_vectorsize` and shapes

<code>.scale_vectorsize</code>	<code>.kind::*</code>	K	Shape of <code>scale_A</code>	Shape of <code>scale_B</code>
<code>.scale_vec::1X</code>	<code>.kind::mxf8f6f4</code>	All supported values of K	M x 1	1 x N
<code>.scale_vec::2X</code>	<code>.kind::mxf4, .kind::mxf4nvf4</code>	All supported values of K	M x 2	2 x N
<code>.scale_vec::4X</code>	<code>.kind::mxf4nvf4</code>	All supported values of K	M x 4	4 x N
<code>.block16</code>	<code>.kind::mxf4nvf4</code>	K = 96	M x 6	6 x N
		All supported values of K except 96	M x 4	4 x N
<code>.block32</code>	<code>.kind::mxf4, .kind::mxf4nvf4</code>	K = 96	M x 3	3 x N
		All supported values of K except 96	M x 2	2 x N
	<code>.kind::mxf8f6f4</code>	All supported values of K	M x 1	1 x N

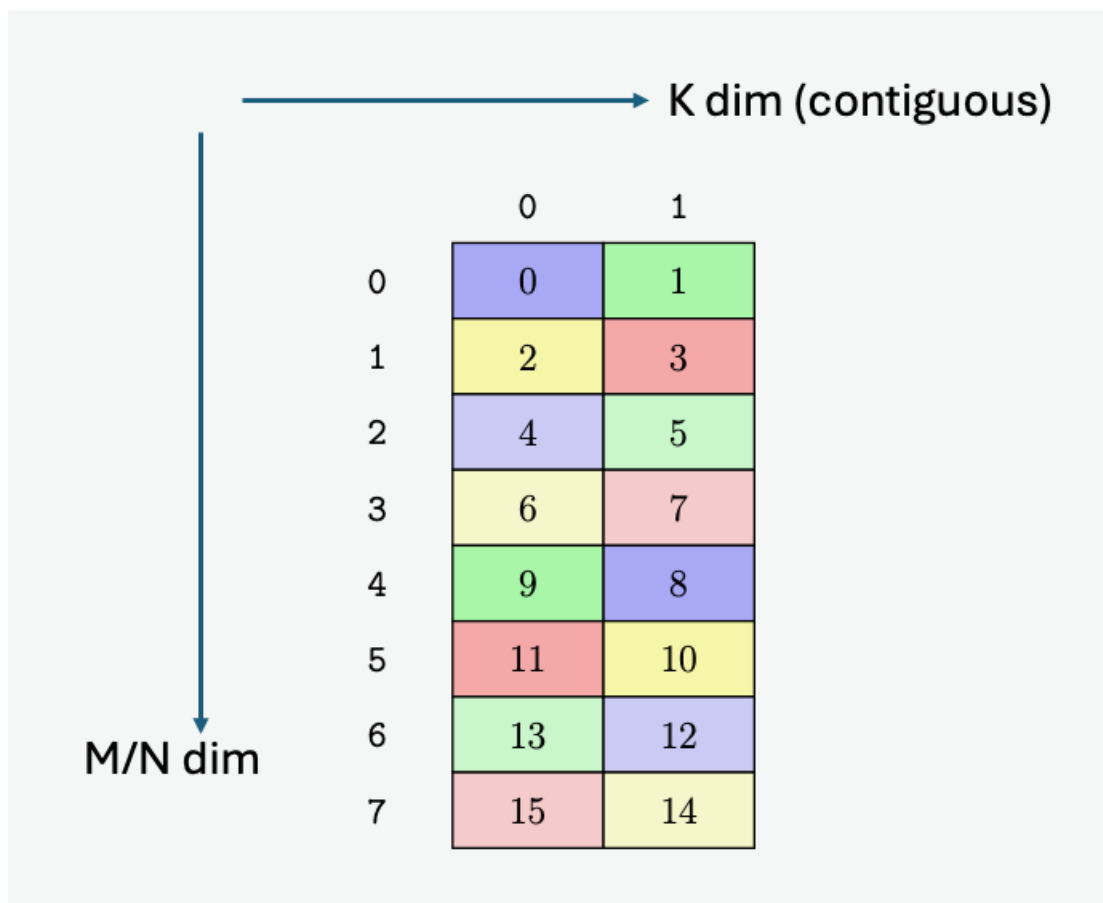


Figure 225: K major 32B swizzling

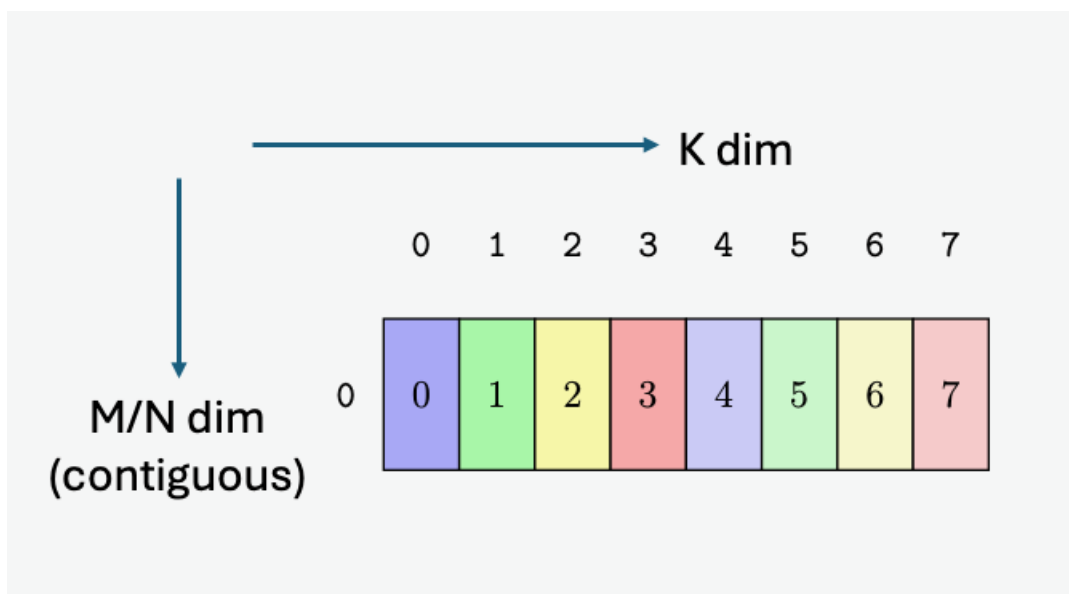


Figure 226: MN major no-swizzling mode

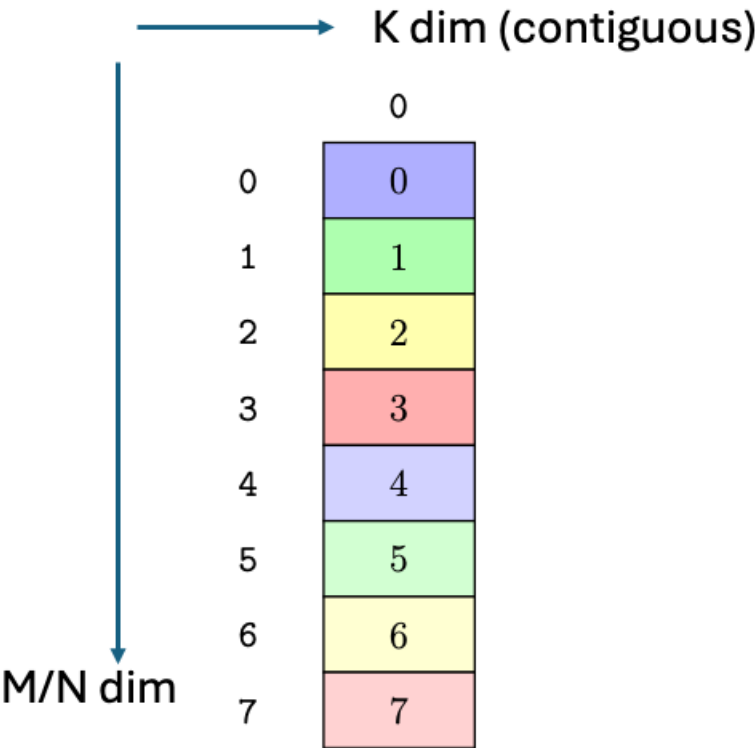


Figure 227: K major no-swizzling mode

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
1	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67
2	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103
3	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139
4	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175
5	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211
6	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247
7	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283

Figure 228: K major

	0	1	2	3	4	5	6	7
0	0	36	72	108	144	180	216	252
1	1	37	73	109	145	181	217	253
2	2	38	74	110	146	182	218	254
3	3	39	75	111	147	183	219	255
4	4	32	76	104	148	176	220	248
5	5	33	77	105	149	177	221	249
6	6	34	78	106	150	178	222	250
7	7	35	79	107	151	179	223	251
8	8	44	64	100	152	188	208	244
9	9	45	65	101	153	189	209	245
10	10	46	66	102	154	190	210	246
11	11	47	67	103	155	191	211	247
12	12	40	68	96	156	184	212	240
13	13	41	69	97	157	185	213	241
14	14	42	70	98	158	186	214	242
15	15	43	71	99	159	187	215	243
16	16	52	88	124	128	164	200	236
17	17	53	89	125	129	165	201	237
18	18	54	90	126	130	166	202	238
19	19	55	91	127	131	167	203	239
20	20	48	92	120	132	160	204	232
21	21	49	93	121	133	161	205	233
22	22	50	94	122	134	162	206	234
23	23	51	95	123	135	163	207	235
24	24	60	80	116	136	172	192	228
25	25	61	81	117	137	173	193	229
26	26	62	82	118	138	174	194	230
27	27	63	83	119	139	175	195	231
28	28	56	84	112	140	168	196	224
29	29	57	85	113	141	169	197	225
30	30	58	86	114	142	170	198	226
31	31	59	87	115	143	171	199	227

Figure 229: MN major

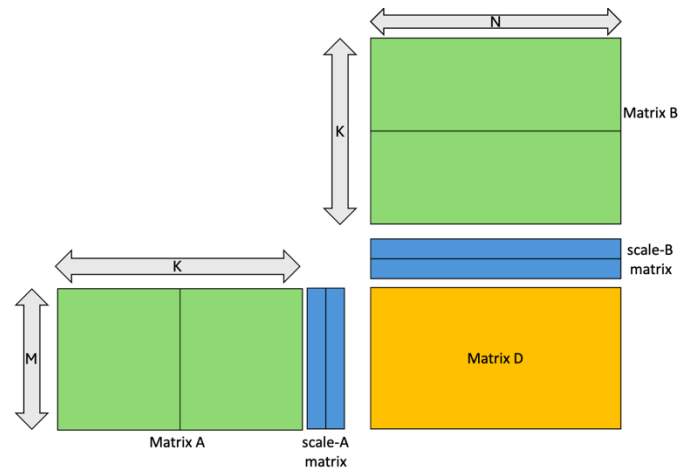


Figure 230: tcgen05.mma with block scaling of scale_vec::2X

The valid combination of the exact element types and the `.scale_vectorsize` are listed in Table 57.

Table 57: Valid combinations of scale_vectorsize with types and MMA-Kind

<code>.kind::*</code>	Element Data Type	Scale Data Type	<code>.scale_vectorsize</code>
<code>.kind::mxfbf64</code>	E4M3, E5M2, E2M3	UE8M0	<code>.scale_vec::1X / .block32</code>
<code>.kind::mxfbf32</code>	E4M3, E5M2, E2M1	UE8M0	<code>.scale_vec::1X / .block32</code>
<code>.kind::mxvf4</code>	E2M1	UE8M0	<code>.scale_vec::2X / .block32</code>
<code>.kind::mxvf4nvf4</code>	E2M1	UE8M0	<code>.scale_vec::2X / .block32</code> , <code>.scale_vec::4X / .block16</code>
	E2M1	UE4M3	<code>.scale_vec::4X / .block16</code>

New `.blockN` qualifiers are aliases for `.scale_vec::NX` qualifiers as:

- `.block32` is alias for `.scale_vec::1X` or `.scale_vec::2X` based on `.kind` and `K` dimension
- `.block16` is alias for `.scale_vec::4X`

9.7.16.10.7.2 Scale Factor A ID

The value of the scale factor A ID selects the sub-columns in the Tensor Memory to form the scale factor A matrix, which is used to scale the matrix A.

The following shows the scale factor matrix layout for various scale vector sizes:

9.7.16.10.7.3 Layout of the Scale Factor A Matrix for scale_vec::1X/block32 with K=32/K=64

There is one scale factor per row of the A matrix with block size as 32 and the scale factor must be provided in 1-byte aligned sub-column of the Tensor Memory. *SFA_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 231 shows which sub-columns get selected for different values of *SFA_ID*.

Lane	Column X				Column X+1				Column X+2				Column X+3			
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
0	M0 SFO	M0 SFO	M0 SFO	M0 SFO	M32 SFO	M32 SFO	M32 SFO	M32 SFO	M64 SFO	M64 SFO	M64 SFO	M64 SFO	M96 SFO	M96 SFO	M96 SFO	M96 SFO
1	M1 SFO	M1 SFO	M1 SFO	M1 SFO	M33 SFO	M33 SFO	M33 SFO	M33 SFO	M65 SFO	M65 SFO	M65 SFO	M65 SFO	M97 SFO	M97 SFO	M97 SFO	M97 SFO
2	M2 SFO	M2 SFO	M2 SFO	M2 SFO	M34 SFO	M34 SFO	M34 SFO	M34 SFO	M66 SFO	M66 SFO	M66 SFO	M66 SFO	M98 SFO	M98 SFO	M98 SFO	M98 SFO
3	M3 SFO	M3 SFO	M3 SFO	M3 SFO	M35 SFO	M35 SFO	M35 SFO	M35 SFO	M67 SFO	M67 SFO	M67 SFO	M67 SFO	M99 SFO	M99 SFO	M99 SFO	M99 SFO
4	M4 SFO	M4 SFO	M4 SFO	M4 SFO	M36 SFO	M36 SFO	M36 SFO	M36 SFO	M68 SFO	M68 SFO	M68 SFO	M68 SFO	M100 SFO	M100 SFO	M100 SFO	M100 SFO
5	M5 SFO	M5 SFO	M5 SFO	M5 SFO	M37 SFO	M37 SFO	M37 SFO	M37 SFO	M69 SFO	M69 SFO	M69 SFO	M69 SFO	M101 SFO	M101 SFO	M101 SFO	M101 SFO
6	M6 SFO	M6 SFO	M6 SFO	M6 SFO	M38 SFO	M38 SFO	M38 SFO	M38 SFO	M70 SFO	M70 SFO	M70 SFO	M70 SFO	M102 SFO	M102 SFO	M102 SFO	M102 SFO
7	M7 SFO	M7 SFO	M7 SFO	M7 SFO	M39 SFO	M39 SFO	M39 SFO	M39 SFO	M71 SFO	M71 SFO	M71 SFO	M71 SFO	M103 SFO	M103 SFO	M103 SFO	M103 SFO
8	M8 SFO	M8 SFO	M8 SFO	M8 SFO	M40 SFO	M40 SFO	M40 SFO	M40 SFO	M72 SFO	M72 SFO	M72 SFO	M72 SFO	M104 SFO	M104 SFO	M104 SFO	M104 SFO
9	M9 SFO	M9 SFO	M9 SFO	M9 SFO	M41 SFO	M41 SFO	M41 SFO	M41 SFO	M73 SFO	M73 SFO	M73 SFO	M73 SFO	M105 SFO	M105 SFO	M105 SFO	M105 SFO
10	M10 SFO	M10 SFO	M10 SFO	M10 SFO	M42 SFO	M42 SFO	M42 SFO	M42 SFO	M74 SFO	M74 SFO	M74 SFO	M74 SFO	M106 SFO	M106 SFO	M106 SFO	M106 SFO
11	M11 SFO	M11 SFO	M11 SFO	M11 SFO	M43 SFO	M43 SFO	M43 SFO	M43 SFO	M75 SFO	M75 SFO	M75 SFO	M75 SFO	M107 SFO	M107 SFO	M107 SFO	M107 SFO
12	M12 SFO	M12 SFO	M12 SFO	M12 SFO	M44 SFO	M44 SFO	M44 SFO	M44 SFO	M76 SFO	M76 SFO	M76 SFO	M76 SFO	M108 SFO	M108 SFO	M108 SFO	M108 SFO
13	M13 SFO	M13 SFO	M13 SFO	M13 SFO	M45 SFO	M45 SFO	M45 SFO	M45 SFO	M77 SFO	M77 SFO	M77 SFO	M77 SFO	M109 SFO	M109 SFO	M109 SFO	M109 SFO
14	M14 SFO	M14 SFO	M14 SFO	M14 SFO	M46 SFO	M46 SFO	M46 SFO	M46 SFO	M78 SFO	M78 SFO	M78 SFO	M78 SFO	M110 SFO	M110 SFO	M110 SFO	M110 SFO
15	M15 SFO	M15 SFO	M15 SFO	M15 SFO	M47 SFO	M47 SFO	M47 SFO	M47 SFO	M79 SFO	M79 SFO	M79 SFO	M79 SFO	M111 SFO	M111 SFO	M111 SFO	M111 SFO
16	M16 SFO	M16 SFO	M16 SFO	M16 SFO	M48 SFO	M48 SFO	M48 SFO	M48 SFO	M80 SFO	M80 SFO	M80 SFO	M80 SFO	M112 SFO	M112 SFO	M112 SFO	M112 SFO
17	M17 SFO	M17 SFO	M17 SFO	M17 SFO	M49 SFO	M49 SFO	M49 SFO	M49 SFO	M81 SFO	M81 SFO	M81 SFO	M81 SFO	M113 SFO	M113 SFO	M113 SFO	M113 SFO
18	M18 SFO	M18 SFO	M18 SFO	M18 SFO	M50 SFO	M50 SFO	M50 SFO	M50 SFO	M82 SFO	M82 SFO	M82 SFO	M82 SFO	M114 SFO	M114 SFO	M114 SFO	M114 SFO
19	M19 SFO	M19 SFO	M19 SFO	M19 SFO	M51 SFO	M51 SFO	M51 SFO	M51 SFO	M83 SFO	M83 SFO	M83 SFO	M83 SFO	M115 SFO	M115 SFO	M115 SFO	M115 SFO
20	M20 SFO	M20 SFO	M20 SFO	M20 SFO	M52 SFO	M52 SFO	M52 SFO	M52 SFO	M84 SFO	M84 SFO	M84 SFO	M84 SFO	M116 SFO	M116 SFO	M116 SFO	M116 SFO
21	M21 SFO	M21 SFO	M21 SFO	M21 SFO	M53 SFO	M53 SFO	M53 SFO	M53 SFO	M85 SFO	M85 SFO	M85 SFO	M85 SFO	M117 SFO	M117 SFO	M117 SFO	M117 SFO
22	M22 SFO	M22 SFO	M22 SFO	M22 SFO	M54 SFO	M54 SFO	M54 SFO	M54 SFO	M86 SFO	M86 SFO	M86 SFO	M86 SFO	M118 SFO	M118 SFO	M118 SFO	M118 SFO
23	M23 SFO	M23 SFO	M23 SFO	M23 SFO	M55 SFO	M55 SFO	M55 SFO	M55 SFO	M87 SFO	M87 SFO	M87 SFO	M87 SFO	M119 SFO	M119 SFO	M119 SFO	M119 SFO
24	M24 SFO	M24 SFO	M24 SFO	M24 SFO	M56 SFO	M56 SFO	M56 SFO	M56 SFO	M88 SFO	M88 SFO	M88 SFO	M88 SFO	M120 SFO	M120 SFO	M120 SFO	M120 SFO
25	M25 SFO	M25 SFO	M25 SFO	M25 SFO	M57 SFO	M57 SFO	M57 SFO	M57 SFO	M89 SFO	M89 SFO	M89 SFO	M89 SFO	M121 SFO	M121 SFO	M121 SFO	M121 SFO
26	M26 SFO	M26 SFO	M26 SFO	M26 SFO	M58 SFO	M58 SFO	M58 SFO	M58 SFO	M90 SFO	M90 SFO	M90 SFO	M90 SFO	M122 SFO	M122 SFO	M122 SFO	M122 SFO
27	M27 SFO	M27 SFO	M27 SFO	M27 SFO	M59 SFO	M59 SFO	M59 SFO	M59 SFO	M91 SFO	M91 SFO	M91 SFO	M91 SFO	M123 SFO	M123 SFO	M123 SFO	M123 SFO
28	M28 SFO	M28 SFO	M28 SFO	M28 SFO	M60 SFO	M60 SFO	M60 SFO	M60 SFO	M92 SFO	M92 SFO	M92 SFO	M92 SFO	M124 SFO	M124 SFO	M124 SFO	M124 SFO
29	M29 SFO	M29 SFO	M29 SFO	M29 SFO	M61 SFO	M61 SFO	M61 SFO	M61 SFO	M93 SFO	M93 SFO	M93 SFO	M93 SFO	M125 SFO	M125 SFO	M125 SFO	M125 SFO
30	M30 SFO	M30 SFO	M30 SFO	M30 SFO	M62 SFO	M62 SFO	M62 SFO	M62 SFO	M94 SFO	M94 SFO	M94 SFO	M94 SFO	M126 SFO	M126 SFO	M126 SFO	M126 SFO
31	M31 SFO	M31 SFO	M31 SFO	M31 SFO	M63 SFO	M63 SFO	M63 SFO	M63 SFO	M95 SFO	M95 SFO	M95 SFO	M95 SFO	M127 SFO	M127 SFO	M127 SFO	M127 SFO

SFA ID = 00

SFA ID = 01

SFA ID = 10

SFA ID = 11

Figure 231: Layout of scale factor A matrix with scale_vec::1X/block32 with K=32/K=64

For example, if *SFA_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, *SFA_ID* values of 1, 2 and 3 would select the blue, yellow, and red columns, respectively.

9.7.16.10.7.4 Layout of the Scale Factor A Matrix for scale_vec::2X/block32 with K=64/K=128

There are two scale factors per row of the A matrix with block size as 32 and the scale factor must be provided in 2-byte aligned sub-column of the Tensor Memory. *SFA_ID* specifies the half word offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 232 shows which sub-columns gets selected for different values of *SFA_ID*.

For example, if *SFA_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, if *SFA_ID* is 2, then all of the blue columns are selected to form the scale factor matrix.

Lane	Column X				Column X+1				Column X+2				Column X+3			
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
0	M0 SF0	M0 SF1	M0 SF0	M0 SF1	M32 SF0	M32 SF1	M32 SF0	M32 SF1	M64 SF0	M64 SF1	M64 SF0	M64 SF1	M96 SF0	M96 SF1	M96 SF0	M96 SF1
1	M1 SF0	M1 SF1	M1 SF0	M1 SF1	M33 SF0	M33 SF1	M33 SF0	M33 SF1	M65 SF0	M65 SF1	M65 SF0	M65 SF1	M97 SF0	M97 SF1	M97 SF0	M97 SF1
2	M2 SF0	M2 SF1	M2 SF0	M2 SF1	M34 SF0	M34 SF1	M34 SF0	M34 SF1	M66 SF0	M66 SF1	M66 SF0	M66 SF1	M98 SF0	M98 SF1	M98 SF0	M98 SF1
3	M3 SF0	M3 SF1	M3 SF0	M3 SF1	M35 SF0	M35 SF1	M35 SF0	M35 SF1	M67 SF0	M67 SF1	M67 SF0	M67 SF1	M99 SF0	M99 SF1	M99 SF0	M99 SF1
4	M4 SF0	M4 SF1	M4 SF0	M4 SF1	M36 SF0	M36 SF1	M36 SF0	M36 SF1	M68 SF0	M68 SF1	M68 SF0	M68 SF1	M100 SF0	M100 SF1	M100 SF0	M100 SF1
5	M5 SF0	M5 SF1	M5 SF0	M5 SF1	M37 SF0	M37 SF1	M37 SF0	M37 SF1	M69 SF0	M69 SF1	M69 SF0	M69 SF1	M101 SF0	M101 SF1	M101 SF0	M101 SF1
6	M6 SF0	M6 SF1	M6 SF0	M6 SF1	M38 SF0	M38 SF1	M38 SF0	M38 SF1	M70 SF0	M70 SF1	M70 SF0	M70 SF1	M102 SF0	M102 SF1	M102 SF0	M102 SF1
7	M7 SF0	M7 SF1	M7 SF0	M7 SF1	M39 SF0	M39 SF1	M39 SF0	M39 SF1	M71 SF0	M71 SF1	M71 SF0	M71 SF1	M103 SF0	M103 SF1	M103 SF0	M103 SF1
8	M8 SF0	M8 SF1	M8 SF0	M8 SF1	M40 SF0	M40 SF1	M40 SF0	M40 SF1	M72 SF0	M72 SF1	M72 SF0	M72 SF1	M104 SF0	M104 SF1	M104 SF0	M104 SF1
9	M9 SF0	M9 SF1	M9 SF0	M9 SF1	M41 SF0	M41 SF1	M41 SF0	M41 SF1	M73 SF0	M73 SF1	M73 SF0	M73 SF1	M105 SF0	M105 SF1	M105 SF0	M105 SF1
10	M10 SF0	M10 SF1	M10 SF0	M10 SF1	M42 SF0	M42 SF1	M42 SF0	M42 SF1	M74 SF0	M74 SF1	M74 SF0	M74 SF1	M106 SF0	M106 SF1	M106 SF0	M106 SF1
11	M11 SF0	M11 SF1	M11 SF0	M11 SF1	M43 SF0	M43 SF1	M43 SF0	M43 SF1	M75 SF0	M75 SF1	M75 SF0	M75 SF1	M107 SF0	M107 SF1	M107 SF0	M107 SF1
12	M12 SF0	M12 SF1	M12 SF0	M12 SF1	M44 SF0	M44 SF1	M44 SF0	M44 SF1	M76 SF0	M76 SF1	M76 SF0	M76 SF1	M108 SF0	M108 SF1	M108 SF0	M108 SF1
13	M13 SF0	M13 SF1	M13 SF0	M13 SF1	M45 SF0	M45 SF1	M45 SF0	M45 SF1	M77 SF0	M77 SF1	M77 SF0	M77 SF1	M109 SF0	M109 SF1	M109 SF0	M109 SF1
14	M14 SF0	M14 SF1	M14 SF0	M14 SF1	M46 SF0	M46 SF1	M46 SF0	M46 SF1	M78 SF0	M78 SF1	M78 SF0	M78 SF1	M110 SF0	M110 SF1	M110 SF0	M110 SF1
15	M15 SF0	M15 SF1	M15 SF0	M15 SF1	M47 SF0	M47 SF1	M47 SF0	M47 SF1	M79 SF0	M79 SF1	M79 SF0	M79 SF1	M111 SF0	M111 SF1	M111 SF0	M111 SF1
16	M16 SF0	M16 SF1	M16 SF0	M16 SF1	M48 SF0	M48 SF1	M48 SF0	M48 SF1	M80 SF0	M80 SF1	M80 SF0	M80 SF1	M112 SF0	M112 SF1	M112 SF0	M112 SF1
17	M17 SF0	M17 SF1	M17 SF0	M17 SF1	M49 SF0	M49 SF1	M49 SF0	M49 SF1	M81 SF0	M81 SF1	M81 SF0	M81 SF1	M113 SF0	M113 SF1	M113 SF0	M113 SF1
18	M18 SF0	M18 SF1	M18 SF0	M18 SF1	M50 SF0	M50 SF1	M50 SF0	M50 SF1	M82 SF0	M82 SF1	M82 SF0	M82 SF1	M114 SF0	M114 SF1	M114 SF0	M114 SF1
19	M19 SF0	M19 SF1	M19 SF0	M19 SF1	M51 SF0	M51 SF1	M51 SF0	M51 SF1	M83 SF0	M83 SF1	M83 SF0	M83 SF1	M115 SF0	M115 SF1	M115 SF0	M115 SF1
20	M20 SF0	M20 SF1	M20 SF0	M20 SF1	M52 SF0	M52 SF1	M52 SF0	M52 SF1	M84 SF0	M84 SF1	M84 SF0	M84 SF1	M116 SF0	M116 SF1	M116 SF0	M116 SF1
21	M21 SF0	M21 SF1	M21 SF0	M21 SF1	M53 SF0	M53 SF1	M53 SF0	M53 SF1	M85 SF0	M85 SF1	M85 SF0	M85 SF1	M117 SF0	M117 SF1	M117 SF0	M117 SF1
22	M22 SF0	M22 SF1	M22 SF0	M22 SF1	M54 SF0	M54 SF1	M54 SF0	M54 SF1	M86 SF0	M86 SF1	M86 SF0	M86 SF1	M118 SF0	M118 SF1	M118 SF0	M118 SF1
23	M23 SF0	M23 SF1	M23 SF0	M23 SF1	M55 SF0	M55 SF1	M55 SF0	M55 SF1	M87 SF0	M87 SF1	M87 SF0	M87 SF1	M119 SF0	M119 SF1	M119 SF0	M119 SF1
24	M24 SF0	M24 SF1	M24 SF0	M24 SF1	M56 SF0	M56 SF1	M56 SF0	M56 SF1	M88 SF0	M88 SF1	M88 SF0	M88 SF1	M120 SF0	M120 SF1	M120 SF0	M120 SF1
25	M25 SF0	M25 SF1	M25 SF0	M25 SF1	M57 SF0	M57 SF1	M57 SF0	M57 SF1	M89 SF0	M89 SF1	M89 SF0	M89 SF1	M121 SF0	M121 SF1	M121 SF0	M121 SF1
26	M26 SF0	M26 SF1	M26 SF0	M26 SF1	M58 SF0	M58 SF1	M58 SF0	M58 SF1	M90 SF0	M90 SF1	M90 SF0	M90 SF1	M122 SF0	M122 SF1	M122 SF0	M122 SF1
27	M27 SF0	M27 SF1	M27 SF0	M27 SF1	M59 SF0	M59 SF1	M59 SF0	M59 SF1	M91 SF0	M91 SF1	M91 SF0	M91 SF1	M123 SF0	M123 SF1	M123 SF0	M123 SF1
28	M28 SF0	M28 SF1	M28 SF0	M28 SF1	M60 SF0	M60 SF1	M60 SF0	M60 SF1	M92 SF0	M92 SF1	M92 SF0	M92 SF1	M124 SF0	M124 SF1	M124 SF0	M124 SF1
29	M29 SF0	M29 SF1	M29 SF0	M29 SF1	M61 SF0	M61 SF1	M61 SF0	M61 SF1	M93 SF0	M93 SF1	M93 SF0	M93 SF1	M125 SF0	M125 SF1	M125 SF0	M125 SF1
30	M30 SF0	M30 SF1	M30 SF0	M30 SF1	M62 SF0	M62 SF1	M62 SF0	M62 SF1	M94 SF0	M94 SF1	M94 SF0	M94 SF1	M126 SF0	M126 SF1	M126 SF0	M126 SF1
31	M31 SF0	M31 SF1	M31 SF0	M31 SF1	M63 SF0	M63 SF1	M63 SF0	M63 SF1	M95 SF0	M95 SF1	M95 SF0	M95 SF1	M127 SF0	M127 SF1	M127 SF0	M127 SF1

SFA_ID = 00

SFA_ID = 10

Figure 232: Layout of scale factor A matrix with scale_vec::2X/block32 with K=64/K=128

9.7.16.10.7.5 Layout of the Scale Factor A Matrix for scale_vec::4X/block16 with K=64/K=128

There are four scale factors per row of the A matrix with block size as 16 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. The *SFA_ID* value must be 0 and this specifies that all of the columns (in green) will be used for the scale factor matrix. Figure 233 shows which sub-columns gets selected for different values of *SFA_ID*.

9.7.16.10.7.6 Layout of the Scale Factor A Matrix for block32 with K=96 (Semantically equivalent to scale_vec::3X)

There are three scale factors per row of the A matrix with block size as 32 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. *SFA_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 234, Figure 235, Figure 236 and Figure 237 show which sub-columns get selected for different values of *SFA_ID*.

For example, if *SFA_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, *SFA_ID* values of 1, 2 and 3 would select the blue, yellow, and red columns, respectively.

9.7.16.10.7.7 Layout of the Scale Factor A Matrix for block16 with K=96 (Semantically equivalent to scale_vec::6X)

There are six scale factors per row of the A matrix with block size as 16 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. *SFA_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 238 and Figure 239 show which sub-columns get selected for different values of *SFA_ID*.

For example, if *SFA_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, if *SFA_ID* is 2, then all of the blue columns are selected to form the scale factor matrix.

Column X				Column X+1				Column X+2				Column X+3				
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
Lane 0	M0 SF0	M0 SF1	M0 SF2		M32 SF0	M32 SF1	M32 SF2		M64 SF0	M64 SF1	M64 SF2		M96 SF0	M96 SF1	M96 SF2	
1	M1 SF0	M1 SF1	M1 SF2		M33 SF0	M33 SF1	M33 SF2		M65 SF0	M65 SF1	M65 SF2		M97 SF0	M97 SF1	M97 SF2	
2	M2 SF0	M2 SF1	M2 SF2		M34 SF0	M34 SF1	M34 SF2		M66 SF0	M66 SF1	M66 SF2		M98 SF0	M98 SF1	M98 SF2	
3	M3 SF0	M3 SF1	M3 SF2		M35 SF0	M35 SF1	M35 SF2		M67 SF0	M67 SF1	M67 SF2		M99 SF0	M99 SF1	M99 SF2	
4	M4 SF0	M4 SF1	M4 SF2		M36 SF0	M36 SF1	M36 SF2		M68 SF0	M68 SF1	M68 SF2		M100 SF0	M100 SF1	M100 SF2	
5	M5 SF0	M5 SF1	M5 SF2		M37 SF0	M37 SF1	M37 SF2		M69 SF0	M69 SF1	M69 SF2		M101 SF0	M101 SF1	M101 SF2	
6	M6 SF0	M6 SF1	M6 SF2		M38 SF0	M38 SF1	M38 SF2		M70 SF0	M70 SF1	M70 SF2		M102 SF0	M102 SF1	M102 SF2	
7	M7 SF0	M7 SF1	M7 SF2		M39 SF0	M39 SF1	M39 SF2		M71 SF0	M71 SF1	M71 SF2		M103 SF0	M103 SF1	M103 SF2	
8	M8 SF0	M8 SF1	M8 SF2		M40 SF0	M40 SF1	M40 SF2		M72 SF0	M72 SF1	M72 SF2		M104 SF0	M104 SF1	M104 SF2	
9	M9 SF0	M9 SF1	M9 SF2		M41 SF0	M41 SF1	M41 SF2		M73 SF0	M73 SF1	M73 SF2		M105 SF0	M105 SF1	M105 SF2	
10	M10 SF0	M10 SF1	M10 SF2		M42 SF0	M42 SF1	M42 SF2		M74 SF0	M74 SF1	M74 SF2		M106 SF0	M106 SF1	M106 SF2	
11	M11 SF0	M11 SF1	M11 SF2		M43 SF0	M43 SF1	M43 SF2		M75 SF0	M75 SF1	M75 SF2		M107 SF0	M107 SF1	M107 SF2	
12	M12 SF0	M12 SF1	M12 SF2		M44 SF0	M44 SF1	M44 SF2		M76 SF0	M76 SF1	M76 SF2		M108 SF0	M108 SF1	M108 SF2	
13	M13 SF0	M13 SF1	M13 SF2		M45 SF0	M45 SF1	M45 SF2		M77 SF0	M77 SF1	M77 SF2		M109 SF0	M109 SF1	M109 SF2	
14	M14 SF0	M14 SF1	M14 SF2		M46 SF0	M46 SF1	M46 SF2		M78 SF0	M78 SF1	M78 SF2		M110 SF0	M110 SF1	M110 SF2	
15	M15 SF0	M15 SF1	M15 SF2		M47 SF0	M47 SF1	M47 SF2		M79 SF0	M79 SF1	M79 SF2		M111 SF0	M111 SF1	M111 SF2	
16	M16 SF0	M16 SF1	M16 SF2		M48 SF0	M48 SF1	M48 SF2		M80 SF0	M80 SF1	M80 SF2		M112 SF0	M112 SF1	M112 SF2	
17	M17 SF0	M17 SF1	M17 SF2		M49 SF0	M49 SF1	M49 SF2		M81 SF0	M81 SF1	M81 SF2		M113 SF0	M113 SF1	M113 SF2	
18	M18 SF0	M18 SF1	M18 SF2		M50 SF0	M50 SF1	M50 SF2		M82 SF0	M82 SF1	M82 SF2		M114 SF0	M114 SF1	M114 SF2	
19	M19 SF0	M19 SF1	M19 SF2		M51 SF0	M51 SF1	M51 SF2		M83 SF0	M83 SF1	M83 SF2		M115 SF0	M115 SF1	M115 SF2	
20	M20 SF0	M20 SF1	M20 SF2		M52 SF0	M52 SF1	M52 SF2		M84 SF0	M84 SF1	M84 SF2		M116 SF0	M116 SF1	M116 SF2	
21	M21 SF0	M21 SF1	M21 SF2		M53 SF0	M53 SF1	M53 SF2		M85 SF0	M85 SF1	M85 SF2		M117 SF0	M117 SF1	M117 SF2	
22	M22 SF0	M22 SF1	M22 SF2		M54 SF0	M54 SF1	M54 SF2		M86 SF0	M86 SF1	M86 SF2		M118 SF0	M118 SF1	M118 SF2	
23	M23 SF0	M23 SF1	M23 SF2		M55 SF0	M55 SF1	M55 SF2		M87 SF0	M87 SF1	M87 SF2		M119 SF0	M119 SF1	M119 SF2	
24	M24 SF0	M24 SF1	M24 SF2		M56 SF0	M56 SF1	M56 SF2		M88 SF0	M88 SF1	M88 SF2		M120 SF0	M120 SF1	M120 SF2	
25	M25 SF0	M25 SF1	M25 SF2		M57 SF0	M57 SF1	M57 SF2		M89 SF0	M89 SF1	M89 SF2		M121 SF0	M121 SF1	M121 SF2	
26	M26 SF0	M26 SF1	M26 SF2		M58 SF0	M58 SF1	M58 SF2		M90 SF0	M90 SF1	M90 SF2		M122 SF0	M122 SF1	M122 SF2	
27	M27 SF0	M27 SF1	M27 SF2		M59 SF0	M59 SF1	M59 SF2		M81 SF0	M81 SF1	M81 SF2		M123 SF0	M123 SF1	M123 SF2	
28	M28 SF0	M28 SF1	M28 SF2		M60 SF0	M60 SF1	M60 SF2		M8							

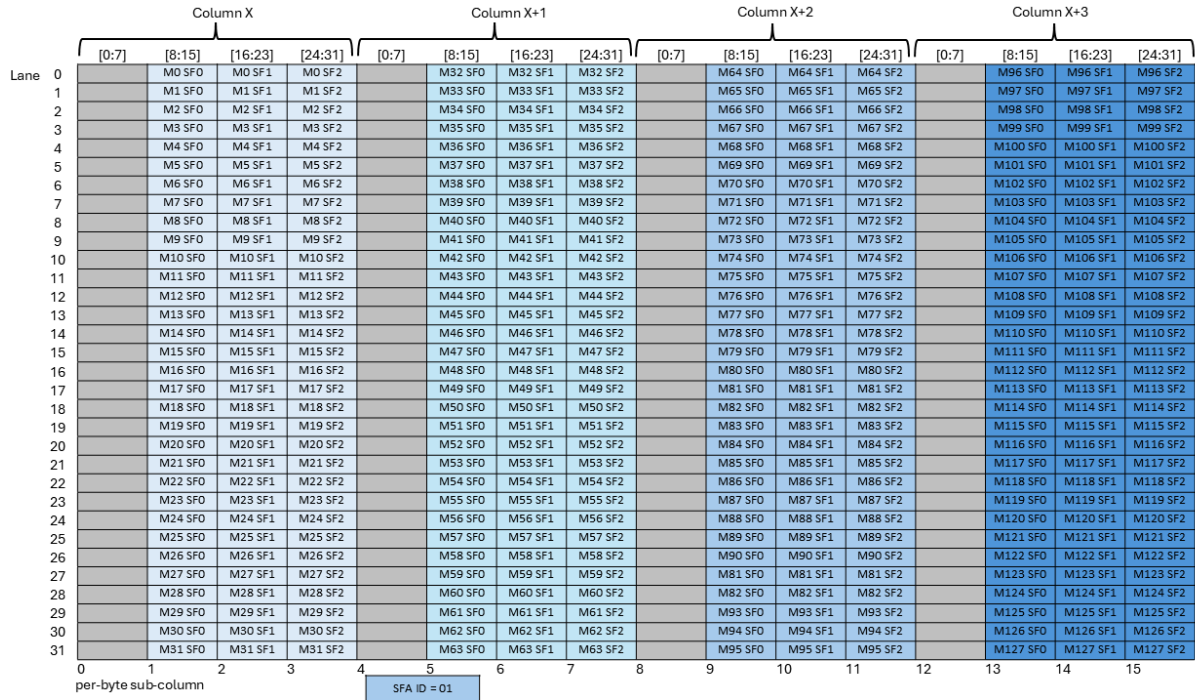


Figure 235: Layout of scale factor A matrix with block32 with K=96 with SFA_ID=01

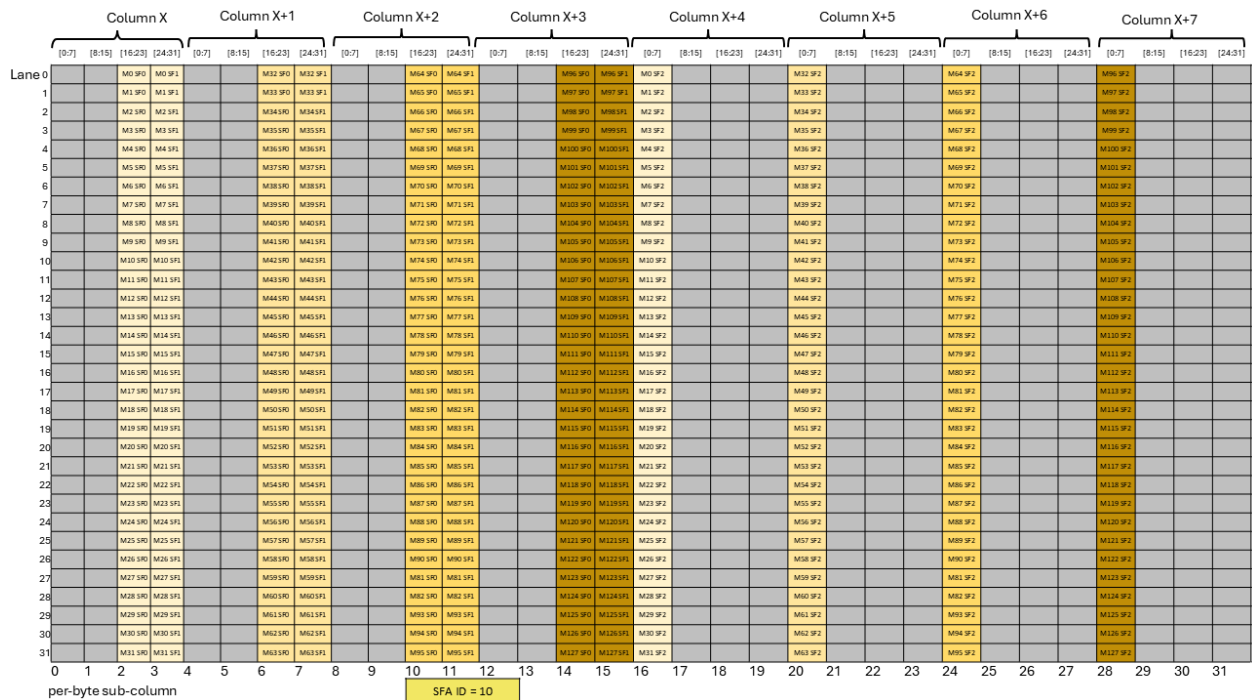


Figure 236: Layout of scale factor A matrix with block32 with K=96 with SFA_ID=10

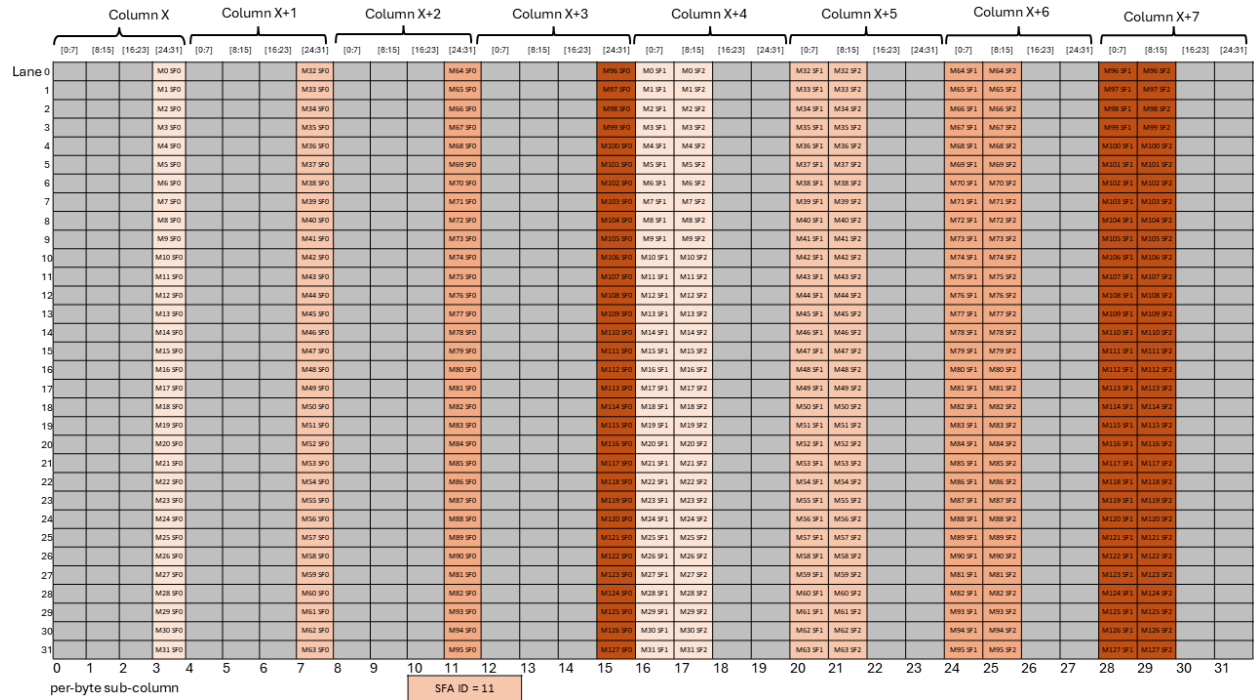


Figure 237: Layout of scale factor A matrix with block32 with K=96 with SFA_ID=11

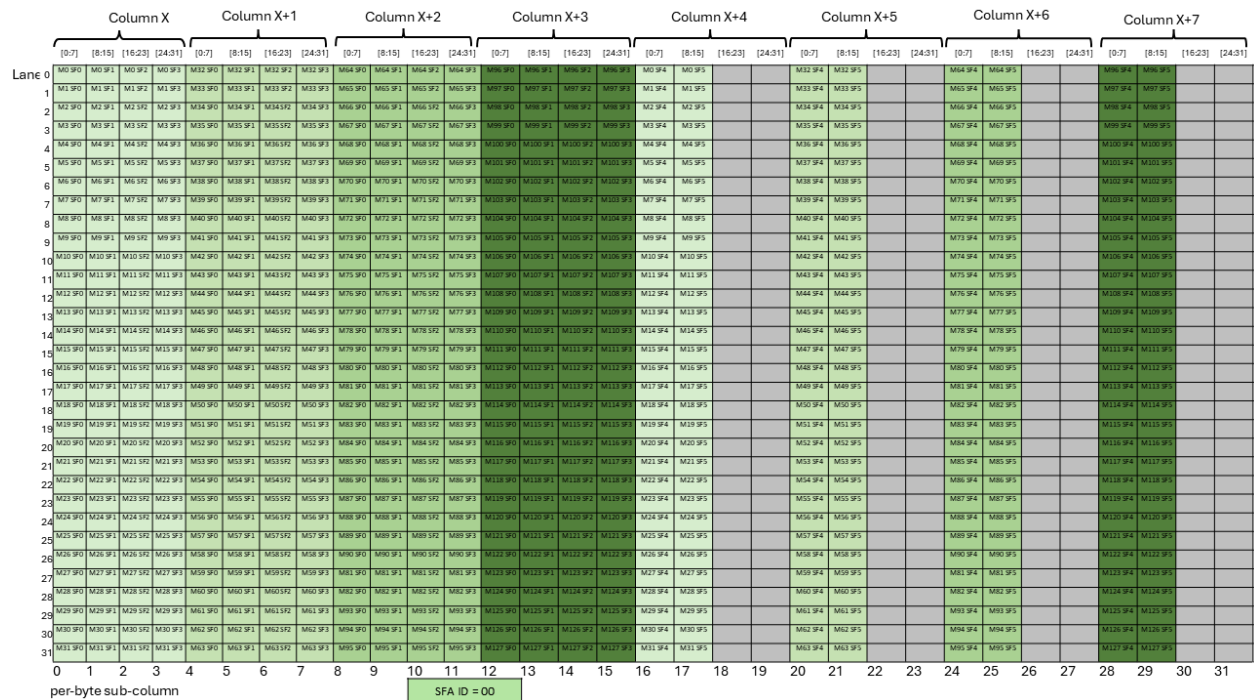


Figure 238: Layout of scale factor A matrix with block16 with K=96 with SFA_ID=00

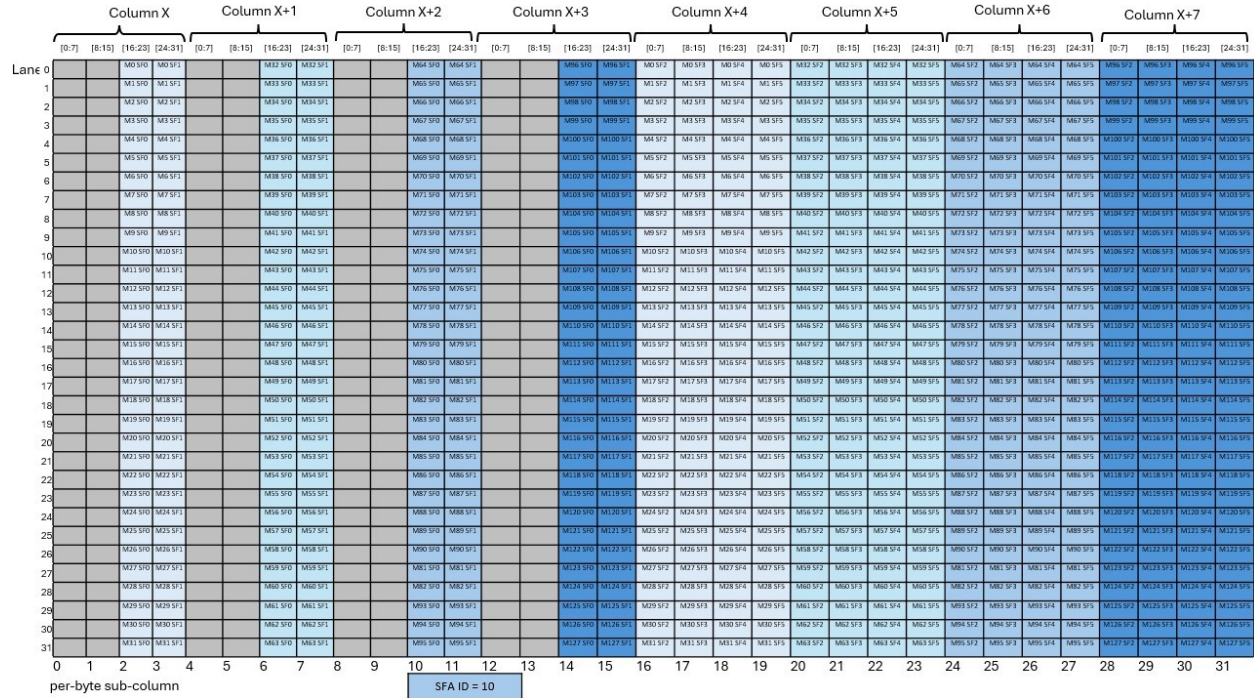


Figure 239: Layout of scale factor A matrix with block16 with K=96 with SFA_ID=10

9.7.16.10.7.8 Scale Factor B ID

The value of the scale factor B ID selects the sub-columns in the Tensor Memory to form the scale factor B matrix, which is used to scale the matrix B.

The following shows the scale factor matrix layout for various scale vector sizes:

9.7.16.10.7.9 Layout of the Scale Factor B Matrix for scale_vec::1X/block32 with K=32/K=64

There is one scale factor per row of the B matrix with block size as 32 and the scale factor must be provided in 1-byte aligned sub-column of the Tensor Memory. *SFB_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 240 shows which sub-columns get selected for different values of *SFB_ID*.

For example, if *SFB_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, *SFB_ID* values of 1, 2 and 3 would select the blue, yellow, and red columns, respectively.

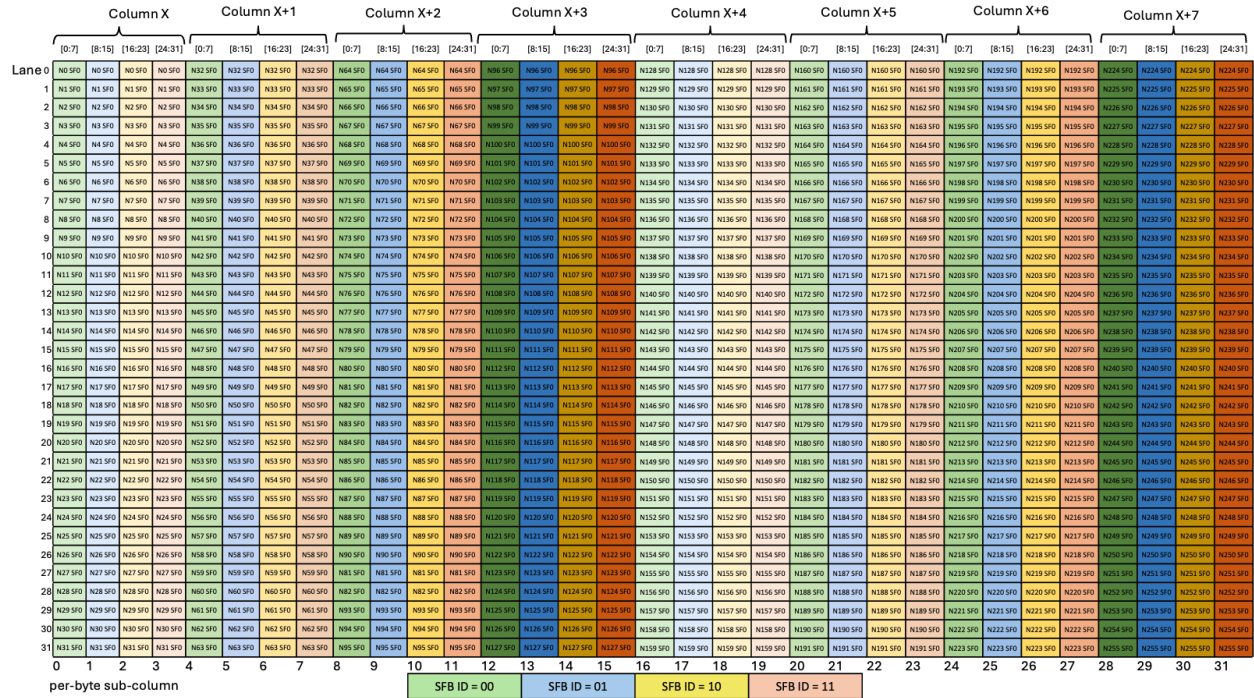


Figure 240: Layout of scale factor B matrix with scale_vec::1X/block32 with K=32/K=64

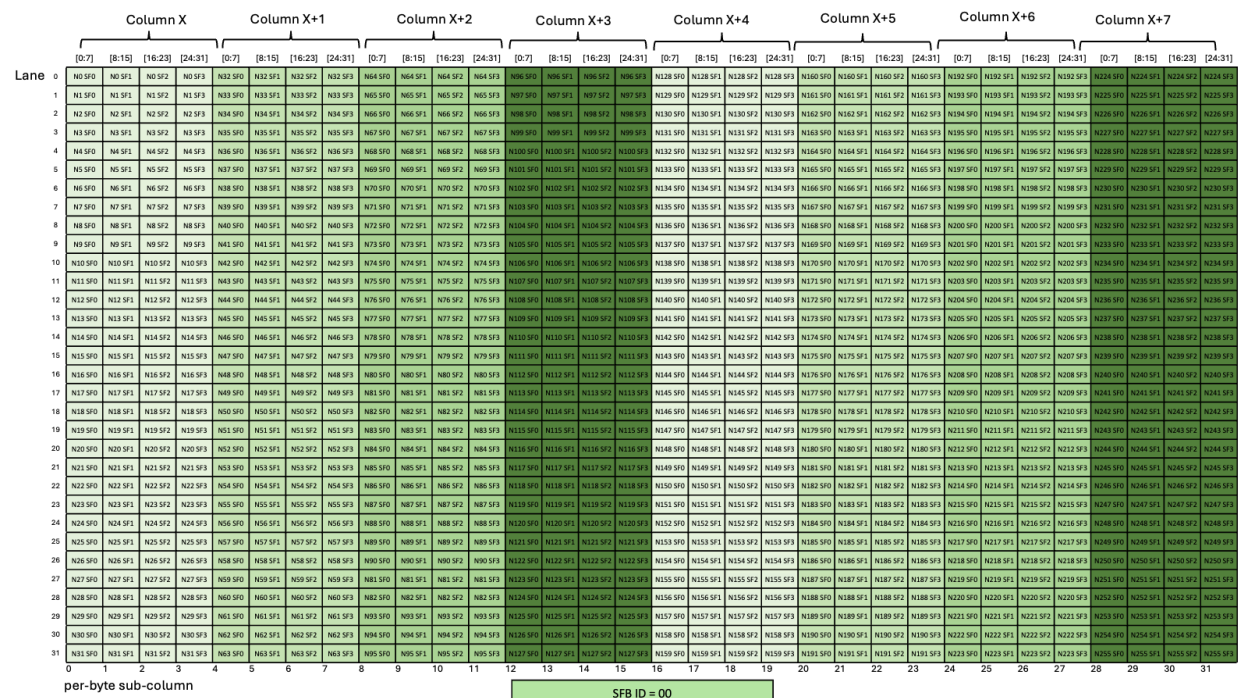
9.7.16.10.7.10 Layout of the Scale Factor B Matrix for scale_vec::2X/block32 with K=64/K=128

There are two scale factors per row of the B matrix with block size as 32 and the scale factor must be provided in 2-byte aligned sub-column of the Tensor Memory. *SFB_ID* specifies the half word offset in the Tensor Memory word that must be used for the scale factor matrix. Figure 241 shows which sub-columns get selected for different values of *SFB_ID*.

For example, if *SFB_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, if *SFB_ID* is 2, then all of the blue columns are selected to form the scale factor matrix.

9.7.16.10.7.11 Layout of the Scale Factor B Matrix for scale_vec::4X/block16 with K=64/K=128

There are four scale factors per row of the B matrix with block size as 16 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. The *SFB_ID* value must be 0 and this specifies that all of the columns (in green) will be used for the scale factor matrix. Figure 242 shows which sub-columns get selected for different values of *SFB_ID*.



9.7.16.10.7.12 Layout of the Scale Factor B Matrix for block32 with K=96 (Semantically equivalent to scale_vec::3X)

There are three scale factors per row of the B matrix with block size as 32 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. *SFB_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix.

For $N \leq 128$, Figure 243, Figure 244, Figure 245 and Figure 246 show which sub-columns get selected for different values of *SFB_ID*.

Lane	Column X				Column X+1				Column X+2				Column X+3			
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
0	N0 SF0	N0 SF1	N0 SF2		N32 SF0	N32 SF1	N32 SF2		N64 SF0	N64 SF1	N64 SF2		N96 SF0	N96 SF1	N96 SF2	
1	N1 SF0	N1 SF1	N1 SF2		N33 SF0	N33 SF1	N33 SF2		N65 SF0	N65 SF1	N65 SF2		N97 SF0	N97 SF1	N97 SF2	
2	N2 SF0	N2 SF1	N2 SF2		N34 SF0	N34 SF1	N34 SF2		N66 SF0	N66 SF1	N66 SF2		N98 SF0	N98 SF1	N98 SF2	
3	N3 SF0	N3 SF1	N3 SF2		N35 SF0	N35 SF1	N35 SF2		N67 SF0	N67 SF1	N67 SF2		N99 SF0	N99 SF1	N99 SF2	
4	N4 SF0	N4 SF1	N4 SF2		N36 SF0	N36 SF1	N36 SF2		N68 SF0	N68 SF1	N68 SF2		N100 SF0	N100 SF1	N100 SF2	
5	N5 SF0	N5 SF1	N5 SF2		N37 SF0	N37 SF1	N37 SF2		N69 SF0	N69 SF1	N69 SF2		N101 SF0	N101 SF1	N101 SF2	
6	N6 SF0	N6 SF1	N6 SF2		N38 SF0	N38 SF1	N38 SF2		N70 SF0	N70 SF1	N70 SF2		N102 SF0	N102 SF1	N102 SF2	
7	N7 SF0	N7 SF1	N7 SF2		N39 SF0	N39 SF1	N39 SF2		N71 SF0	N71 SF1	N71 SF2		N103 SF0	N103 SF1	N103 SF2	
8	N8 SF0	N8 SF1	N8 SF2		N40 SF0	N40 SF1	N40 SF2		N72 SF0	N72 SF1	N72 SF2		N104 SF0	N104 SF1	N104 SF2	
9	N9 SF0	N9 SF1	N9 SF2		N41 SF0	N41 SF1	N41 SF2		N73 SF0	N73 SF1	N73 SF2		N105 SF0	N105 SF1	N105 SF2	
10	N10 SF0	N10 SF1	N10 SF2		N42 SF0	N42 SF1	N42 SF2		N74 SF0	N74 SF1	N74 SF2		N106 SF0	N106 SF1	N106 SF2	
11	N11 SF0	N11 SF1	N11 SF2		N43 SF0	N43 SF1	N43 SF2		N75 SF0	N75 SF1	N75 SF2		N107 SF0	N107 SF1	N107 SF2	
12	N12 SF0	N12 SF1	N12 SF2		N44 SF0	N44 SF1	N44 SF2		N76 SF0	N76 SF1	N76 SF2		N108 SF0	N108 SF1	N108 SF2	
13	N13 SF0	N13 SF1	N13 SF2		N45 SF0	N45 SF1	N45 SF2		N77 SF0	N77 SF1	N77 SF2		N109 SF0	N109 SF1	N109 SF2	
14	N14 SF0	N14 SF1	N14 SF2		N46 SF0	N46 SF1	N46 SF2		N78 SF0	N78 SF1	N78 SF2		N110 SF0	N110 SF1	N110 SF2	
15	N15 SF0	N15 SF1	N15 SF2		N47 SF0	N47 SF1	N47 SF2		N79 SF0	N79 SF1	N79 SF2		N111 SF0	N111 SF1	N111 SF2	
16	N16 SF0	N16 SF1	N16 SF2		N48 SF0	N48 SF1	N48 SF2		N80 SF0	N80 SF1	N80 SF2		N112 SF0	N112 SF1	N112 SF2	
17	N17 SF0	N17 SF1	N17 SF2		N49 SF0	N49 SF1	N49 SF2		N81 SF0	N81 SF1	N81 SF2		N113 SF0	N113 SF1	N113 SF2	
18	N18 SF0	N18 SF1	N18 SF2		N50 SF0	N50 SF1	N50 SF2		N82 SF0	N82 SF1	N82 SF2		N114 SF0	N114 SF1	N114 SF2	
19	N19 SF0	N19 SF1	N19 SF2		N51 SF0	N51 SF1	N51 SF2		N83 SF0	N83 SF1	N83 SF2		N115 SF0	N115 SF1	N115 SF2	
20	N20 SF0	N20 SF1	N20 SF2		N52 SF0	N52 SF1	N52 SF2		N84 SF0	N84 SF1	N84 SF2		N116 SF0	N116 SF1	N116 SF2	
21	N21 SF0	N21 SF1	N21 SF2		N53 SF0	N53 SF1	N53 SF2		N85 SF0	N85 SF1	N85 SF2		N117 SF0	N117 SF1	N117 SF2	
22	N22 SF0	N22 SF1	N22 SF2		N54 SF0	N54 SF1	N54 SF2		N86 SF0	N86 SF1	N86 SF2		N118 SF0	N118 SF1	N118 SF2	
23	N23 SF0	N23 SF1	N23 SF2		N55 SF0	N55 SF1	N55 SF2		N87 SF0	N87 SF1	N87 SF2		N119 SF0	N119 SF1	N119 SF2	
24	N24 SF0	N24 SF1	N24 SF2		N56 SF0	N56 SF1	N56 SF2		N88 SF0	N88 SF1	N88 SF2		N120 SF0	N120 SF1	N120 SF2	
25	N25 SF0	N25 SF1	N25 SF2		N57 SF0	N57 SF1	N57 SF2		N89 SF0	N89 SF1	N89 SF2		N121 SF0	N121 SF1	N121 SF2	
26	N26 SF0	N26 SF1	N26 SF2		N58 SF0	N58 SF1	N58 SF2		N90 SF0	N90 SF1	N90 SF2		N122 SF0	N122 SF1	N122 SF2	
27	N27 SF0	N27 SF1	N27 SF2		N59 SF0	N59 SF1	N59 SF2		N91 SF0	N91 SF1	N91 SF2		N123 SF0	N123 SF1	N123 SF2	
28	N28 SF0	N28 SF1	N28 SF2		N60 SF0	N60 SF1	N60 SF2		N92 SF0	N92 SF1	N92 SF2		N124 SF0	N124 SF1	N124 SF2	
29	N29 SF0	N29 SF1	N29 SF2		N61 SF0	N61 SF1	N61 SF2		N93 SF0	N93 SF1	N93 SF2		N125 SF0	N125 SF1	N125 SF2	
30	N30 SF0	N30 SF1	N30 SF2		N62 SF0	N62 SF1	N62 SF2		N94 SF0	N94 SF1	N94 SF2		N126 SF0	N126 SF1	N126 SF2	
31	N31 SF0	N31 SF1	N31 SF2		N63 SF0	N63 SF1	N63 SF2		N95 SF0	N95 SF1	N95 SF2		N127 SF0	N127 SF1	N127 SF2	

per-byte sub-column

SFB ID = 00

Figure 243: Layout of scale factor B matrix with block32 with K=96 and $N \leq 128$ with SFA_ID=00

For $N > 128$, Figure 247, Figure 248, Figure 249, Figure 250, Figure 251 and Figure 252 show which sub-columns get selected for different values of *SFB_ID*.

For example, if *SFB_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, *SFB_ID* values of 1, 2 and 3 would select the blue, yellow, and red columns, respectively.

9.7.16.10.7.13 Layout of the Scale Factor B Matrix for block16 with K=96 (Semantically equivalent to scale_vec::6X)

There are six scale factors per row of the B matrix with block size as 16 and the scale factor must be provided in 4-byte aligned sub-column of the Tensor Memory. *SFB_ID* specifies the byte offset in the Tensor Memory word that must be used for the scale factor matrix.

For $N \leq 128$, Figure 253 and Figure 254 show which sub-columns get selected for different values of *SFB_ID*.

For $N > 128$, Figure 255, Figure 256, Figure 257 and Figure 258 show which sub-columns get selected for different values of *SFB_ID*.

Lane	Column X				Column X+1				Column X+2				Column X+3			
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
0	N0 SFO	N0 SF1	N0 SF2		N32 SFO	N32 SF1	N32 SF2		N64 SFO	N64 SF1	N64 SF2		N96 SFO	N96 SF1	N96 SF2	
1	N1 SFO	N1 SF1	N1 SF2		N33 SFO	N33 SF1	N33 SF2		N65 SFO	N65 SF1	N65 SF2		N97 SFO	N97 SF1	N97 SF2	
2	N2 SFO	N2 SF1	N2 SF2		N34 SFO	N34 SF1	N34 SF2		N66 SFO	N66 SF1	N66 SF2		N98 SFO	N98 SF1	N98 SF2	
3	N3 SFO	N3 SF1	N3 SF2		N35 SFO	N35 SF1	N35 SF2		N67 SFO	N67 SF1	N67 SF2		N99 SFO	N99 SF1	N99 SF2	
4	N4 SFO	N4 SF1	N4 SF2		N36 SFO	N36 SF1	N36 SF2		N68 SFO	N68 SF1	N68 SF2		N100 SFO	N100 SF1	N100 SF2	
5	N5 SFO	N5 SF1	N5 SF2		N37 SFO	N37 SF1	N37 SF2		N69 SFO	N69 SF1	N69 SF2		N101 SFO	N101 SF1	N101 SF2	
6	N6 SFO	N6 SF1	N6 SF2		N38 SFO	N38 SF1	N38 SF2		N70 SFO	N70 SF1	N70 SF2		N102 SFO	N102 SF1	N102 SF2	
7	N7 SFO	N7 SF1	N7 SF2		N39 SFO	N39 SF1	N39 SF2		N71 SFO	N71 SF1	N71 SF2		N103 SFO	N103 SF1	N103 SF2	
8	N8 SFO	N8 SF1	N8 SF2		N40 SFO	N40 SF1	N40 SF2		N72 SFO	N72 SF1	N72 SF2		N104 SFO	N104 SF1	N104 SF2	
9	N9 SFO	N9 SF1	N9 SF2		N41 SFO	N41 SF1	N41 SF2		N73 SFO	N73 SF1	N73 SF2		N105 SFO	N105 SF1	N105 SF2	
10	N10 SFO	N10 SF1	N10 SF2		N42 SFO	N42 SF1	N42 SF2		N74 SFO	N74 SF1	N74 SF2		N106 SFO	N106 SF1	N106 SF2	
11	N11 SFO	N11 SF1	N11 SF2		N43 SFO	N43 SF1	N43 SF2		N75 SFO	N75 SF1	N75 SF2		N107 SFO	N107 SF1	N107 SF2	
12	N12 SFO	N12 SF1	N12 SF2		N44 SFO	N44 SF1	N44 SF2		N76 SFO	N76 SF1	N76 SF2		N108 SFO	N108 SF1	N108 SF2	
13	N13 SFO	N13 SF1	N13 SF2		N45 SFO	N45 SF1	N45 SF2		N77 SFO	N77 SF1	N77 SF2		N109 SFO	N109 SF1	N109 SF2	
14	N14 SFO	N14 SF1	N14 SF2		N46 SFO	N46 SF1	N46 SF2		N78 SFO	N78 SF1	N78 SF2		N110 SFO	N110 SF1	N110 SF2	
15	N15 SFO	N15 SF1	N15 SF2		N47 SFO	N47 SF1	N47 SF2		N79 SFO	N79 SF1	N79 SF2		N111 SFO	N111 SF1	N111 SF2	
16	N16 SFO	N16 SF1	N16 SF2		N48 SFO	N48 SF1	N48 SF2		N80 SFO	N80 SF1	N80 SF2		N112 SFO	N112 SF1	N112 SF2	
17	N17 SFO	N17 SF1	N17 SF2		N49 SFO	N49 SF1	N49 SF2		N81 SFO	N81 SF1	N81 SF2		N113 SFO	N113 SF1	N113 SF2	
18	N18 SFO	N18 SF1	N18 SF2		N50 SFO	N50 SF1	N50 SF2		N82 SFO	N82 SF1	N82 SF2		N114 SFO	N114 SF1	N114 SF2	
19	N19 SFO	N19 SF1	N19 SF2		N51 SFO	N51 SF1	N51 SF2		N83 SFO	N83 SF1	N83 SF2		N115 SFO	N115 SF1	N115 SF2	
20	N20 SFO	N20 SF1	N20 SF2		N52 SFO	N52 SF1	N52 SF2		N84 SFO	N84 SF1	N84 SF2		N116 SFO	N116 SF1	N116 SF2	
21	N21 SFO	N21 SF1	N21 SF2		N53 SFO	N53 SF1	N53 SF2		N85 SFO	N85 SF1	N85 SF2		N117 SFO	N117 SF1	N117 SF2	
22	N22 SFO	N22 SF1	N22 SF2		N54 SFO	N54 SF1	N54 SF2		N86 SFO	N86 SF1	N86 SF2		N118 SFO	N118 SF1	N118 SF2	
23	N23 SFO	N23 SF1	N23 SF2		N55 SFO	N55 SF1	N55 SF2		N87 SFO	N87 SF1	N87 SF2		N119 SFO	N119 SF1	N119 SF2	
24	N24 SFO	N24 SF1	N24 SF2		N56 SFO	N56 SF1	N56 SF2		N88 SFO	N88 SF1	N88 SF2		N120 SFO	N120 SF1	N120 SF2	
25	N25 SFO	N25 SF1	N25 SF2		N57 SFO	N57 SF1	N57 SF2		N89 SFO	N89 SF1	N89 SF2		N121 SFO	N121 SF1	N121 SF2	
26	N26 SFO	N26 SF1	N26 SF2		N58 SFO	N58 SF1	N58 SF2		N90 SFO	N90 SF1	N90 SF2		N122 SFO	N122 SF1	N122 SF2	
27	N27 SFO	N27 SF1	N27 SF2		N59 SFO	N59 SF1	N59 SF2		N91 SFO	N91 SF1	N91 SF2		N123 SFO	N123 SF1	N123 SF2	
28	N28 SFO	N28 SF1	N28 SF2		N60 SFO	N60 SF1	N60 SF2		N92 SFO	N92 SF1	N92 SF2		N124 SFO	N124 SF1	N124 SF2	
29	N29 SFO	N29 SF1	N29 SF2		N61 SFO	N61 SF1	N61 SF2		N93 SFO	N93 SF1	N93 SF2		N125 SFO	N125 SF1	N125 SF2	
30	N30 SFO	N30 SF1	N30 SF2		N62 SFO	N62 SF1	N62 SF2		N94 SFO	N94 SF1	N94 SF2		N126 SFO	N126 SF1	N126 SF2	
31	N31 SFO	N31 SF1	N31 SF2		N63 SFO	N63 SF1	N63 SF2		N95 SFO	N95 SF1	N95 SF2		N127 SFO	N127 SF1	N127 SF2	

per-byte sub-column

SFB ID = 01

Figure 244: Layout of scale factor B matrix with block32 with K=96 and N<=128 with SFA_ID=01

Lane 0	Column X				Column X+1				Column X+2				Column X+3				Column X+4				Column X+5				Column X+6				Column X+7			
	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]	[0:7]	[8:15]	[16:23]	[24:31]
0	N0 SFO	N0 SF1			N32 SFO	N32 SF1			N64 SFO	N64 SF1			N96 SFO	N96 SF1	N96 SF2		N128 SFO			N160 SFO					N192 SFO				N224 SFO			
1	N1 SFO	N1 SF1	N1 SF2		N33 SFO	N33 SF1	N33 SF2		N65 SFO	N65 SF1	N65 SF2		N97 SFO	N97 SF1	N97 SF2	N97 SF3	N129 SFO	N129 SF1	N129 SF2	N161 SFO	N161 SF1	N161 SF2		N193 SFO	N193 SF1	N193 SF2		N225 SFO	N225 SF1	N225 SF2		
2	N2 SFO	N2 SF1			N34 SFO	N34 SF1			N66 SFO	N66 SF1			N98 SFO	N98 SF1	N98 SF2	N98 SF3	N130 SFO			N162 SFO				N194 SFO				N226 SFO	N226 SF1	N226 SF2		
3	N3 SFO	N3 SF1	N3 SF2		N35 SFO	N35 SF1	N35 SF2		N67 SFO	N67 SF1	N67 SF2		N99 SFO	N99 SF1	N99 SF2	N99 SF3	N131 SFO	N131 SF1	N131 SF2	N163 SFO	N163 SF1	N163 SF2		N195 SFO	N195 SF1	N195 SF2		N227 SFO	N227 SF1	N227 SF2		
4	N4 SFO	N4 SF1			N36 SFO	N36 SF1			N68 SFO	N68 SF1			N100 SFO	N100 SF1	N100 SF2	N100 SF3	N132 SFO			N164 SFO				N196 SFO				N228 SFO	N228 SF1	N228 SF2		
5	N5 SFO	N5 SF1	N5 SF2		N37 SFO	N37 SF1	N37 SF2		N69 SFO	N69 SF1	N69 SF2		N101 SFO	N101 SF1	N101 SF2	N101 SF3	N133 SFO	N133 SF1	N133 SF2	N165 SFO	N165 SF1	N165 SF2		N197 SFO	N197 SF1	N197 SF2		N229 SFO	N229 SF1	N229 SF2		
6	N6 SFO	N6 SF1			N38 SFO	N38 SF1			N70 SFO	N70 SF1			N102 SFO	N102 SF1	N102 SF2	N102 SF3	N134 SFO			N166 SFO				N198 SFO				N230 SFO	N230 SF1	N230 SF2		
7	N7 SFO	N7 SF1	N7 SF2		N39 SFO	N39 SF1	N39 SF2		N71 SFO	N71 SF1	N71 SF2		N103 SFO	N103 SF1	N103 SF2	N103 SF3	N135 SFO	N135 SF1	N135 SF2	N167 SFO	N167 SF1	N167 SF2		N199 SFO	N199 SF1	N199 SF2		N231 SFO	N231 SF1	N231 SF2		
8	N8 SFO	N8 SF1			N40 SFO	N40 SF1			N72 SFO	N72 SF1			N104 SFO	N104 SF1	N104 SF2	N104 SF3	N136 SFO			N168 SFO				N200 SFO				N232 SFO	N232 SF1	N232 SF2		
9	N9 SFO	N9 SF1	N9 SF2		N41 SFO	N41 SF1	N41 SF2		N73 SFO	N73 SF1	N73 SF2		N105 SFO	N105 SF1	N105 SF2	N105 SF3	N137 SFO	N137 SF1	N137 SF2	N169 SFO	N169 SF1	N169 SF2		N201 SFO	N201 SF1	N201 SF2		N233 SFO	N233 SF1	N233 SF2		
10	N10 SFO	N10 SF1			N42 SFO	N42 SF1			N74 SFO	N74 SF1			N106 SFO	N106 SF1	N106 SF2	N106 SF3	N138 SFO			N170 SFO				N202 SFO				N234 SFO	N234 SF1	N234 SF2		
11	N11 SFO	N11 SF1	N11 SF2		N43 SFO	N43 SF1	N43 SF2		N75 SFO	N75 SF1	N75 SF2		N107 SFO	N107 SF1	N107 SF2	N107 SF3	N139 SFO	N139 SF1	N139 SF2	N171 SFO	N171 SF1	N171 SF2		N203 SFO	N203 SF1	N203 SF2		N235 SFO	N235 SF1	N235 SF2		
12	N12 SFO	N12 SF1			N44 SFO	N44 SF1			N76 SFO	N76 SF1			N108 SFO	N108 SF1	N108 SF2	N108 SF3	N140 SFO			N172 SFO				N204 SFO				N236 SFO	N236 SF1	N236 SF2		
13	N13 SFO	N13 SF1	N13 SF2		N45 SFO	N45 SF1	N45 SF2		N77 SFO	N77 SF1	N77 SF2		N109 SFO	N109 SF1	N109 SF2	N109 SF3	N141 SFO	N141 SF1	N141 SF2	N173 SFO	N173 SF1	N173 SF2		N205 SFO	N205 SF1	N205 SF2		N237 SFO	N237 SF1	N237 SF2		
14	N14 SFO	N14 SF1			N46 SFO	N46 SF1			N78 SFO	N78 SF1			N110 SFO	N110 SF1	N110 SF2	N110 SF3	N142 SFO			N174 SFO				N206 SFO				N238 SFO	N238 SF1	N238 SF2		
15	N15 SFO	N15 SF1	N15 SF2		N47 SFO	N47 SF1	N47 SF2		N79 SFO	N79 SF1	N79 SF2		N111 SFO	N111 SF1	N111 SF2	N111 SF3	N143 SFO	N143 SF1	N143 SF2	N175 SFO	N175 SF1	N175 SF2		N207 SFO	N207 SF1	N207 SF2		N239 SFO	N239 SF1	N239 SF2		
16	N16 SFO	N16 SF1			N48 SFO	N48 SF1			N80 SFO	N80 SF1			N112 SFO	N112 SF1	N112 SF2	N112 SF3	N144 SFO			N176 SFO				N208 SFO				N240 SFO	N240 SF1	N240 SF2		
17	N17 SFO	N17 SF1	N17 SF2		N49 SFO	N49 SF1	N49 SF2		N81 SFO	N81 SF1	N81 SF2		N113 SFO	N113 SF1	N113 SF2	N113 SF3	N145 SFO	N145 SF1	N145 SF2	N177 SFO	N177 SF1	N177 SF2		N209 SFO	N209 SF1	N209 SF2		N241 SFO	N241 SF1	N241 SF2		
18	N18 SFO	N18 SF1			N50 SFO	N50 SF1			N82 SFO	N82 SF1			N114 SFO	N114 SF1	N114 SF2	N114 SF3	N146 SFO			N178 SFO				N210 SFO				N242 SFO	N242 SF1	N242 SF2		
19	N19 SFO	N19 SF1	N19 SF2		N51 SFO	N51 SF1	N51 SF2		N83 SFO	N83 SF1	N83 SF2		N115 SFO	N115 SF1	N115 SF2	N115 SF3	N147 SFO	N147 SF1	N147 SF2	N179 SFO	N179 SF1	N179 SF2		N211 SFO	N211 SF1	N211 SF2		N243 SFO	N243 SF1	N243 SF2		
20	N20 SFO	N20 SF1			N52 SFO	N52 SF1			N84 SFO	N84 SF1			N116 SFO	N116 SF1	N116 SF2	N116 SF3	N148 SFO			N180 SFO				N212 SFO				N244 SFO	N244 SF1	N244 SF2		
21	N21 SFO	N21 SF1	N21 SF2		N53 SFO	N53 SF1	N53 SF2		N85 SFO	N85 SF1	N85 SF2		N117 SFO	N117 SF1	N117 SF2	N117 SF3	N149 SFO	N149 SF1	N149 SF2	N181 SFO	N181 SF1	N181 SF2		N213 SFO	N213 SF1	N213 SF2		N245 SFO	N245 SF1	N245 SF2		
22	N22 SFO	N22 SF1			N54 SFO	N54 SF1			N86 SFO	N86 SF1			N118 SFO	N118 SF1	N118 SF2	N118 SF3	N150 SFO			N182 SFO				N214 SFO				N246 SFO	N246 SF1	N246 SF2		
23	N23 SFO	N23 SF1	N23 SF2		N55 SFO	N55 SF1	N55 SF2		N87 SFO	N87 SF1	N87 SF2		N119 SFO	N119 SF1	N119 SF2	N119 SF3	N151 SFO	N151 SF1	N151 SF2	N183 SFO	N183 SF1	N183 SF2		N215 SFO	N215 SF1	N215 SF2		N247 SFO	N247 SF1	N247 SF2		
24	N24 SFO	N24 SF1			N56 SFO	N56 SF1			N88 SFO	N88 SF1			N120 SFO	N120 SF1	N120 SF2	N120 SF3	N152 SFO			N184 SFO				N216 SFO				N248 SFO	N248 SF1	N248 SF2		
25	N25 SFO	N25 SF1	N25 SF2		N57 SFO	N57 SF1	N57 SF2		N89 SFO	N89 SF1	N89 SF2		N121 SFO	N121 SF1	N121 SF2	N121 SF3	N153 SFO	N153 SF1	N153 SF2	N185 SFO	N185 SF1	N185 SF2		N217 SFO	N217 SF1	N217 SF2		N249 SFO	N249 SF1	N249 SF2		
26	N26 SFO	N26 SF1			N58 SFO	N58 SF1			N90 SFO	N90 SF1			N122 SFO	N122 SF1	N122 SF2	N122 SF3	N154 SFO			N186 SFO				N218 SFO				N250 SFO	N250 SF1	N250 SF2		
27	N27 SFO	N27 SF1	N27 SF2		N59 SFO	N59 SF1	N59 SF2		N91 SFO	N91 SF1	N91 SF2		N123 SFO	N123 SF1	N123 SF2	N123 SF3	N155 SFO	N155 SF1	N155 SF2	N187 SFO	N187 SF1	N187 SF2		N219 SFO	N219 SF1	N219 SF2		N251 SFO	N251 SF1	N251 SF2		
28	N28 SFO	N28 SF1			N60 SFO	N60 SF1			N92 SFO	N92 SF1			N124 SFO	N124 SF1	N124 SF2	N124 SF3	N156 SFO			N188 SFO				N220 SFO				N252 SFO	N252 SF1	N252 SF2		
29	N29 SFO	N29 SF1	N29 SF2		N61 SFO	N61 SF1	N61 SF2		N93 SFO	N93 SF1	N93 SF2		N125 SFO	N125 SF1	N125 SF2	N125 SF3	N157 SFO	N157 SF1	N157 SF2	N189 SFO	N189 SF1	N189 SF2		N221 SFO	N221 SF1	N221 SF2		N253 SFO	N253 SF1	N253 SF2		
30	N30 SFO	N30 SF1			N62 SFO	N62 SF1			N94 SFO	N94 SF1			N126 SFO	N126 SF1	N126 SF2	N126 SF3	N158 SFO			N190 SFO				N222 SFO				N254 SFO	N254 SF1	N254 SF2		
31	N31 SFO	N31 SF1	N31 SF2		N63 SFO	N63 SF1	N63 SF2		N95 SFO	N95 SF1	N95 SF2		N127 SFO	N127 SF1	N127 SF2	N127 SF3	N159 SFO	N159 SF1	N159 SF2	N191 SFO	N191 SF1	N191 SF2		N223 SFO	N223 SF1	N223 SF2		N255 SFO	N255 SF1	N255 SF2		
per-byte sub-column																																
SFB ID = 10																																

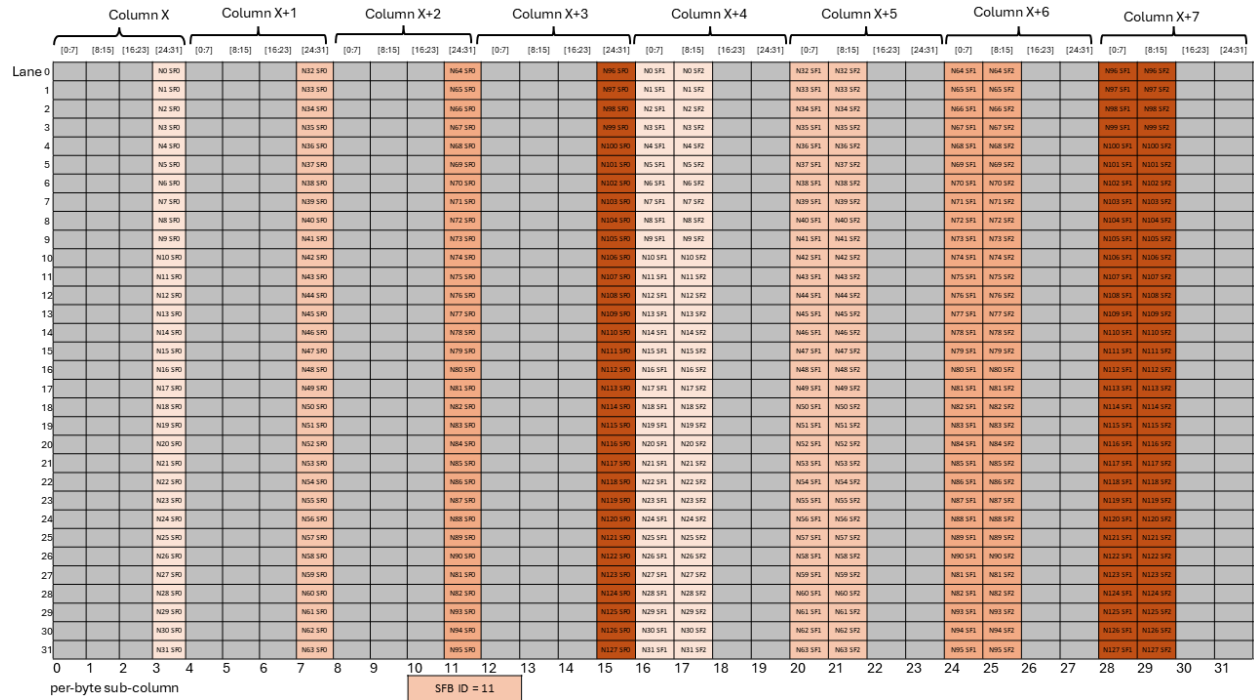


Figure 246: Layout of scale factor B matrix with block32 with K=96 and N≤128 with SFA_ID=11

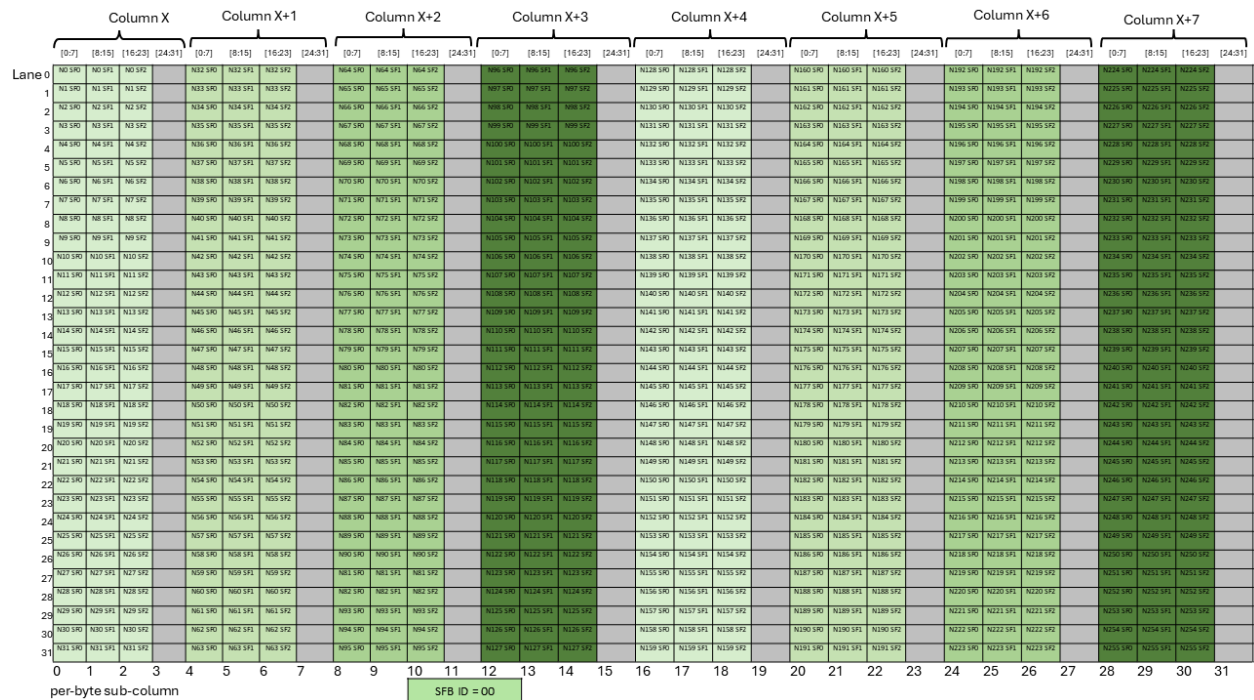


Figure 247: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=00

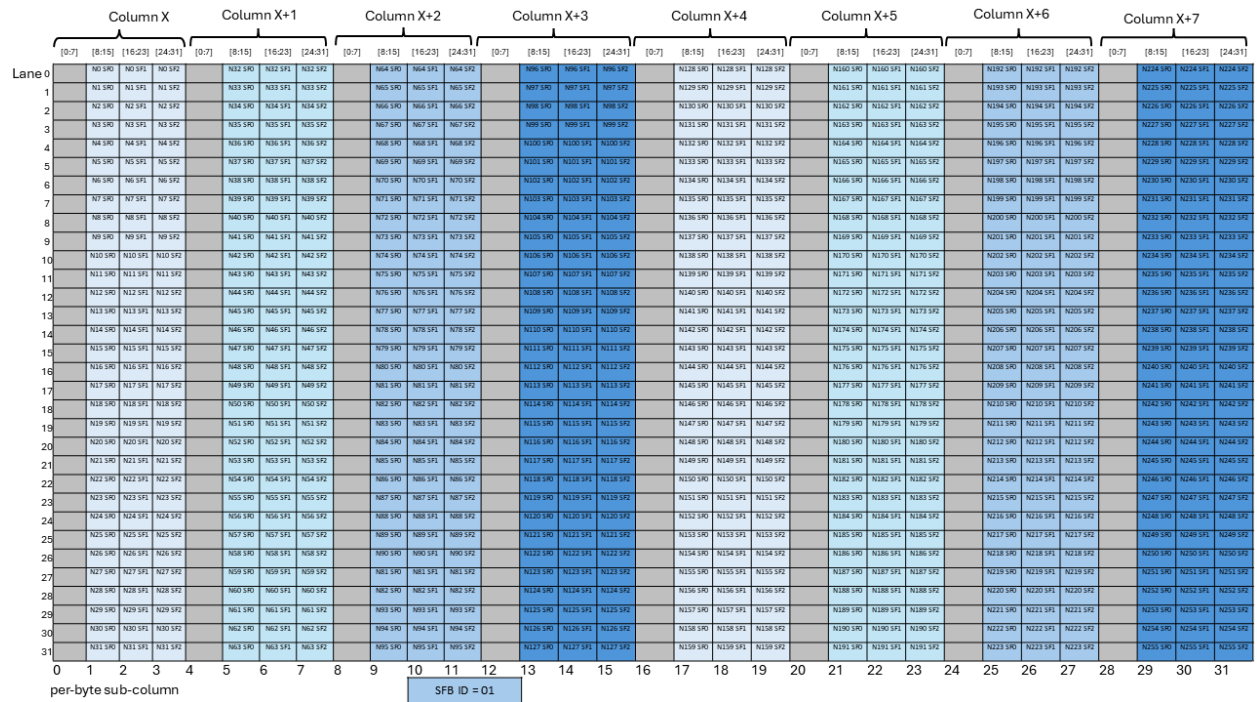


Figure 248: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=01

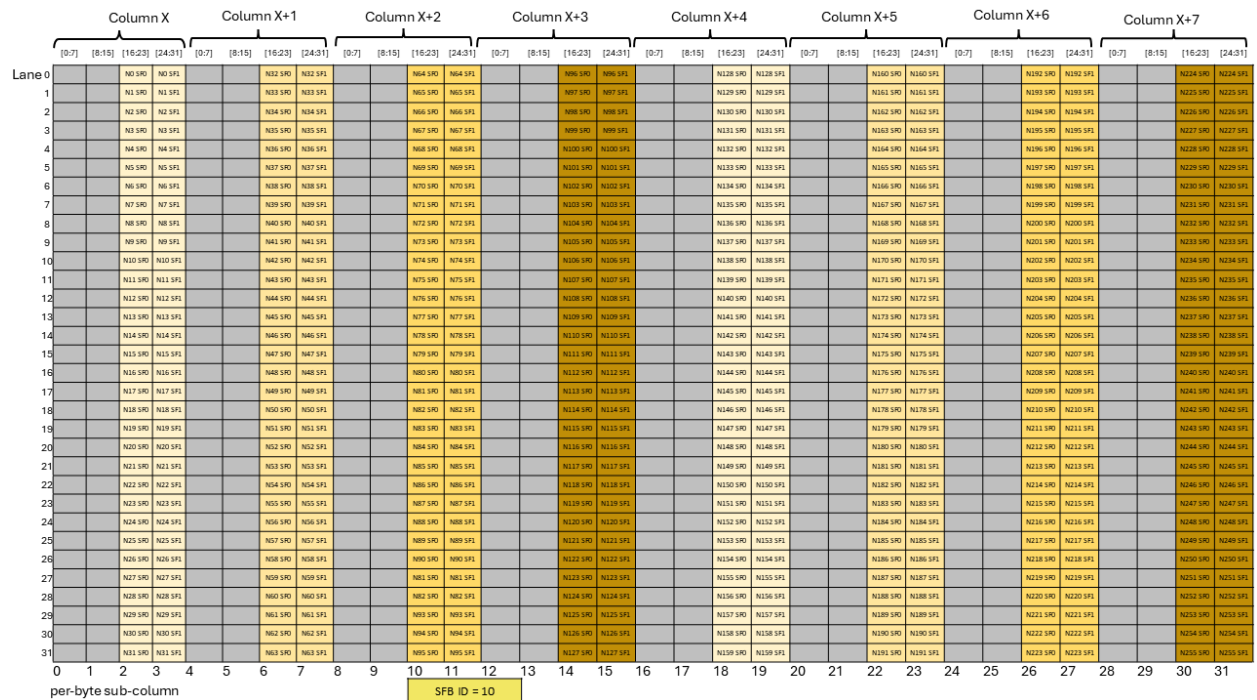


Figure 249: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=10

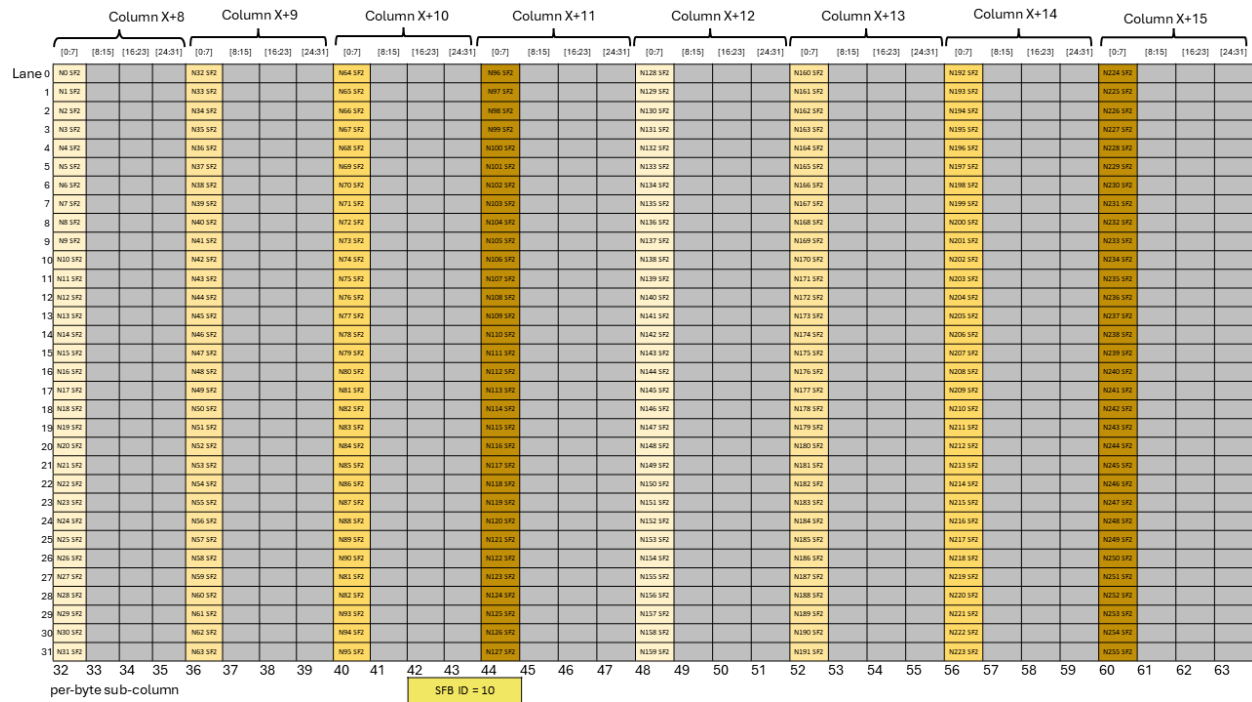


Figure 250: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=10

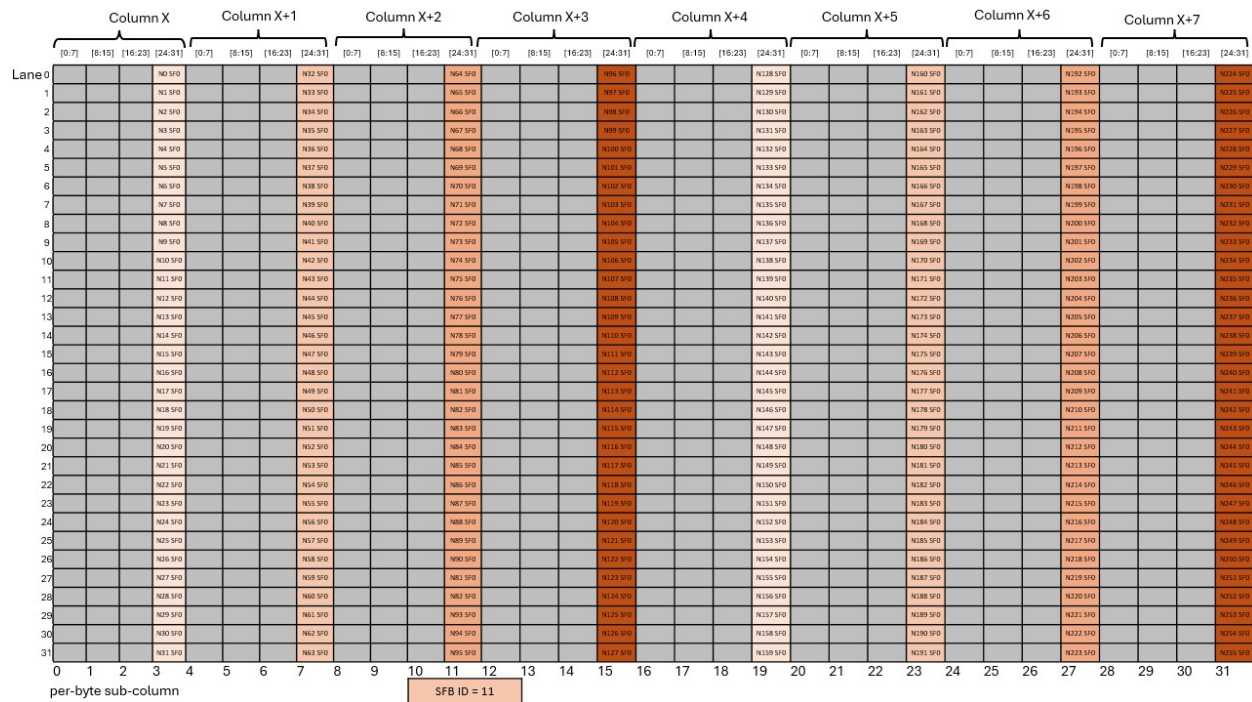


Figure 251: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=11

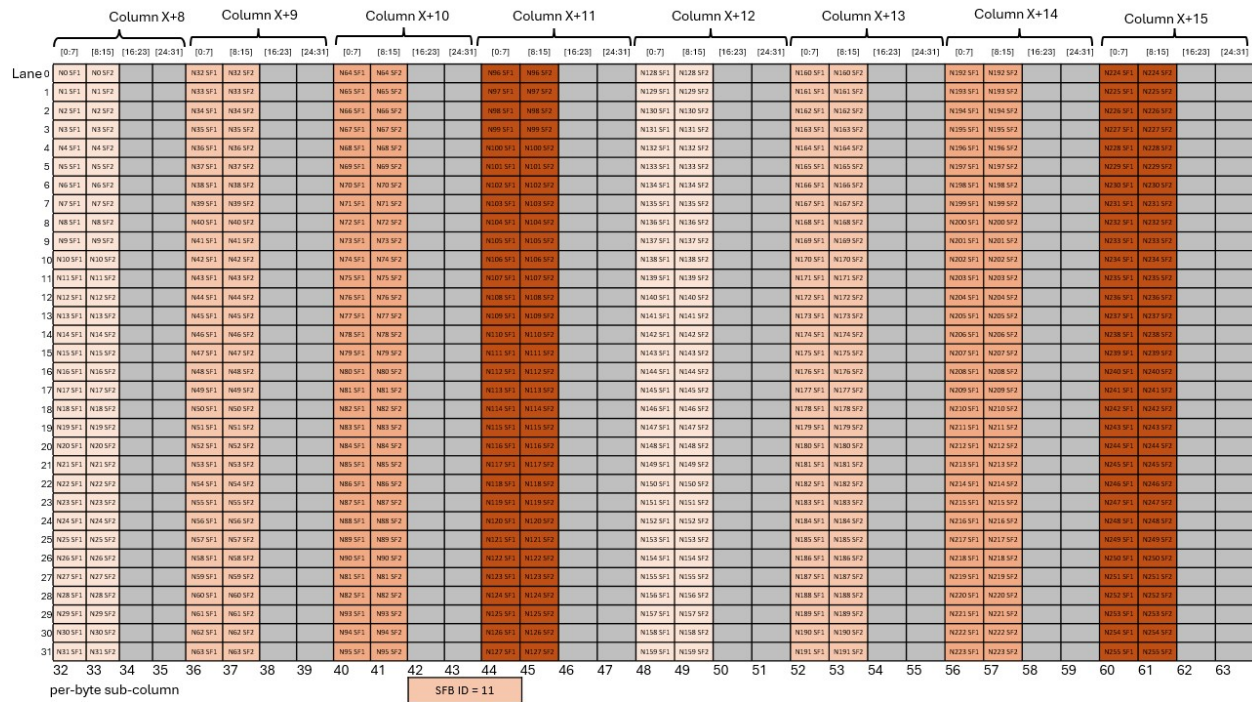


Figure 252: Layout of scale factor B matrix with block32 with K=96 and N>128 with SFA_ID=11

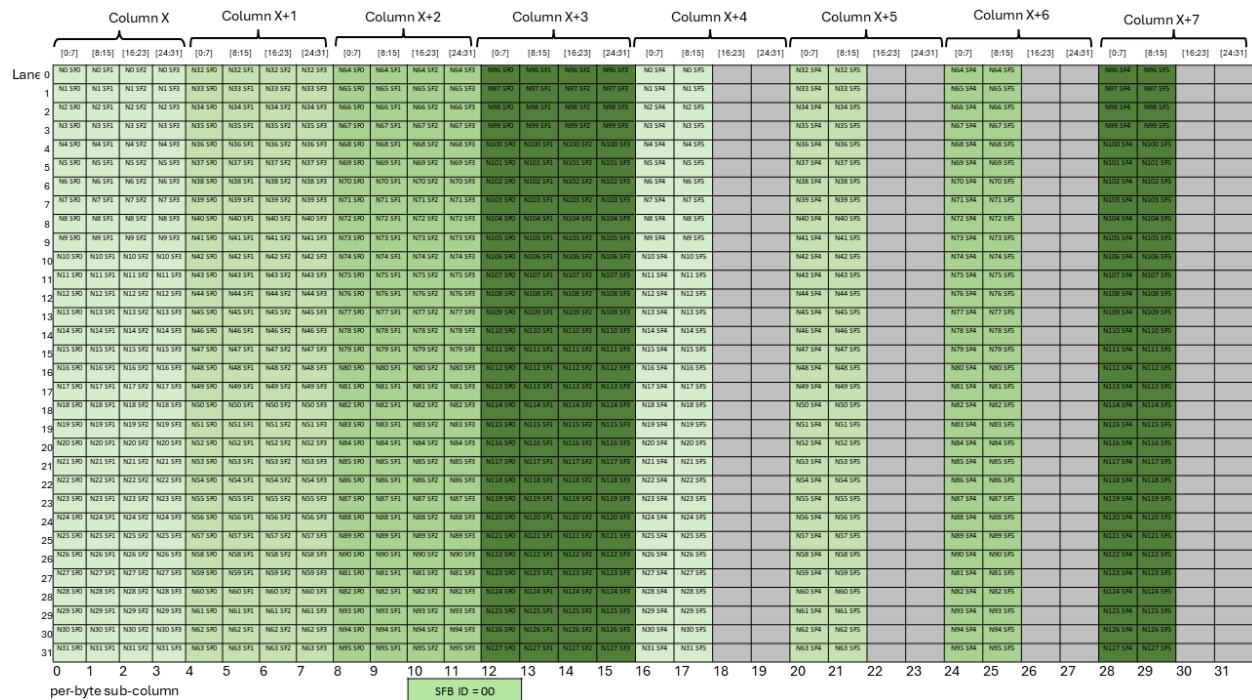


Figure 253: Layout of scale factor B matrix with block16 with K=96 and N<=128 with SFA_ID=00

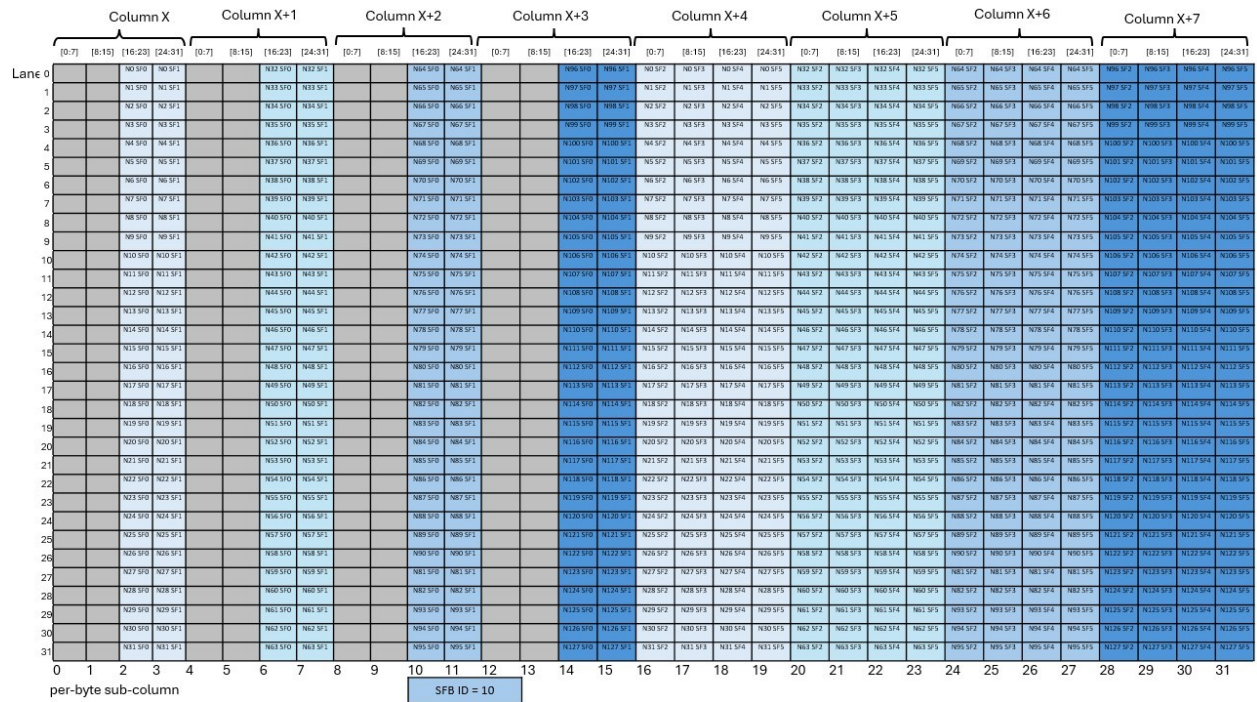


Figure 254: Layout of scale factor B matrix with block 16 with K=96 and N≤128 with SFA_ID=10

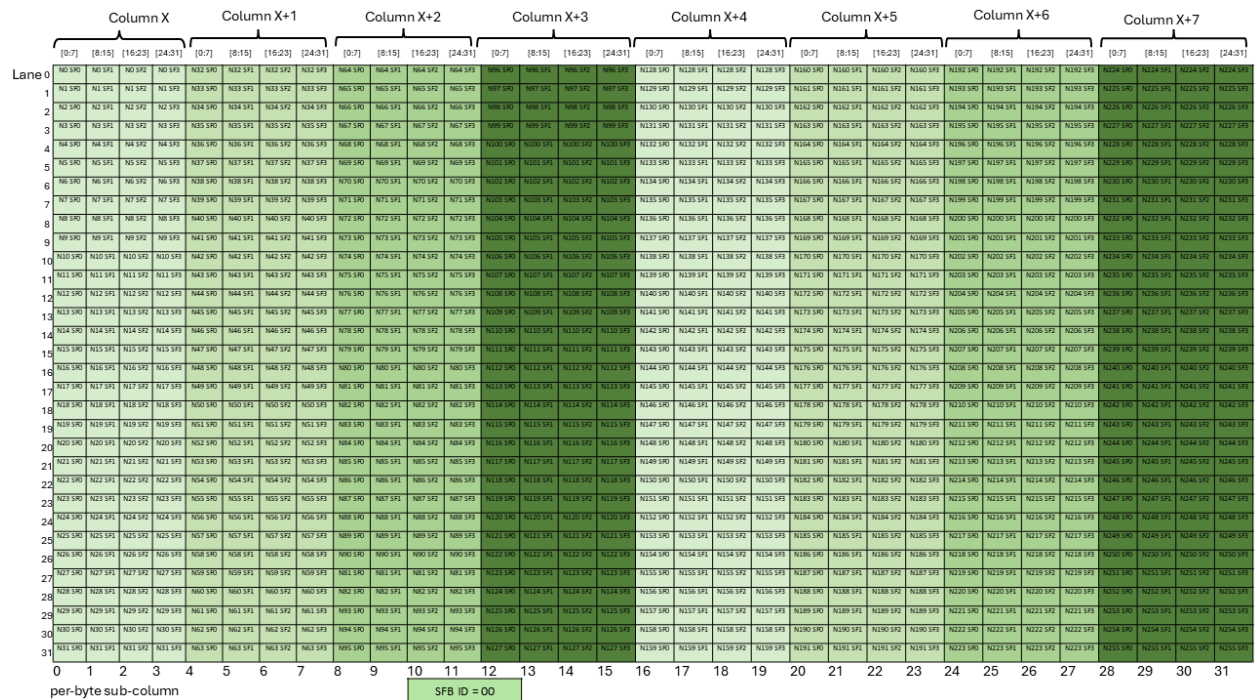


Figure 255: Layout of scale factor B matrix with block 16 with K=96 and N>128 with SFA_ID=00

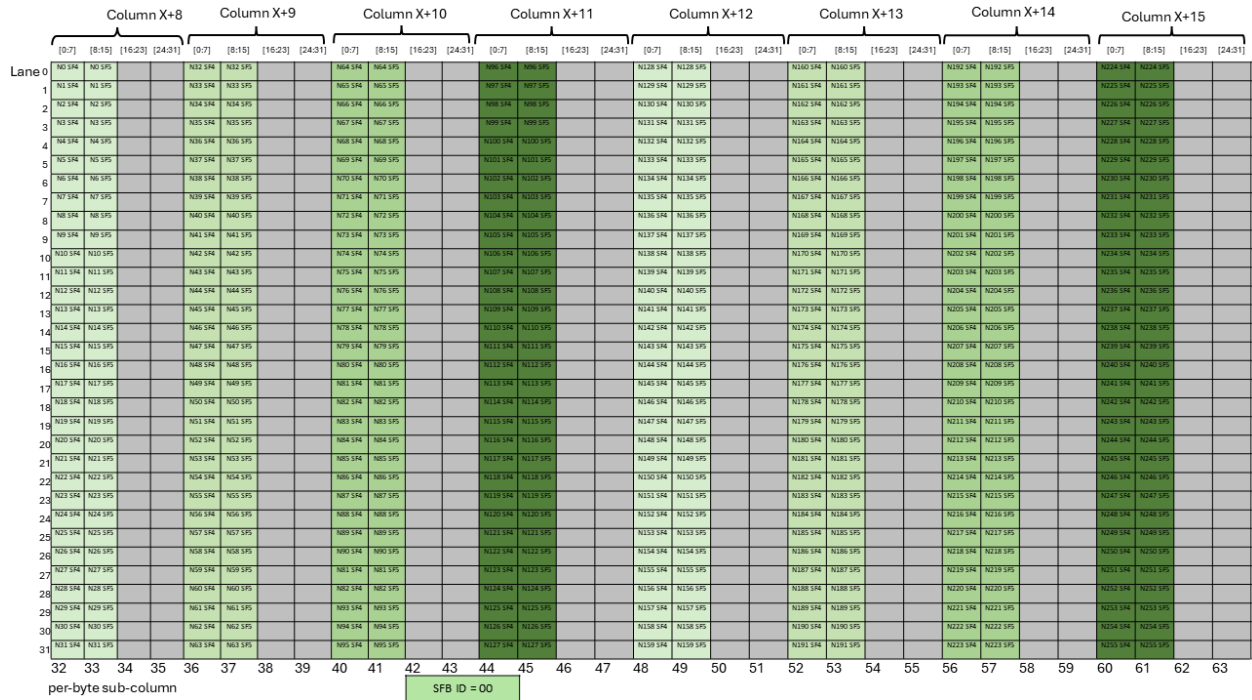


Figure 256: Layout of scale factor B matrix with block16 with K=96 and N>128 with SFA_ID=00

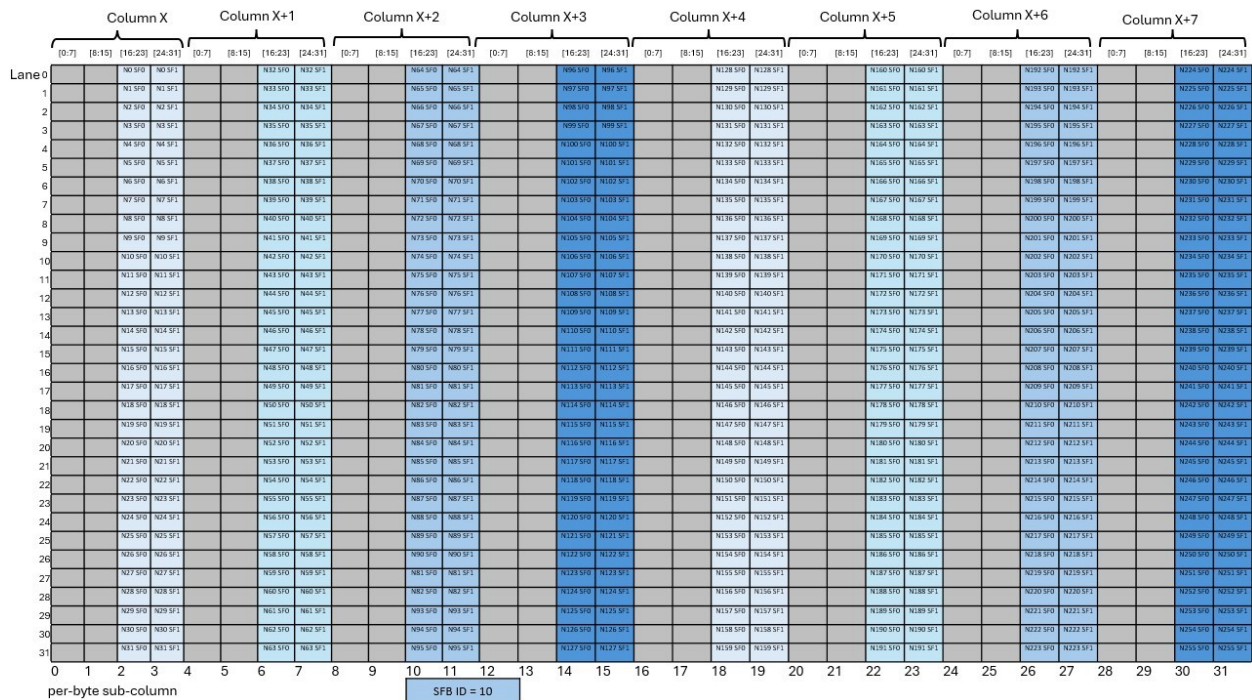


Figure 257: Layout of scale factor B matrix with block16 with K=96 and N>128 with SFA_ID=10

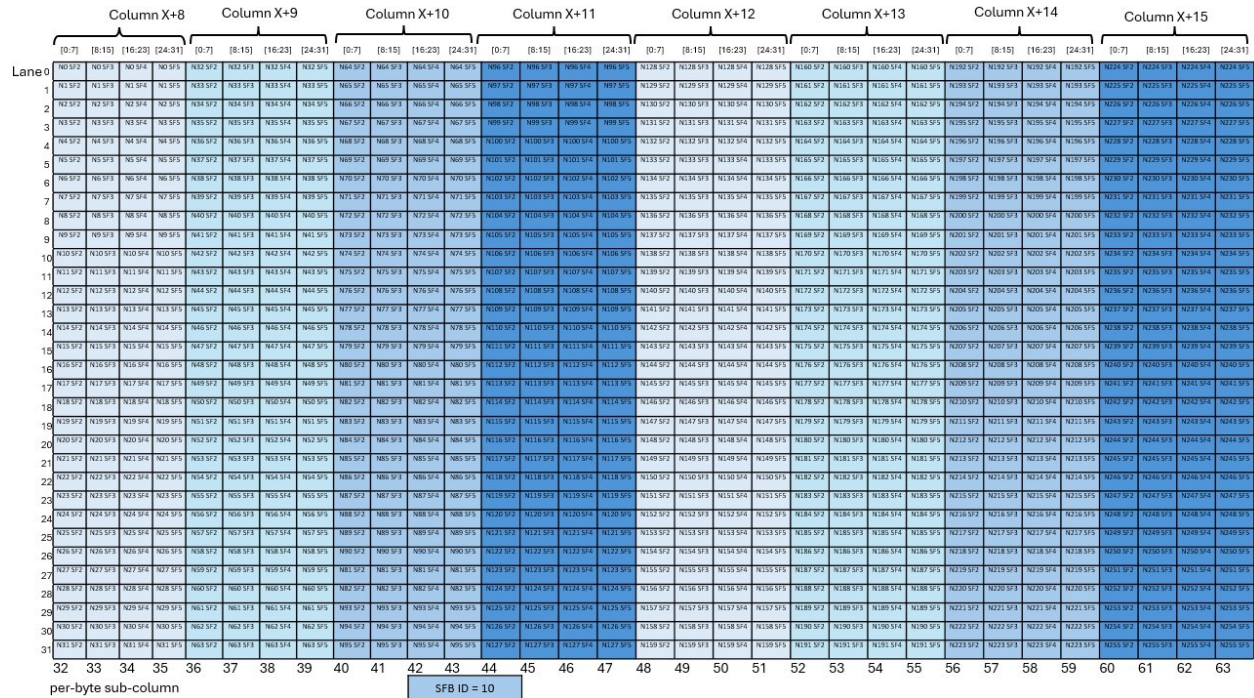


Figure 258: Layout of scale factor B matrix with block16 with K=96 and N>128 with SFA_ID=10

For example, if *SFB_ID* is 0, then all the green columns are selected to form the scale factor matrix. Similarly, if *SFB_ID* is 2, then all of the blue columns are selected to form the scale factor matrix.

9.7.16.10.8 Sparse Matrices

This instruction `tcgen05.mma.sp` can be used when the matrix A is a structured sparse matrix with 50% zeros in each row distributed as per its sparse granularity.

In a $M \times N \times K$ sparse `tcgen05.mma.sp` operation, the matrix A of shape $M \times K$ is stored in a packed form as $M \times (K/2)$ in memory. For each K -wide row of matrix A, 50% of elements are zeros and the remaining $K/2$ non-zero elements are stored in memory. The metadata specifies the mapping of the $K/2$ non-zero elements to the K elements before performing the MMA operation.

Granularity of sparse matrix A is defined as the ratio of the number of non-zero elements in a sub-chunk of the matrix row to the total number of elements in that sub-chunk where the size of the sub-chunk is shape-specific. The following table lists the granularity of different `tcgen05.mma.sp` variants:

.kind of tcgen05.mma	Sparse Granularity
<code>.kind::tf32</code>	1:2
<code>.kind::f16</code>	2:4
<code>.kind::f8f6f4</code>	
<code>.kind::mxf8f6f4</code>	
<code>.kind::i8</code>	
<code>.kind::mxf4</code>	4:8 (in pairs)

9.7.16.10.8.1 Sparse tcgen05.mma.sp with `.kind::tf32`

For `.kind::tf32`, matrix A is structured sparse at a granularity of 1:2. In other words, each chunk of two adjacent elements in a row of matrix A has one zero and one non-zero element. Only the non-zero element is stored in memory and the 4-bit index in the metadata indicates the position of the non-zero element in the two-wide chunk. The only meaningful values of the index are:

- 0b1110
- 0b0100

Rest of the values result in undefined behavior.

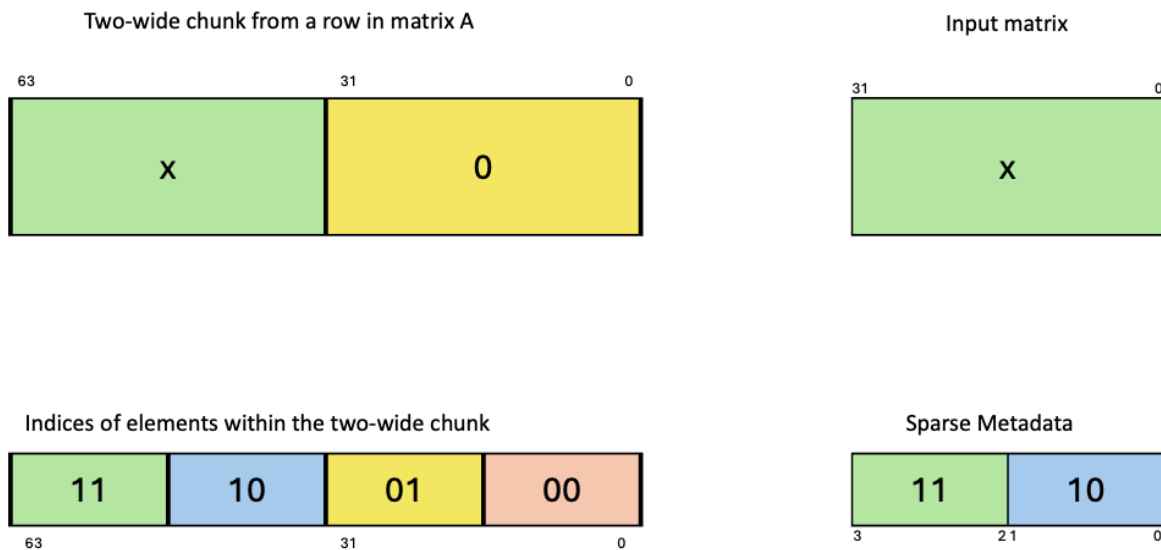


Figure 259: Sparse tcgen05.mma metadata example for `tf32` kind

9.7.16.10.8.2 Sparse tcgen05.mma.sp with .kind::f16, .kind::f8f6f4, .kind::mxf8f6f4, .kind::i8

For the following .kind variants of tcgen05.mma:

- .kind::f16
- .kind::f8f8f4
- .kind::mxf8f6f4
- .kind::i8

matrix A is structured sparse at a granularity of 2:4. In other words, each chunk of four adjacent elements in a row of matrix A has two zero and two non-zero elements. Only the non-zero elements are stored in memory and the two 2-bit indices in the metadata indicates the position of the two non-zero elements in the four-wide chunk. The only meaningful values of the index are:

- 0b0100
- 0b1000
- 0b1100
- 0b1001
- 0b1101
- 0b0110
- 0b1110

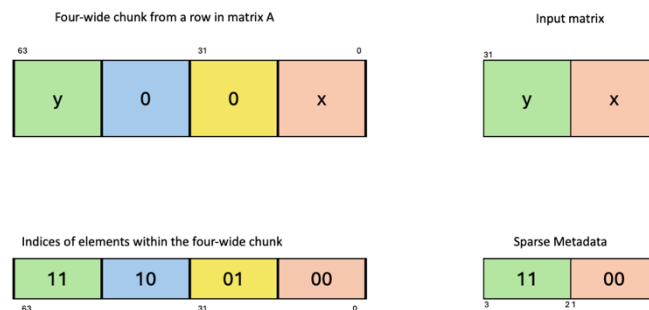


Figure 260: Sparse tcgen05.mma metadata example for f16/f8f6f4/mxf8f6f4 kind

9.7.16.10.8.3 Sparse tcgen05.mma.sp with .kind::mxf4 and .kind::mxf4nvf4

For .kind::mxf4 and .kind::mxf4nvf4, matrix A is pair-wise structured sparse at a granularity of 4:8. In other words, each chunk of eight adjacent elements in a row of matrix A has four zero and four non-zero elements. The zero and non-zero elements are clustered in sub-chunks of two elements each within the eight-wide chunk, so each two-wide sub-chunk within the eight-wide chunk must be all zeros or all non-zeros. Only the four non-zero elements are stored in memory and the two 2-bit indices in the metadata indicates the position of the two two-wide sub-chunks with non-zero values in the eight-wide chunk of a row of matrix A. The only meaningful values of the index are:

- 0b0100
- 0b1000

- ▶ 0b1100
- ▶ 0b1001
- ▶ 0b1101
- ▶ 0b0110
- ▶ 0b1110

Rest of the values result in undefined behavior.

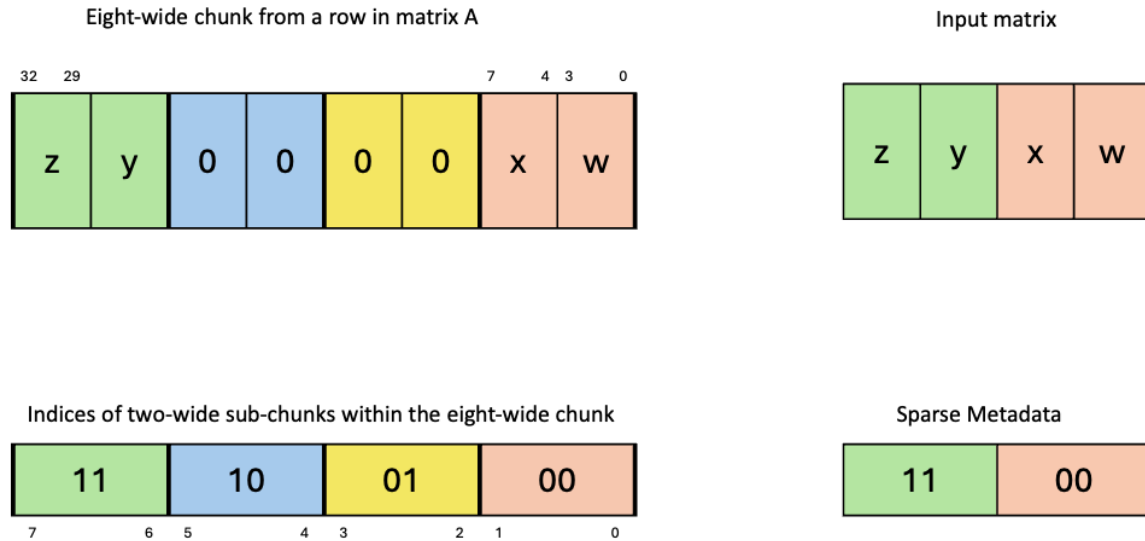


Figure 261: Sparse tcgen05.mma metadata example for `mx4` kind

9.7.16.10.8.4 Sparsity selector

The value of the sparsity selector selects the sub-columns in the Tensor Memory to form the sparsity metadata matrix, which is used with matrix A to form the multiplicand matrix.

The following shows the sparse metadata matrix layout in Tensor Memory for various MMA variants:

9.7.16.10.8.5 Layout of the Sparsity Metadata Matrix for `M = 64` for `.kind::f16`

Figure 262 shows which sub-columns gets selected for different values of Sparsity Selector.

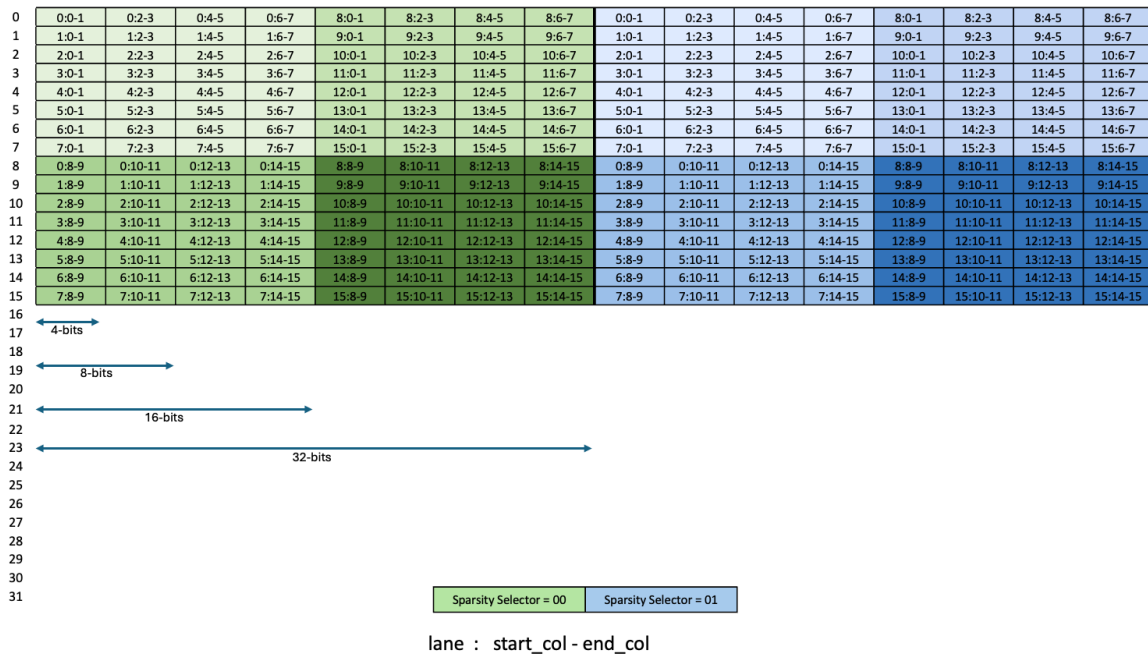


Figure 262: Sparsity Metadata Layout for M = 64 for .kind::f16

9.7.16.10.8.6 Layout of the Sparsity Metadata Matrix for M = 128 / M = 256 for .kind::f16

Figure 263 shows which sub-columns gets selected for different values of Sparsity Selector.

9.7.16.10.8.7 Layout of the Sparsity Metadata Matrix for M = 64 for .kind::tf32

Figure 264 shows which sub-columns gets selected for different values of Sparsity Selector.

9.7.16.10.8.8 Layout of the Sparsity Metadata Matrix for M = 128 / M = 256 for .kind::tf32

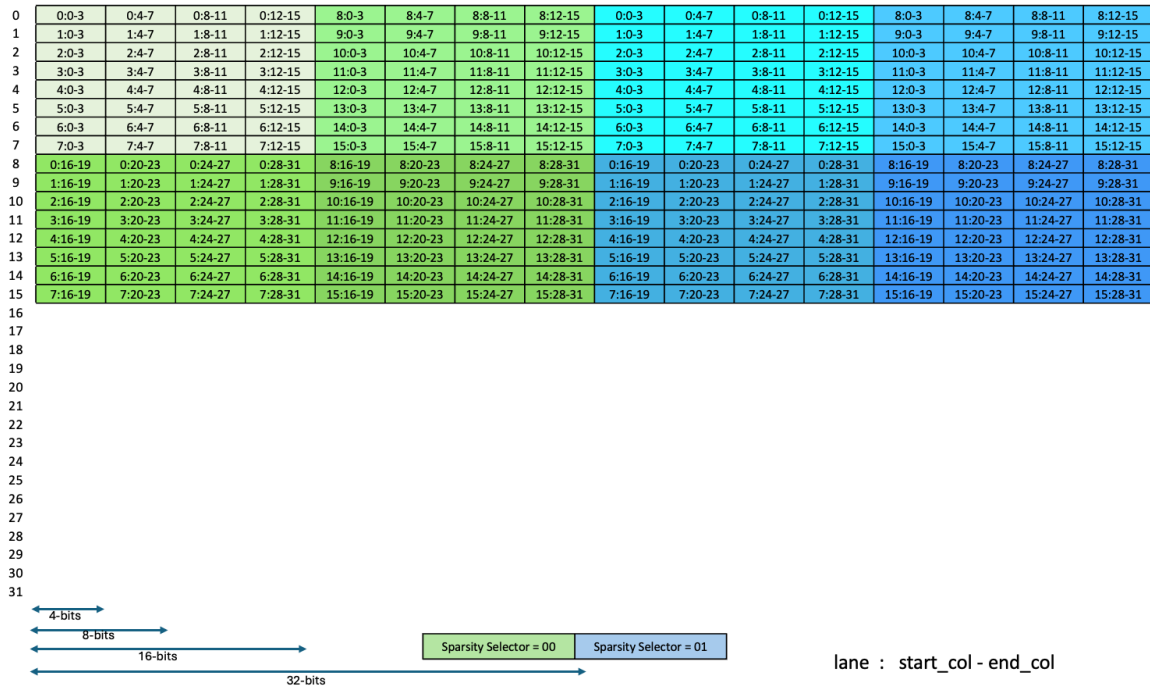
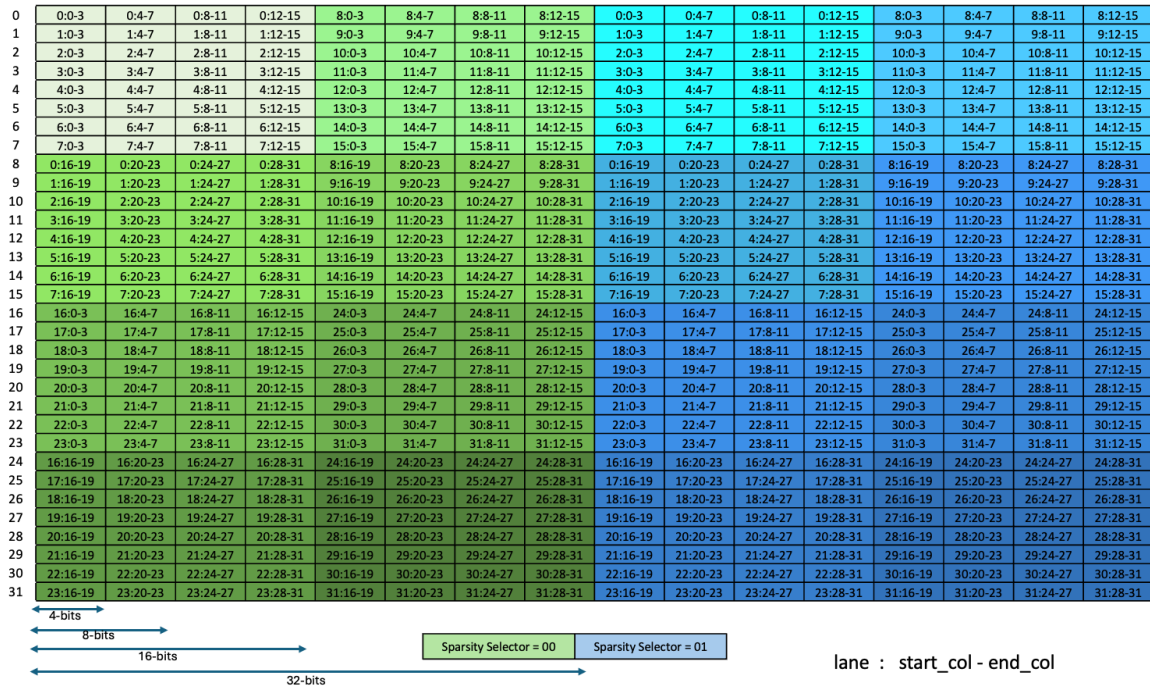
Figure 265 shows which sub-columns gets selected for different values of Sparsity Selector.

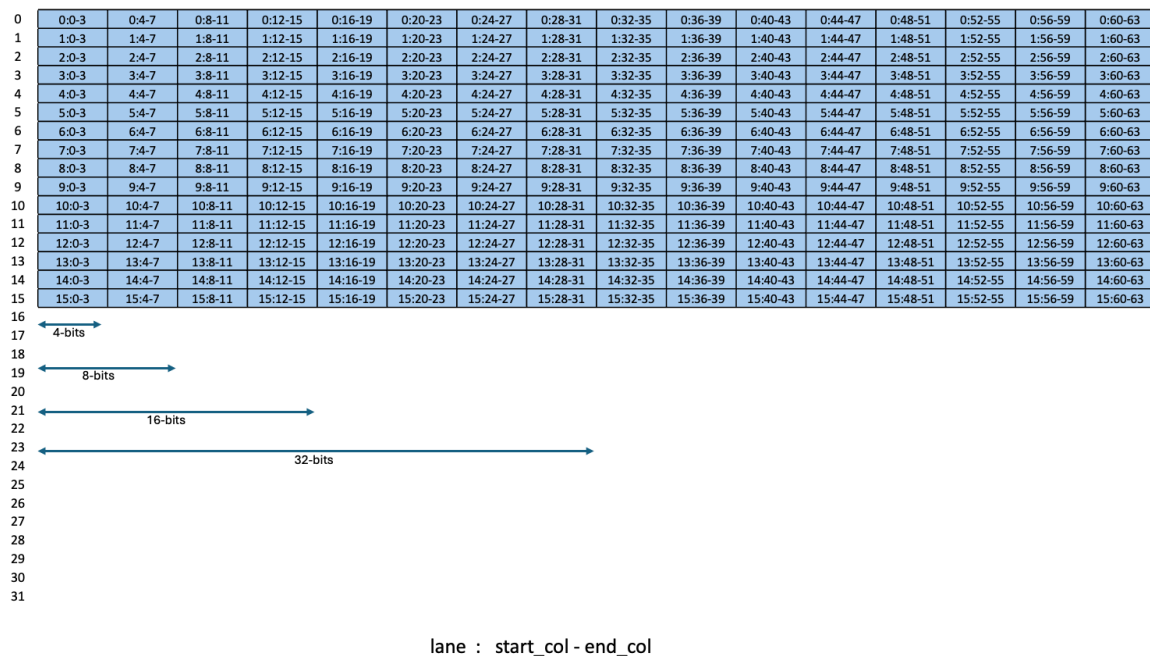
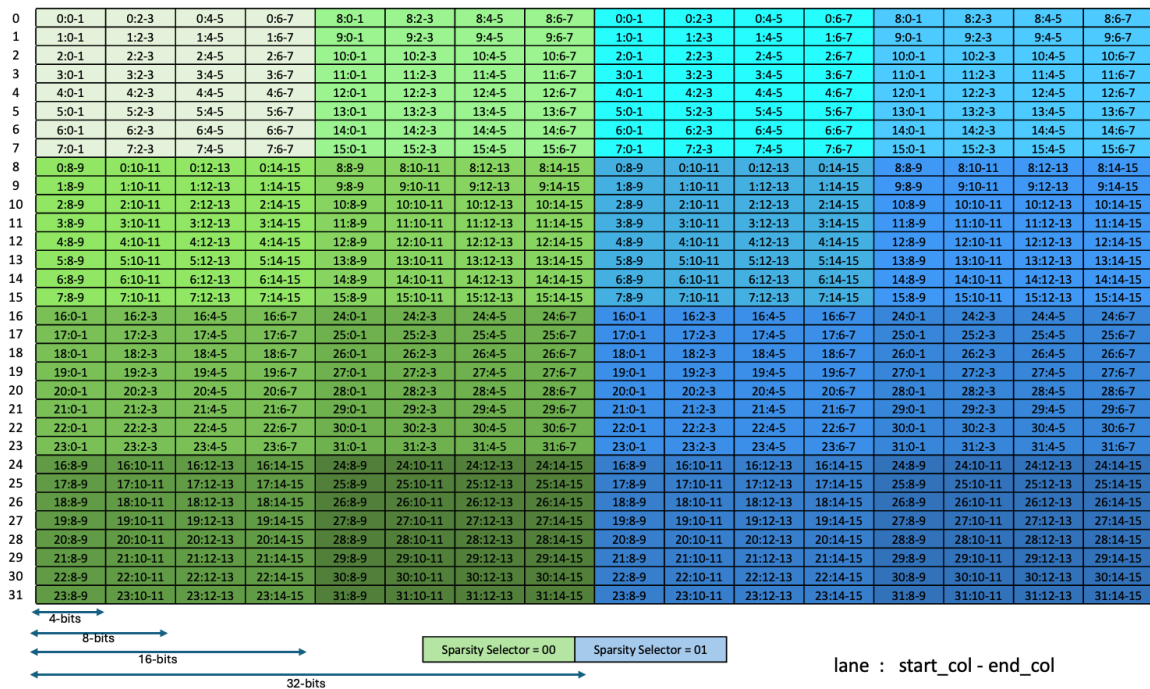
9.7.16.10.8.9 Layout of the Sparsity Metadata Matrix for M = 64 for .kind::f8f6f4, .kind::mxf8f6f4, .kind::i8, .kind::mxf4, .kind::mxf4nvvf4

The value of the sparsity selector:

- must be 0 for .kind::i8 and .kind::f8f6f4
- is assumed to be 0 for .kind::mxf8f6f4, .kind::mxf4 and .kind::mxf4nvvf4

and all of the columns are selected as shown in Figure 266





9.7.16.10.8.10 Layout of the Sparsity Metadata Matrix for M = 128 / M = 256 for .kind::f8f6f4, .kind::mx8f6f4, .kind::i8, .kind::mx4, .kind::mx4nvvf4

The value of the sparsity selector:

- must be 0 for .kind::i8 and .kind::f8f6f4
- is assumed to be 0 for .kind::mx8f6f4, .kind::mx4 and .kind::mx4nvvf4

and all of the columns are selected as shown in Figure 267

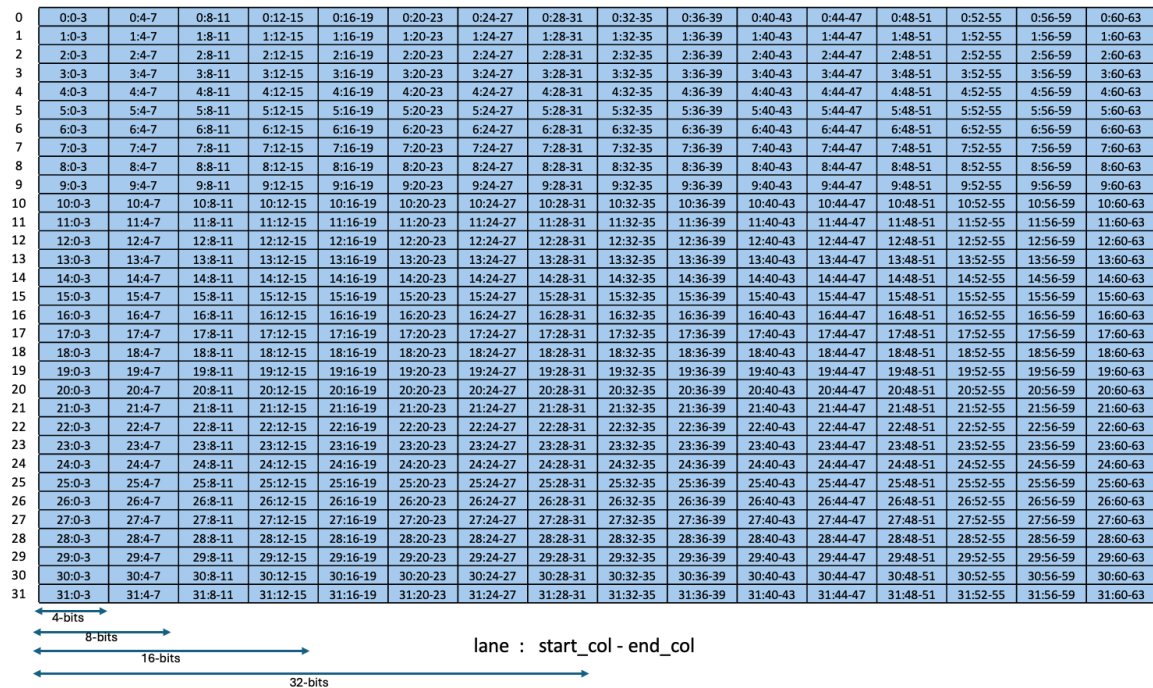


Figure 267: Sparsity Metadata Layout for M = 128 / M = 256 for .kind::f8f6f4, .kind::mx8f6f4, .kind::i8, .kind::mx4, .kind::mx4nvvf4

9.7.16.10.8.11 Alignment restriction

The layouts which utilize only half the datapath lanes as specified in *Data Path Layout Organization*, i.e. *Layout F* and *Layout C*, must use the same alignment across matrices A, D and the sparsity metadata matrix.

9.7.16.10.9 TensorCore 5th Generation of MMA Instructions

9.7.16.10.9.1 TensorCore 5th Generation Instructions: tcgen05.mma

tcgen05.mma

Perform the 5th generation of matrix multiply and accumulate operation.

Syntax

```
// 1. Floating-point type without block scaling:

tcgen05.mma.cta_group.kind    [d-tmem], a-desc, b-desc, idesc,
                               { disable-output-lane }, enable-input-d {, scale-input-d}
↪;

tcgen05.mma.cta_group.kind    [d-tmem], [a-tmem], b-desc, idesc,
                               { disable-output-lane }, enable-input-d {, scale-input-d}
↪;

.kind      = { .kind::f16, .kind::tf32, .kind::f8f6f4 }
.cta_group = { .cta_group::1, .cta_group::2 }

-----

// 2. Floating-point type with block scaling:

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}
                                     [d-tmem], a-desc, b-desc, idesc,
                                     [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}
                                     [d-tmem], [a-tmem], b-desc, idesc,
                                     [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

.kind = { .kind::mxf8f6f4, .kind::mxf4, .kind::mxf4nvf4 }
.cta_group      = { .cta_group::1, .cta_group::2 }
.scale_vectorsize = { .scale_vec::1X, .scale_vec::2X, .scale_vec::4X, .block16, .
↪block32 }

-----

// 3. Convolution MMA for floating-point type without block scaling:

tcgen05.mma.cta_group.kind.collector_usage [d-tmem], a-desc, b-desc, idesc,
                                             { disable-output-lane }, enable-input-d {,
↪scale-input-d};

tcgen05.mma.cta_group.kind{.ashift}.collector_usage [d-tmem], [a-tmem], b-desc, idesc,
                                                       { disable-output-lane }, enable-
↪input-d {, scale-input-d};

tcgen05.mma.cta_group.kind.ashift{.collector_usage} [d-tmem], [a-tmem], b-desc, idesc,
                                                       { disable-output-lane }, enable-
↪input-d {, scale-input-d};

.kind      = { .kind::f16, .kind::tf32, .kind::f8f6f4 }
.cta_group = { .cta_group::1, .cta_group::2 }
.collector_usage = { .collector::buffer::op }
::buffer        = { ::a }
::op             = { ::fill, ::use, ::lastuse, ::discard* }

-----
```

(continues on next page)

(continued from previous page)

```
// 4. Activation Stationary MMA for floating-point type with block scaling:

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}.collector_usage
                                [d-tmem], a-desc, b-desc, idesc,
                                [scale-A-tmem], [scale-B-tmem], enable-
↪input-d;

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}.collector_usage
                                [d-tmem], [a-tmem], b-desc, idesc,
                                [scale-A-tmem], [scale-B-tmem], enable-
↪input-d;

.cta_group      = { .cta_group::1,   .cta_group::2 }
.scale_vectorsize = { .scale_vec::1X, .scale_vec::2X, .scale_vec::4X, .block16, .
↪block32 }
.kind           = { .kind::mxf8f6f4, .kind::mxf4, .kind::mxf4nvf4 }
.collector_usage = { .collector::buffer::op }
::buffer        = { ::a }
::op            = { ::fill, ::use, ::lastuse, ::discard* }

-----

// 5. Integer type:

tcgen05.mma.cta_group.kind::i8 [d-tmem], a-desc, b-desc, idesc,
                                { disable-output-lane }, enable-input-d;

tcgen05.mma.cta_group.kind::i8 [d-tmem], [a-tmem], b-desc, idesc,
                                { disable-output-lane }, enable-input-d;

.cta_group = { .cta_group::1,   .cta_group::2 }

-----

// 6. Convolution MMA for integer type:

tcgen05.mma.cta_group.kind::i8.collector_usage [d-tmem], a-desc, b-desc,
↪idesc,
                                { disable-output-lane },
↪enable-input-d;

tcgen05.mma.cta_group.kind::i8.ashift{.collector_usage} [d-tmem], [a-tmem], b-desc,
↪idesc,
                                { disable-output-lane },
↪enable-input-d;

tcgen05.mma.cta_group.kind::i8{.ashift}.collector_usage [d-tmem], [a-tmem], b-desc,
↪idesc,
                                { disable-output-lane },
↪enable-input-d;

.cta_group      = { .cta_group::1,   .cta_group::2 }
.collector_usage = { .collector::buffer::op }
::buffer        = { ::a }
::op            = { ::fill, ::use, ::lastuse, ::discard* }
```

Description

Instruction `tcgen05.mma` is an asynchronous instruction which initiates an $M \times N \times K$ matrix multiply and accumulate operation, $D = A * B + D$ where the A matrix is $M \times K$, the B matrix is $K \times N$, and the D matrix is $M \times N$.

The operation of the form $D = A * B$ is issued when the input predicate argument `enable-input-d` is false.

The optional immediate argument `scale-input-d` can be specified to scale the input matrix D as follows: $D = A * B + D * (2^{-\text{scale-input-d}})$

The valid range of values for argument `scale-input-d` is [0, 15]. The argument `scale-input-d` is only valid for `.kind::tf32` and `.kind::f16`.

The 32-bit register operand `idesc` is the instruction descriptor as described in [Instruction descriptor](#), specifies the shapes, exact types, sparsity and other details of the input matrices, output matrix and the matrix multiply and accumulate operation.

The qualifier `.cta_group::1` specifies that the matrix multiply and accumulate operation is performed on the [Tensor Memory](#) of the executing thread's CTA only. The qualifier `.cta_group::2` specifies that the matrix multiply and accumulate operation is performed on the [Tensor Memory](#) of the executing thread's CTA and its [peer CTA](#).

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The instruction `tcgen05.mma` has single thread semantics, unlike the collective instructions `mma.sync` or `wgmma.mma_async`. So, a single thread issuing the `tcgen05.mma` will result in the initiation of the whole matrix multiply and accumulate operation. Refer to the section [Issue Granularity](#).

The qualifier `.kind` specifies the general kind of the element types of the multiplicand matrices. The exact types of the elements of the input and output matrices for each MMA-kind are specified in the [Instruction descriptor](#).

The address operand `d-tmem` specifies the address of the destination and the accumulation matrix D in the [Tensor Memory](#). The address operand `a-tmem` specifies the address of the matrix A in the [Tensor Memory](#). The 64-bit register operand `a-desc` and `b-desc` are the matrix descriptors which represent the matrices A and B in shared memory respectively. The format of the matrix descriptor is described in [Matrix Descriptors](#).

The vector operand `disable-output-lane` specifies the lane(s) in the [Tensor Memory](#) that should be not be updated with the resultant matrix D. Elements of the vector operand `disable-output-lane` forms a mask where each bit corresponds to a lane of the [Tensor Memory](#), with least significant bit of the first element of the vector (leftmost in syntax) corresponding to the lane 0 of the [Tensor Memory](#). If a bit in the mask is 1, then the corresponding lane in the Tensor Memory for the resultant matrix D will not be updated. The size of the vector is as follows:

<code>.cta_group</code>	Size of the vector <code>disable-output-lane</code>
<code>::1</code>	4
<code>::2</code>	8

Qualifier `.block_scale` specifies that the matrices A and B are scaled with `scale_A` and `scale_B` matrices respectively before performing the matrix multiply and accumulate operation as specified in the section [Block Scaling](#). The address operand `scale-A-tmem` and `scale-B-tmem` specify the base address the matrices `scale_A` and `scale_B` respectively in the [Tensor Memory](#).

For qualifier `.scale_vectorsize`,

- If `.scale_vec::NX` is specified: N specifies the number of columns in `scale_A` matrix and number of rows in `scale_B` matrix.
- If `.blockN` is specified: N specifies the block size for which single scale factor will be applied. In this form, value of N is same as the K-dimension / (N of `.scale_vec::NX`).

Aliased `.scale_vectorsize` variants:

1. `.block16` is aliased with:
 1. `.scale_vec::4X` when `.kind = .kind::mxf4nvf4` and $K = 64$ or 128
2. `.block32` is aliased with:
 1. `.scale_vec::1X` when `.kind = .kind::mxf8f6f4` for all supported values of K
 2. `.scale_vec::2X` when `.kind = .kind::mxf4` or `.kind::mxf4nvf4` and $K = 64$ or 128

The valid combinations of MMA-kind and `.scale_vectorsize` are described in [Table 56](#). For `.kind::mxf4` when the qualifier `.scale_vectorsize` is not specified, then it defaults to `.block32`. For `.kind::mxf4nvf4`, the qualifier `.scale_vectorsize` must be explicitly specified.

The qualifier `.ashift` shifts the rows of the A matrix down by one row, except for the last row in the [Tensor Memory](#). Qualifier `.ashift` is only allowed with $M = 128$ or $M = 256$.

The qualifier `.collector_usage` specifies the usage of collector buffer for matrix A. Following collector buffer operations can be specified:

<code>.collector_usage</code>	Semantics
<code>.collector::a::fill</code>	Specifies that the A matrix read from the memory should be filled in collector buffer.
<code>.collector::a::use</code>	Specifies that the A matrix can be read from the collector buffer. This requires a previous fill to the collector buffer to be still valid.
<code>.collector::a::lastuse</code>	Specifies that the A matrix can be read from the collector buffer and the contents of the collector buffer can be discarded. This requires a previous fill to the collector buffer to be valid till the collector buffer is read.
<code>.collector::a::discard</code>	Specifies that the contents of the collector buffer for A can be discarded.

If no `.collector_usage` qualifier is specified, then it defaults to `.collector::a::discard`. It is illegal to specify either of `.collector::a::use` or `.collector::a::fill` along with `.ashift`.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Qualifier `.kind::mxf4nvf4` introduced in PTX ISA version 8.7.

Qualifiers `.block16` and `.block32` introduced in PTX ISA version 8.8.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8 except `.kind::i8`:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.kind::i8` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_110a`

Argument `scale-input-d` requires `sm_100a` and is supported on `sm_100f` or higher in the same family from PTX ISA version 8.8.

For `.scale_vectorsize`,

- ▶ `.scale_vec::1X`, `.scale_vec::2X`, `.scale_vec::4X` requires `sm_100a`.
- ▶ `.block16`, `.block32` requires `sm_100f` or `sm_110f`.

For Target ISA details on matrix shape, check [Target ISA Note](#).

For Target ISA details on shared memory descriptor, check [Target ISA Note](#).

Examples

```
tcgen05.mma.cta_group::1.kind::tf32      [taddr0], adesc, bdesc, idesc, {m0, m1, m2,  
↪ m3}, p;  
tcgen05.mma.cta_group::1.kind::mxf8f6f4 [taddr2], [taddr1], bdesc, idesc,  
                                         [tmem_scaleA], [tmem_scaleB], p;  
  
tcgen05.commit.cta_group::1.mbarrier::arrive::one.b64 [mbarObj0];  
  
loop:  
mbarrier.try_wait.parity.b64 p, [mbarObj0], 0;  
@!p bra loop;
```

9.7.16.10.9.2 TensorCore 5th Generation Instructions: `tcgen05.mma.sp`

`tcgen05.mma.sp`

Perform the 5th generation of matrix multiply and accumulate operation with sparse A matrix.

Syntax

```
// 1. Floating-point type without block scaling:

tcgen05.mma.sp.cta_group.kind [d-tmem], a-desc, b-desc, [sp-meta-tmem] , idesc,
                               { disable-output-lane }, enable-input-d{, scale-input-
↪d};

tcgen05.mma.sp.cta_group.kind [d-tmem], [a-tmem], b-desc, [sp-meta-tmem] , idesc,
                               { disable-output-lane }, enable-input-d{, scale-input-
↪d};

.kind          = { .kind::f16, , .kind::tf32, .kind::f8f6f4 }
.cta_group     = { .cta_group::1, .cta_group::2 }

-----

// 2. Floating-point type with block scaling:

tcgen05.mma.sp.cta_group.kind.block_scale{.scale_vectorsize}
                                         [d-tmem], a-desc, b-desc , [sp-meta-tmem] ,
↪ idesc,
                                         [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

tcgen05.mma.sp.cta_group.kind.block_scale{.scale_vectorsize}
                                         [d-tmem], [a-tmem], b-desc , [sp-meta-tmem] ,
↪ idesc,
                                         [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

.scale_vectorsize = { .scale_vec::1X, .scale_vec::2X, .scale_vec::4X, .block16, .
↪block32 }
.cta_group        = { .cta_group::1, .cta_group::2 }
.kind             = { .kind::mxf8f6f4, .kind::mxf4, .kind::mxf4nvf4 }

-----

// 3. Convolution MMA with floating-point type without block scaling:

tcgen05.mma.sp.cta_group.kind.collector_usage      [d-tmem], a-desc, b-desc,
                                                    [sp-meta-tmem] , idesc,
↪enable-input-d                                     { disable-output-lane },
                                                    {, scale-input-d};

tcgen05.mma.sp.cta_group.kind.ashift{.collector_usage} [d-tmem], [a-tmem], b-desc,
                                                         [sp-meta-tmem] , idesc,
↪enable-input-d                                     { disable-output-lane },
```

(continues on next page)

(continued from previous page)

```

                                {, scale-input-d};

tcgen05.mma.sp.cta_group.kind{.ashift}.collector_usage [d-tmem], [a-tmem], b-desc,
                                                        [sp-meta-tmem] , idesc,
                                                        { disable-output-lane },
↪enable-input-d
                                                        {, scale-input-d};

.kind          = { .kind::f16, .kind::tf32, .kind::f8f6f4 }
.collector_usage = { .collector::buffer::op }
::buffer        = { ::a }
::op             = { ::fill, ::use, ::lastuse, ::discard* }

-----

// 4. Activation Stationary MMA with floating-point type with block scaling:

tcgen05.mma.sp.cta_group.kind.block_scale{.scale_vectorsize}.collector_usage
                                [d-tmem], a-desc, b-desc , [sp-meta-tmem] ,
↪ idesc,
                                [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

tcgen05.mma.sp.cta_group.kind.block_scale{.scale_vectorsize}.collector_usage
                                [d-tmem], [a-tmem], b-desc , [sp-meta-tmem] ,
↪ idesc,
                                [scale-A-tmem], [scale-B-tmem], enable-input-
↪d;

.kind = { .kind::mxf8f6f4, .kind::mxf4, .kind::mxf4nvf4 }
.scale_vectorsize = { .scale_vec::1X, .scale_vec::2X, .scale_vec::4X, .block16, .
↪block32 }
.collector_usage = { .collector::buffer::op }
::buffer         = { ::a }
::op              = { ::fill, ::use, ::lastuse, ::discard* }

-----

// 5. Integer type:

tcgen05.mma.sp.cta_group.kind::i8 [d-tmem], a-desc, b-desc, [sp-meta-tmem] , idesc,
                                { disable-output-lane }, enable-input-d;

tcgen05.mma.sp.cta_group.kind::i8 [d-tmem], [a-tmem], b-desc, [sp-meta-tmem] , idesc,
                                { disable-output-lane }, enable-input-d;

.cta_group          = { .cta_group::1, .cta_group::2 }

-----

// 6. Convolution MMA with Integer type:

tcgen05.mma.sp.cta_group.kind::i8.collector_usage [d-tmem], a-desc, b-desc,
                                                        [sp-meta-tmem] , idesc,
                                                        { disable-output-lane },
↪enable-input-d;

```

(continues on next page)

(continued from previous page)

```

tcgen05.mma.sp.cta_group.kind::i8.ashift{.collector_usage} [d-tmem], [a-tmem], b-desc,
                                                         [sp-meta-tmem], idesc ,
                                                         { disable-output-lane },
↪enable-input-d;

tcgen05.mma.sp.cta_group.kind::i8{.ashift}.collector_usage [d-tmem], [a-tmem], b-desc,
                                                         [sp-meta-tmem], idesc ,
                                                         { disable-output-lane },
↪enable-input-d;

.collector_usage = { .collector::buffer::op }
::buffer         = { ::a }
::op              = { ::fill, ::use, ::lastuse, ::discard* }

```

Description

Instruction `tcgen05.mma.sp` is an asynchronous instruction which initiates an $M \times N \times K$ matrix multiply and accumulate operation of the form $D = A * B + D$ where the A matrix is $M \times (K/2)$, the B matrix is $K \times N$, and the D matrix is $M \times N$. *Sparse Matrices* describes the details of the sparsity.

The operation of the form $D = A * B$ is issued when the input predicate argument `enable-input-d` is false.

The optional immediate argument `scale-input-d` can be specified to scale the input matrix D as follows: $D = A * B + D * (2^{\text{scale-input-d}})$

The valid range of values for argument `scale-input-d` is [0, 15]. The argument `scale-input-d` is only valid for `.kind::tf32` and `.kind::f16`.

The 32-bit register operand `idesc` is the instruction descriptor as described in *Instruction descriptor*, specifies the shapes, exact types, sparsity and other details of the input matrices, output matrix and the matrix multiply and accumulate operation.

The qualifier `.cta_group::1` specifies that the matrix multiply and accumulate operation is performed on the *Tensor Memory* of the executing thread's CTA only. The qualifier `.cta_group::2` specifies that the matrix multiply and accumulate operation is performed on the *Tensor Memory* of the executing thread's CTA and its *peer CTA*.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The instruction `tcgen05.mma.sp` has single thread semantics, unlike the collective instructions `mma.sync` or `wgmma.mma_async`. So, a single thread issuing the `tcgen05.mma.sp` will result in the initiation of the whole matrix multiply and accumulate operation. Refer to the section *Issue Granularity*.

The qualifier `.kind` specifies the general kind of the element types of the multiplicand matrices. The exact types of the elements of the input and output matrices for each MMA-kind are specified in the *Instruction descriptor*.

The address operand `d-tmem` specifies the address of the destination and the accumulation matrix D in the *Tensor Memory*. The address operand `a-tmem` specifies the address of the matrix A in the *Tensor Memory*. The 64-bit register operand `a-desc` and `b-desc` are the matrix descriptors which represent the matrices A and B in shared memory respectively. The format of the matrix descriptor is described in *Matrix Descriptors*.

The vector operand `disable-output-lane` specifies the lane(s) in the *Tensor Memory* that should be not be updated with the resultant matrix D. Elements of the vector operand `disable-output-lane`

forms a mask where each bit corresponds to a lane of the *Tensor Memory*. with least significant bit of the first element of the vector (leftmost in syntax) corresponding to the lane 0 of the Tensor Memory. If a bit in the mask is 1, then the corresponding lane in the Tensor Memory for the resultant matrix D will not be updated. The size of the vector is as follows:

.cta_group	Size of the vector disable-output-lane
::1	4
::2	8

Qualifier `.block_scale` specifies that the matrices A and B are scaled with `scale_A` and `scale_B` matrices respectively before performing the matrix multiply and accumulate operation as specified in the section *Block Scaling*. The address operand `scale-A-tmem` and `scale-B-tmem` specify the base address the matrices `scale_A` and `scale_B` respectively in the *Tensor Memory*.

For qualifier `.scale_vectorsize`,

- If `.scale_vec::NX` is specified: N specifies the number of columns in `scale_A` matrix and number of rows in `scale_B` matrix.
- If `.blockN` is specified: N specifies the block size for which single scale factor will be applied. In this form, value of N is same as the K-dimension / (N of `.scale_vec::NX`).

Aliased `.scale_vectorsize` variants:

1. `.block16` is aliased with:
 1. `.scale_vec::4X` when `.kind = .kind::mxf4nvf4` and $K = 64$ or 128
2. `.block32` is aliased with:
 1. `.scale_vec::1X` when `.kind = .kind::mxf8f6f4` for all supported values of K
 2. `.scale_vec::2X` when `.kind = .kind::mxf4` or `.kind::mxf4nvf4` and $K = 64$ or 128

The valid combinations of MMA-kind and `.scale_vectorsize` are described in *Table 56*. For `.kind::mxf4` when the qualifier `.scale_vectorsize` is not specified, then it defaults to `.block32`. For `.kind::mxf4nvf4`, the qualifier `.scale_vectorsize` must be explicitly specified.

The qualifier `.ashift` shifts the rows of the A matrix down by one row, except for the last row in the *Tensor Memory*. Qualifier `.ashift` is only allowed with $M = 128$ or $M = 256$.

The qualifier `.collector_usage` specifies the usage of collector buffer for matrix A. Following collector buffer operations can be specified:

.collector_usage	Semantics
<code>.collector::a::fill</code>	Specifies that the A matrix read from the memory should be filled in collector buffer.
<code>.collector::a::use</code>	Specifies that the A matrix can be read from the collector buffer. This requires a previous fill to the collector buffer to be still valid.

continues on next page

Table 59 – continued from previous page

.collector_usage	Semantics
<code>.collector::a::lastuse</code>	Specifies that the A matrix can be read from the collector buffer and the contents of the collector buffer can be discarded. This requires a previous fill to the collector buffer to be valid till the collector buffer is read.
<code>.collector::a::discard</code>	Specifies that the contents of the collector buffer for A can be discarded.

If no `.collector_usage` qualifier is specified, then it defaults to `.collector::a::discard`. It is illegal to specify either of `.collector::a::use` or `.collector::a::fill` along with `.ashift`.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Qualifier `.kind::mxf4nvf4` introduced in PTX ISA version 8.7.

Qualifiers `.block16` and `.block32` introduced in PTX ISA version 8.8.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8 except `.kind::i8/.kind::mxf4nvf4/.kind::mxf4`:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.kind::i8` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_110a`

Qualifiers `.kind::mxf4nvf4` and `.kind::mxf4` are supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_103a`
- ▶ `sm_110a`

Argument `scale-input-d` requires `sm_100a` and is supported on `sm_100f` or higher in the same family from PTX ISA version 8.8.

For `.scale_vectorsize`,

- ▶ `.scale_vec::1X`, `.scale_vec::2X`, `.scale_vec::4X` requires `sm_100a`.
- ▶ `.block16`, `.block32` requires `sm_100f` or `sm_110f`.

For Target ISA details on matrix shape, check [Target ISA Note](#).

For Target ISA details on shared memory descriptor, check [Target ISA Note](#).

Examples

```
tcgen05.mma.sp.cta_group::1.kind::f16      [taddr0], adesc, bdesc, [tmem_spmeta0],
↪ idesc, p;

tcgen05.mma.sp.cta_group::1.kind::mxf8f6f4.collector::a:fill
                                         [taddr2], [taddr1], bdesc, [tmem_
↪ spmeta1], idesc,
                                         [tmem_scaleA], [tmem_scaleB], p;

tcgen05.commit.cta_group::1.mbarrier::arrive::one.b64 [mbarObj0];

loop:
mbarrier.try_wait.parity.b64 p, [mbarObj0], 0;
@!p bra loop;
```

9.7.16.10.9.3 TensorCore 5th Generation Instructions: `tcgen05.mma.ws`

`tcgen05.mma.ws`

Perform the 5th generation of weight stationary convolution matrix multiply and accumulate operation.

Syntax

```
// 1. Floating-point type without block scaling:

tcgen05.mma.ws.cta_group::1.kind{.collector_usage} [d-tmem], a-desc, b-desc,
↪ idesc,
enable-input-d {, zero-column-
↪ mask-desc };

tcgen05.mma.ws.cta_group::1.kind{.collector_usage} [d-tmem], [a-tmem], b-desc,
↪ idesc,
enable-input-d {, zero-column-
↪ mask-desc };

.kind = { .kind::f16, .kind::tf32, .kind::f8f6f4 }

-----

// 2. Integer type:
```

(continues on next page)

(continued from previous page)

```

tcgen05.mma.ws.cta_group::1.kind::i8{.collector_usage} [d-tmem], a-desc, b-desc,
↪ idesc,
                                                    enable-input-d {, zero-column-
↪ mask-desc};

tcgen05.mma.ws.cta_group::1.kind::i8{.collector_usage} [d-tmem], [a-tmem], b-desc,
↪ idesc,
                                                    enable-input-d {, zero-column-
↪ mask-desc};

.collector_usage = { .collector::buffer::op }
::buffer = { ::b0, ::b1, ::b2, ::b3 }
::op      = { ::fill, ::use, ::lastuse, ::discard}

```

Description

Instruction `tcgen05.mma.ws` is an asynchronous instruction which initiates an $M \times N \times K$ matrix multiply and accumulate operation, $D = A * B + D$ where the A matrix is $M \times K$, the B matrix is $K \times N$, and the D matrix is $M \times N$.

The operation of the form $D = A * B$ is issued when the input predicate argument `enable-input-d` is false.

The 32-bit register operand `idesc` is the instruction descriptor as described in *Instruction descriptor*, specifies the shapes, exact types, sparsity and other details of the input matrices, output matrix and the matrix multiply and accumulate operation.

The qualifier `.cta_group::1` specifies that the matrix multiply and accumulate operation is performed on the *Tensor Memory* of the executing thread's CTA only.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The instruction `tcgen05.mma.ws` has single thread semantics, unlike the collective instructions `mma.sync` or `wgmma.mma_async`. So, a single thread issuing the `tcgen05.mma.ws` will result in the initiation of the whole matrix multiply and accumulate operation. Refer to the section *Issue Granularity*.

The qualifier `.kind` specifies the general kind of the element types of the multiplicand matrices. The exact types of the elements of the input and output matrices for each MMA-kind are specified in the *Instruction descriptor*.

The address operand `d-tmem` specifies the address of the destination and the accumulation matrix D in the *Tensor Memory*. The address operand `a-tmem` specifies the address of the matrix A in the *Tensor Memory*. The 64-bit register operand `a-desc` and `b-desc` are the matrix descriptors which represent the matrices A and B in shared memory respectively. The format of the matrix descriptor is described in *Matrix Descriptors*.

The optional operand `zero-column-mask-desc` is a 64-bit register which specifies the *Zero-Column Mask Descriptor*. The zero-column mask descriptor is used to generate a mask that specifies which columns of B matrix will have zero value for the matrix multiply and accumulate operation regardless of the values present in the shared memory.

The qualifier `.collector_usage` specifies the usage of collector buffer for Matrix B. Following collector buffer operations can be specified:

.collector_usage	Semantics
<code>.collector::bN::fill</code>	Specifies that the B matrix read from the memory should be filled in collector buffer #N.
<code>.collector::bN::use</code>	Specifies that the B matrix can be read from the collector buffer #N. This requires a previous fill to the collector buffer #N to be still valid.
<code>.collector::bN::lastuse</code>	Specifies that the B matrix can be read from the collector buffer #N after which the contents of the collector buffer #N can be discarded. This requires a previous fill to the collector buffer #N to be valid till the collector buffer #N is read.
<code>.collector::bN::discard</code>	Specifies that the contents of the collector buffer #N can be discarded.

If no `.collector_usage` qualifier is specified, then it defaults to `.collector::b0::discard`.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8 except `.kind::i8`:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.kind::i8` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_110a`

Examples

```
tcgen05.mma.ws.cta_group::1.kind::i8.collector::b2:use [taddr2], [taddr1], bdesc,
↳ idesc, p;
tcgen05.commit.cta_group::1.mbarrier::arrive::one.b64 [mbarObj0];

loop:
mbarrier.try_wait.parity.b64 p, [mbarObj0], 0;
@!p bra loop;
```

9.7.16.10.9.4 TensorCore 5th Generation Instructions: `tcgen05.mma.ws.sp`

`tcgen05.mma.ws.sp`

Perform the 5th generation of weight stationary convolution matrix multiply and accumulate operation with sparse A matrix.

Syntax

```
// 1. Floating-point type without block scaling:

tcgen05.mma.ws.sp.cta_group::1.kind{.collector_usage} [d-tmem], a-desc, b-desc,
                                                         [sp-meta-tmem] , idesc,
                                                         enable-input-d {, zero-column-
↳ mask-desc};

tcgen05.mma.ws.sp.cta_group::1.kind{.collector_usage} [d-tmem], [a-tmem], b-desc,
                                                         [sp-meta-tmem] , idesc,
                                                         enable-input-d {, zero-column-
↳ mask-desc};

.kind = { .kind::f16, .kind::tf32, .kind::f8f6f4 }

-----

// 2. Integer type:

tcgen05.mma.ws.sp.cta_group::1.kind::i8{.collector_usage} [d-tmem], a-desc, b-desc,
                                                         [sp-meta-tmem] , idesc,
                                                         enable-input-d {, zero-
↳ column-mask-desc};

tcgen05.mma.ws.sp.cta_group::1.kind::i8{.collector_usage} [d-tmem], [a-tmem], b-desc,
                                                         [sp-meta-tmem] , idesc,
                                                         enable-input-d {, zero-
↳ column-mask-desc};

.collector_usage = { .collector::buffer::op }
::buffer = { ::b0, ::b1, ::b2, ::b3 }
::op      = { ::fill, ::use, ::lastuse, ::discard}
```

Description

Instruction `tcgen05.mma.ws.sp` is an asynchronous instruction which initiates an $M \times N \times K$ matrix multiply and accumulate operation, $D = A * B + D$ where the A matrix is $M \times (K/2)$, the B matrix is $K \times N$, and the D matrix is $M \times N$. *Sparse Matrices* describes the details of the sparsity.

The operation of the form $D = A * B$ is issued when the input predicate argument `enable-input-d` is false.

The 32-bit register operand `idesc` is the instruction descriptor as described in *Instruction descriptor*, specifies the shapes, exact types, sparsity and other details of the input matrices, output matrix and the matrix multiply and accumulate operation.

The qualifier `.cta_group::1` specifies that the matrix multiply and accumulate operation is performed on the Tensor Memory of the executing thread's CTA only.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The instruction `tcgen05.mma.ws.sp` has single thread semantics, unlike the collective instructions `mma.sync` or `wgmma.mma_async`. So, a single thread issuing the `tcgen05.mma.ws.sp` will result in the initiation of the whole matrix multiply and accumulate operation. Refer to the section *Issue Granularity*.

The qualifier `.kind` specifies the general kind of the element types of the multiplicand matrices. The exact types of the elements of the input and output matrices for each MMA-kind are specified in the *Instruction descriptor*.

The address operand `d-tmem` specifies the address of the destination and the accumulation matrix D in the *Tensor Memory*. The address operand `a-tmem` specifies the address of the matrix A in the *Tensor Memory*. The 64-bit register operand `a-desc` and `b-desc` are the matrix descriptors which represent the matrices A and B in shared memory respectively. The format of the matrix descriptor is described in *Matrix Descriptors*.

The optional operand `zero-column-mask-desc` is a 64-bit register which specifies the *Zero-Column Mask Descriptor*. The zero-column mask descriptor is used to generate a mask that specifies which columns of B matrix will have zero value for the matrix multiply and accumulate operation regardless of the values present in the shared memory.

The qualifier `.collector_usage` specifies the usage of collector buffer for Matrix B. Following collector buffer operations can be specified:

<code>.collector_usage</code>	Semantics
<code>.collector::bN::fill</code>	Specifies that the B matrix read from the memory should be filled in collector buffer #N.
<code>.collector::bN::use</code>	Specifies that the B matrix can be read from the collector buffer #N. This requires a previous fill to the collector buffer #N to be still valid.

continues on next page

Table 61 – continued from previous page

.collector_usage	Semantics
<code>.collector::bN::lastuse</code>	Specifies that the B matrix can be read from the collector buffer #N after which the contents of the collector buffer #N can be discarded. This requires a previous fill to the collector buffer #N to be valid till the collector buffer #N is read.
<code>.collector::bN::discard</code>	Specifies that the contents of the collector buffer #N can be discarded.

If no `.collector_usage` qualifier is specified, then it defaults to `.collector::b0::discard`.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8 except `.kind::i8`:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Qualifier `.kind::i8` is supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ `sm_110a`

Examples

```
tcgen05.mma.ws.sp.cta_group::1.kind::tf32.collector::b1::fill [taddr1], [taddr0],
↪ bdesc,
                                                                    [tmem_spmeta0], idesc,
↪ p;
tcgen05.commit.cta_group::1.mbarrier::arrive::one.b64 [mbarObj0];
loop:
```

(continues on next page)

(continued from previous page)

```
mbarrier.try_wait.parity.b64 p, [mbarObj0], 0;  
@!p bra loop;
```

9.7.16.11 TensorCore 5th Generation Specialized Synchronization Operations

9.7.16.11.1 TensorCore 5th Generation Instructions: `tcgen05.fence`

`tcgen05.fence`

Specialized fence for the asynchronous `tcgen05` operations.

Syntax

```
tcgen05.fence::before_thread_sync ;  
tcgen05.fence::after_thread_sync ;
```

Description

The instruction `tcgen05.fence::before_thread_sync` orders all the prior asynchronous `tcgen05` operations with respect to the subsequent `tcgen05` and the execution ordering operations.

The instruction `tcgen05.fence::after_thread_sync` orders all the subsequent asynchronous `tcgen05` operations with respect to the prior `tcgen05` and the execution ordering operations.

The `tcgen05.fence::*` instructions compose with execution ordering instructions across a thread scope and provide ordering between `tcgen05` instructions across the same scope.

The `tcgen05.fence::before_thread_sync` instructions behave as code motion fence for prior `tcgen05` instructions as they cannot be hoisted across. The `tcgen05.fence::after_thread_sync` instructions behave as code motion fence for subsequent `tcgen05` instructions as they cannot be hoisted across.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```
// Producer thread:

tcgen05.cp.cta_group::1.128x256b [taddr0], sdesc0;

tcgen05.fence::before_thread_sync;
st.relaxed.b32 [flag], 1;

// Consumer thread:

loop:
ld.relaxed.b32 r, [flag];
setp.eq.u32 p, r, 1;
@!p bra loop;

tcgen05.fence::after_thread_sync;
tcgen05.mma.cta_group.kind [taddr0], adesc, bdesc, idesc, p;
```

9.7.16.12 TensorCore 5th Generation Async Synchronization Operations

9.7.16.12.1 TensorCore 5th Generation Instructions: `tcgen05.commit`

`tcgen05.commit`

Makes the mbarrier object track the completion of all prior async-tcgen05 operations initiated by the executing thread.

Syntax

```
tcgen05.commit.cta_group.completion_mechanism{.shared::cluster}{.multicast}.b64
                                                [mbar] {, ctaMask};

.completion_mechanism = { .mbarrier::arrive::one }
.cta_group             = { .cta_group::1, .cta_group::2 }
.multicast             = { .multicast::cluster }
```

Description

The instruction `tcgen05.commit` is an asynchronous instruction which makes the mbarrier object, specified by the address operand `mbar`, track the completion of all the prior asynchronous `tcgen05` operations, as listed in *mbarrier based completion mechanism*, initiated by the executing thread. Upon the completion of the tracked asynchronous `tcgen05` operations, the signal specified by the `.completion_mechanism` is triggered by the system on the mbarrier object.

The instruction `tcgen05.commit.cta_group::1` tracks for the completion of all prior asynchronous `tcgen05` operations with `.cta_group::1` issued by the current thread. Similarly, the instruction `tcgen05.commit.cta_group::2` tracks for the completion of all prior asynchronous `tcgen05` operations with `.cta_group::2` issued by the current thread.

All `tcgen05` instructions within a kernel must specify the same value for the `.cta_group` qualifier.

The qualifier `.mbarrier::arrive::one` indicates that upon the completion of the prior asynchronous `tcgen05` operation issued by the current thread, an arrive-on operation, with the count argument of 1, is signaled on the mbarrier object. The scope of the arrive-on operation is the cluster scope.

The optional qualifier `.multicast::cluster` allows signaling on the mbarrier objects of multiple CTAs in the cluster. Operand `ctaMask` specifies the CTAs in the cluster such that each bit position in the 16-bit `ctaMask` operand corresponds to the `%cluster_ctarank` of the destination CTA. The mbarrier signal is multicast to the same offset as `mbar` in the shared memory of each destination CTA.

If no state space is specified then *Generic Addressing* is used. If the address specified by `mbar` does not fall within the address window of `.shared::cluster` state space then the behavior is undefined.

PTX ISA Notes

Introduced in PTX ISA version 8.6.

Target ISA Notes

Supported on following architectures:

- ▶ `sm_100a`
- ▶ `sm_101a` (Renamed to `sm_110a` from PTX ISA version 9.0)
- ▶ And is supported on following family-specific architectures from PTX ISA version 8.8:
 - ▶ `sm_100f` or higher in the same family
 - ▶ `sm_101f` or higher in the same family (Renamed to `sm_110f` from PTX ISA version 9.0)
- ▶ `sm_110f` or higher in the same family

Examples

```
Example 1:
tcgen05.cp.cta_group::1.128x256b          [taddr0], sdesc0;
tcgen05.commit.cta_group::1.mbarrier::arrive::one.b64 [mbarObj1];

loop:
mbarrier.try_wait.parity.b64 p, [mbarObj1], 0;
@!p bra loop;

Example 2:
tcgen05.mma.cta_group::2.kind::tf32      [taddr0], adesc, bdesc, idesc, p;
tcgen05.commit.cta_group::2.mbarrier::arrive::one.b64 [mbarObj2];

loop:
mbarrier.try_wait.parity.b64 p, [mbarObj2], 0;
@!p bra loop;
```